Heidelberg University

Department of Physics and Astronomy

Lecture Notes

# Fundamentals of Numerical Programming I

*Leonard Storcks*

December 20, 2025

**Abstract**

For numerous mathematical and scientific problems, from integration to optimization and data analysis, analytical approaches carried out by hand are insufficient. Numerical methods can open new viewing angles and solution strategies to these problems. In this course we cover some *fundamentals of numerical programming* including numerical solutions to linear systems, numerical derivatives and interpolation, optimization and numerical integration.

# Contents

# 1 Introduction

From the Gaussian integral $\int_a^b \exp(-x^2)\,dx$ and the motion of three bodies to fluid dynamics and weather prediction, many problems from mathematics, science and engineering don't have *nice analytical solutions*, i.e. compact expressions comprised of elementary functions.

The ability of computers to rapidly perform simple arithmetic operations opens new angles of attack for such problems.[1] Let us approximate the Gaussian integral only using elementary arithmetic operations. We can approximate the exponential function with a truncated power series

$$\exp(x) = \sum_{k=0}^{\infty} \frac{x^k}{k!} \approx \sum_{k=0}^{N_{\text{order}}} \frac{x^k}{k!} \tag{1}$$

and the integral with

$$\int_a^b f(x)\,dx \approx \sum_{i=1}^{N_{\text{eval}}} f(x_i)\Delta x, \quad \Delta x = \frac{b-a}{N_{\text{eval}}}, \quad x_i = a + i\,\Delta x \tag{2}$$

The approximation of the integral is illustrated in Fig. 1, `Python` code on the example is given in Code-Snipped 1.

**Question to the reader:** How can we control the accuracy of our approximation to $\int_a^b \exp(-x^2)\,dx$?

However, numerical algorithms and computation are not without caveats. For example, the precision with which we can handle real numbers is limited by storage and compute. This has very profound implications all through numerics, e.g. bounds derived under exact arithmetic might not hold under approximate arithmetic on the computer.

Our plan for the course is as follows:

1. we will introduce how numbers are represented digitally and raise awareness for the finite precision of floating point operations
2. we will discuss how linear systems can be solved numerically
3. we will introduce finite difference approximations to derivatives, discuss truncation

[1] For reference, the ENIAC computer developed in the Second World War could perform around 500 simple operations per second while a modern GPU can perform more than $10^{12}$ of such *floating-point operations* per second.

```python
import math
import numpy as np

def exp_series(x, n_terms=20):
    """
    Approximate exp(x) by a truncated series.
    """
    result = 1.0
    term = 1.0

    for k in range(1, n_terms):
        term = term * x / k
        result = result + term

    return result

def integrate(func, a, b, n_points=10_000):
    """
    Simple integral approximation, function
    evaluated at left interval edges.
    """

    dx = (b - a) / n_points
    x = a + dx * np.arange(n_points)

    return np.sum(func(x) * dx)

# approximate the integral of exp(-x^2)
# from a to b
func = lambda x: exp_series(-x ** 2)
a, b = 0.0, 1.0
approx = integrate(func, a, b)
```

Code-Snippet 1: Python code for approximating $\int_a^b \exp\left(-x^2\right) dx$.

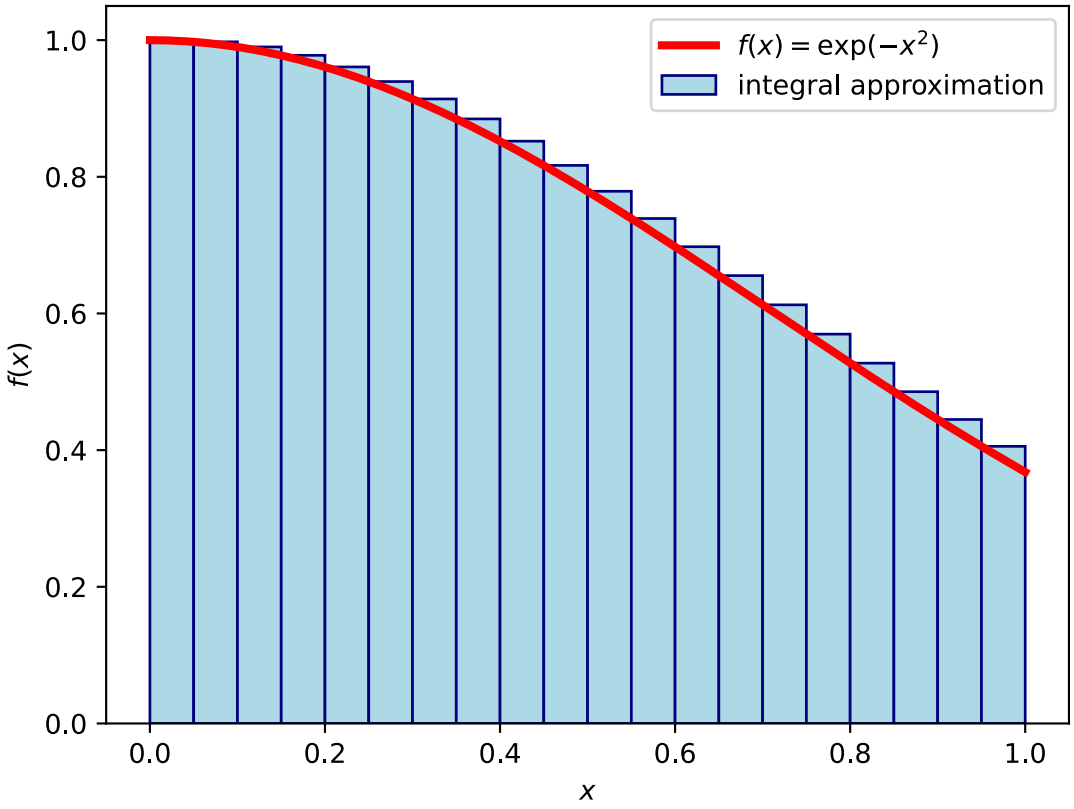



Figure 1: Approximation of the integral $\int_a^b \exp(-x^2)\,dx$.

errors, hint at powerful applications of finite differencing to differential equations and will give an outlook to automatic differentiation, the backbone of machine learning

4. we will discuss numerical solutions to non-linear equations and optimization problems
5. we will go through methods for numerical integration

The methods you will learn in this course can empower you to solve a wide range of problems - and we're excited to see what you will use them on :)

# 2 Digital Representation of Numbers

In C, `int x = 100 * 200 * 300 * 400;` will surprisingly yield $-1894967296$ (for a 32-bit integer representation) (*overflow*), `float f1 = (2.3 + 1e20) - 1e20;` yields 0.0 (*round-off-error*) but `float f2 = 2.3 + (1e20 - 1e20);` correctly gives 2.3.

We want to understand and mitigate such caveats of arithmetic on computers, where we want quick calculations while also using as little storage as possible - simulations can quickly become big in storage (e.g. lots of particles).

Further details can be found in Higham, 2002 and Bryant and O'Hallaron, 2011.

## 2.1 Integer Arithmetic

In C, integers ($\in \mathbb{Z}$) are stored as fixed-size bit-sequences. The respective size dictates a range. C provides both unsigned and signed integer types, with negative numbers in the signed case represented using *two's-complements*.

### 2.1.1 Unsigned integers

For a bit-vector $\underline{x} = [x_{\omega-1}, x_{\omega-2}, \ldots, x_0]$ the unsigned conversion is

$$B2U_\omega := \sum_{i=0}^{\omega-1} x_i 2^i \tag{3}$$

(illustrated in figure 2) and the range is $0, \ldots, 2^\omega - 1$. In case of overflow in operations, the overflow is truncated in the most significant bits (see figure 3). As $B2U_\omega[x_{\omega-1}, x_{\omega-2}, \ldots, x_0] \mod 2^k = B2U_\omega[x_{k-1}, x_{k-2}, \ldots, x_0]$ we effectively store arithmetic result $\mod 2^\omega$.

$x_{\omega-1}$ is called most significant bit (MSB) and $x_0$ least significant bit (LSB).

**Problem:** When the result of an arithmetic operation exceeds the range of an integer type, unexpected results occur (overflow) (and also underflow in the signed case)

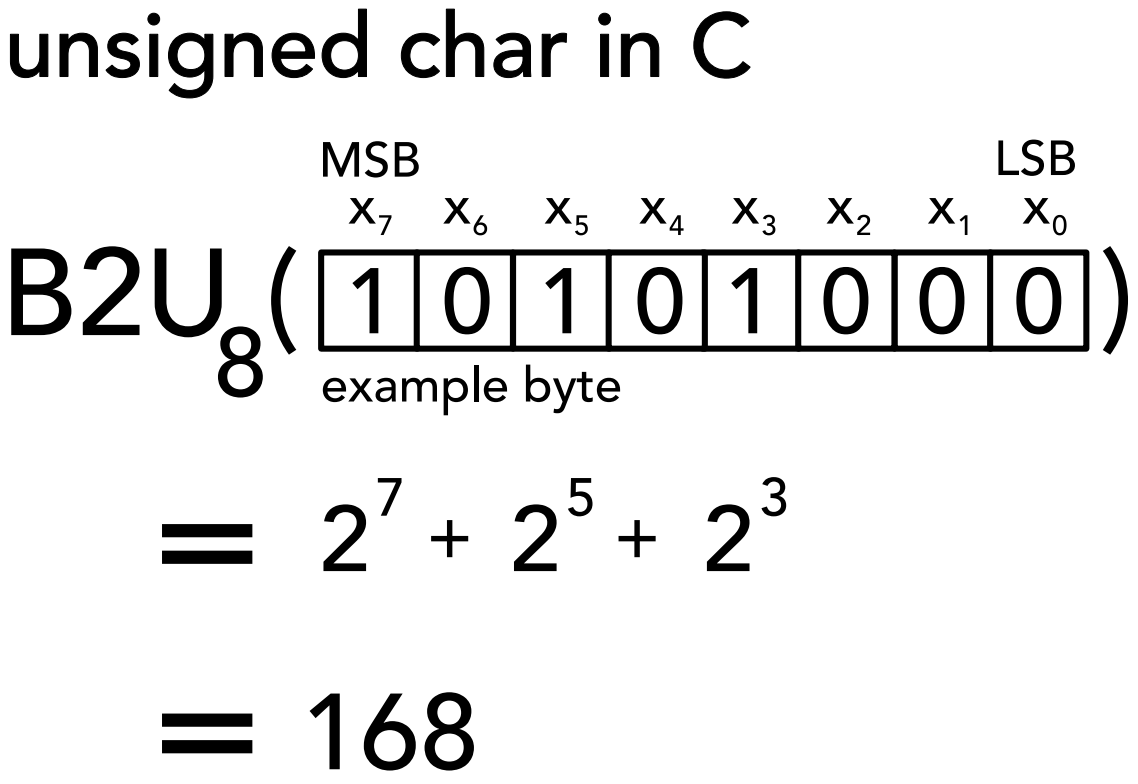



Figure 2: Example of an unsigned char in C.

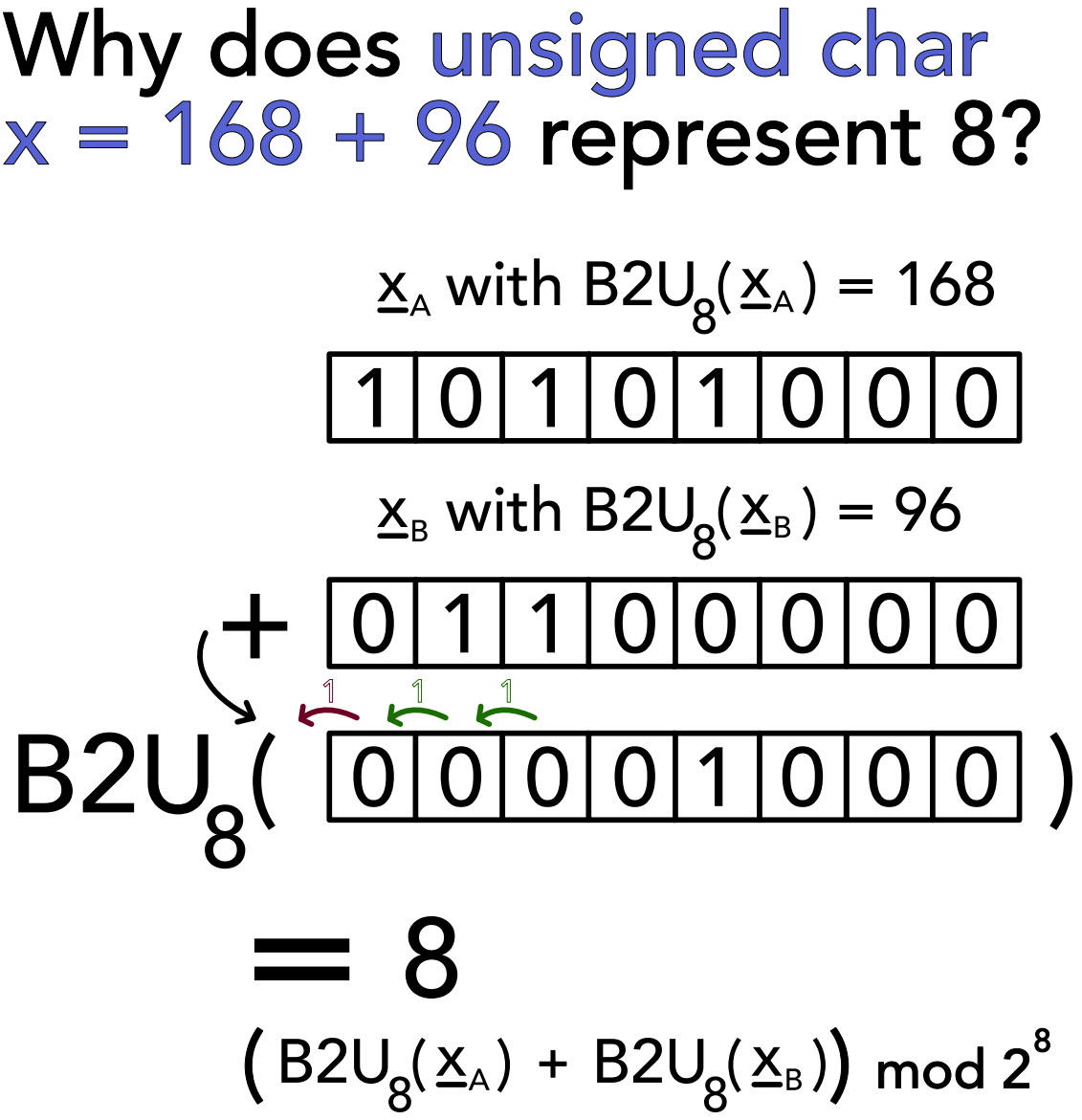



Figure 3: Example of unsigned char overflow.

### 2.1.2 Two's complement for negative numbers

Note that addition of integers by bitwise addition with carry-on is very fast.

**Why can't we just let the MSB encode the sign?:** A representation of the form

$$B2S_\omega(\underline{x}) := (-1)^{x_{\omega-1}} \cdot \sum_{i=0}^{\omega-2} x_i 2^i \tag{4}$$

(sign magnitude) has the disadvantage, that normal bitwise addition with carry-on does not work. Also, zero is encoded twice, as $\pm 0$.

**Idea of the two's-complement:** The MSB flags the sign in $\underline{x} = [x_{\omega-1}, x_{\omega-2}, \ldots, x_0]$ by having a weighting factor $-2^{\omega-1}$

$$B2T_\omega(\underline{x}) := -x_{\omega-1}2^{\omega-1} + \sum_{i=0}^{\omega-2} x_i 2^i \tag{5}$$

Carry-on to the $\omega$-th bit in addition is again ignored. As we really just have to add positive numbers in bits $x_0$ to $x_{\omega-2}$ with correct sign-switch by carry-on, no special handling is necessary (see figure 4). The range is $-2^{\omega-1} \ldots 2^{\omega-1} - 1$.

From an unsigned int to the two's complement and vice versa we can get by

1. invert all bits
2. add $+1$ to the result[2]

If we want $\underline{x}$ with $B2T_\omega(\underline{x}) = -u$ then the bits following the MSB must encode $k$ with $-u = -2^{\omega-1} + k$, so the encoded unsigned number must be $B2T_\omega(\underline{x}) = \underbrace{2^{\omega-1}}_{\text{sign bit}} + k = 2^\omega - u$ - a *two's complement.*

[2]These rules intuitively follow from the constraint, that bitwise addition with carry-on of $-u$ and $u$ should result in an all-zero bitvector. Adding a bitvector to its inverted self results in an all-1 bitvector, adding one more then results in all zeros, as the last carry-on is discarded.

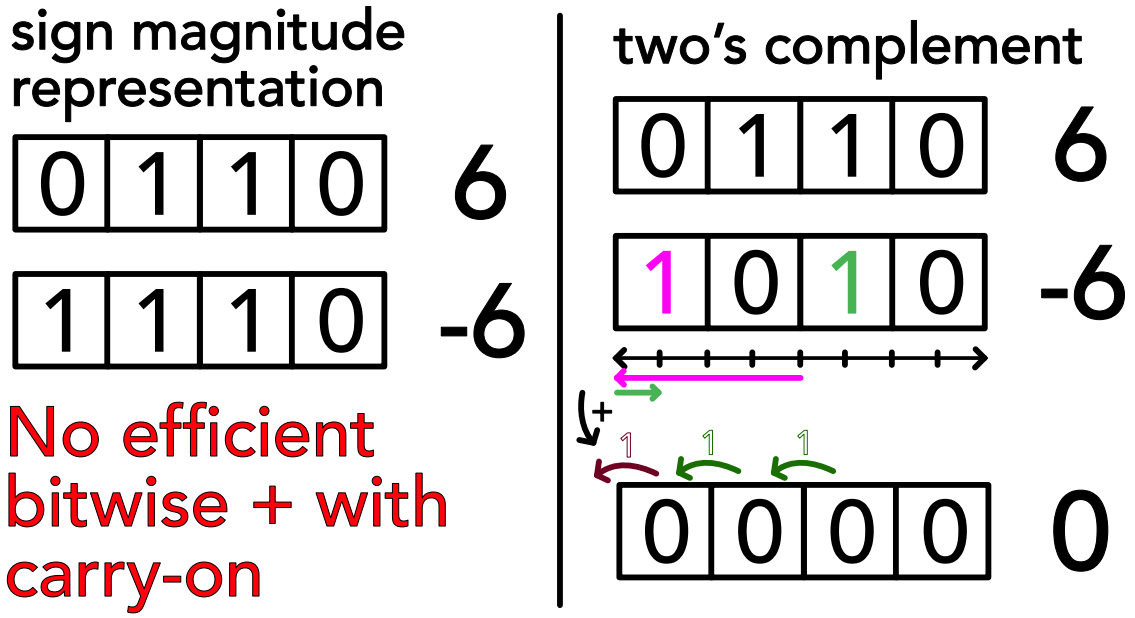



Figure 4: Illustration of the defining feature of the two's complement - addition is simple.

### 2.1.3 Integer types in C

Some common integer types with respective minimum sizes are given in table 1.

| Type | Minimum Size $\omega$ |
|---|---|
| `char` | 8 bits |
| `short` | 16 bits |
| `int` | 16 bits |
| `long` | 32 bits |
| `long long` | 64 bits |

Table 1: Common integer types in C, signed ranges are $-2^{\omega-1} \dots 2^{\omega-1}-1$, unsigned ranges are $0 \dots 2^{\omega}-1$.

**Problem:** Using unsigned can come with unexpected results, when being cast from a negative number. E.g. `unsigned int x = -1;` will result in $2^{32}-1$ in a 32-bit system (the two's complement is read as if an unsigned representation). More deviously, $-1 < 0U$ will evaluate to false, as all integers in a comparison are cast to unsigned, when one of them is unsigned.

### 2.1.4 Byte Ordering in Storage: Big and Little Endian

Bytes of e.g. a multi-byte integer can be stored from most significant byte to least significant byte (big endian) or vice versa (little endian), see figure 5.

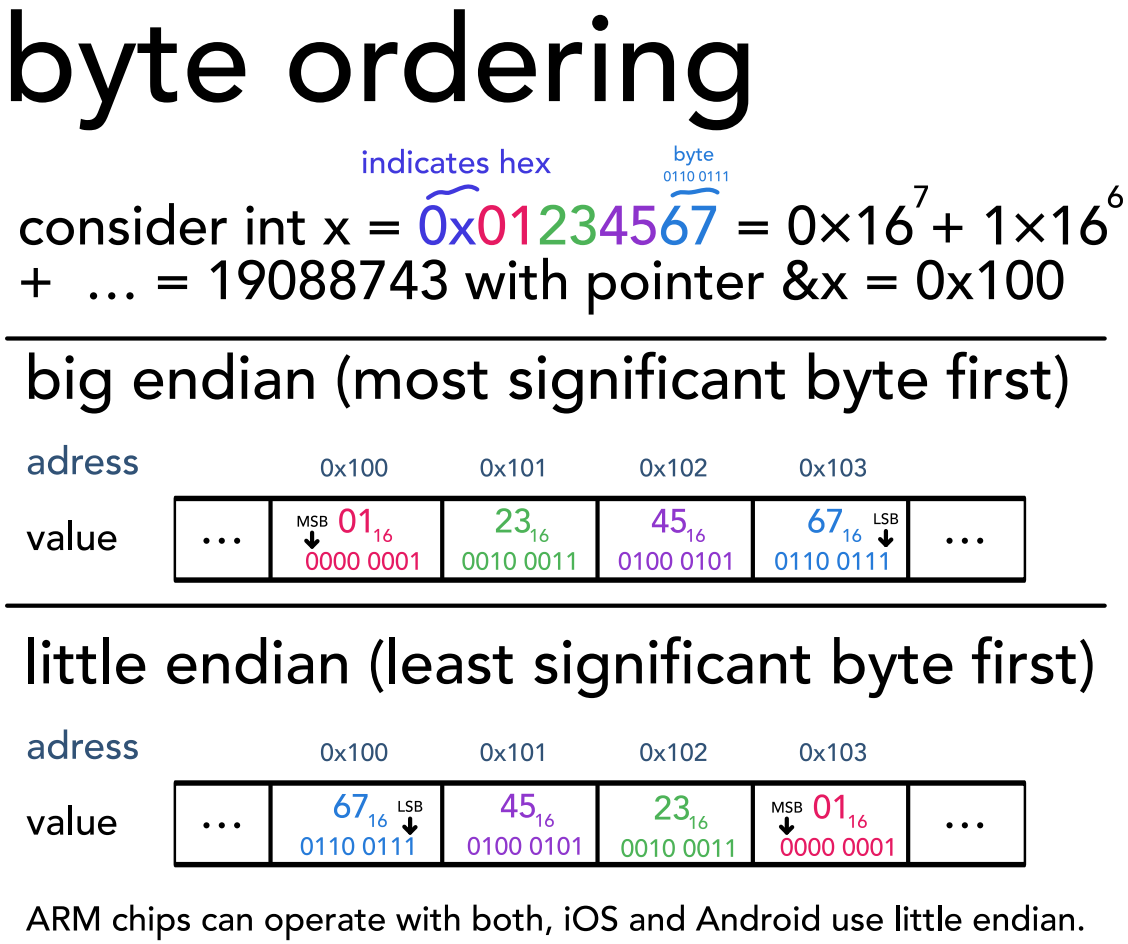



Figure 5: Big and Little Endian.

### 2.1.5 Properties and Caveats of Integer Arithmetic

Typical problems in integer arithmetic are overflow, integer division, the modulo operation and implicit type conversion.

**Overflow:** The respective integer ranges have finite ranges, overflow in arithmetic operations is cut-off. We must choose a type with sufficient range - mind that choosing types with too large of a footprint (e.g. always long) wastes storage and compute.

*Example I:* In `char c = 100 * 4; // range -128 to 127` the result is $-112$ as 400 in bits is 000110010000 where a cut-off to one byte means 10010000 so $-2^7+2^4 = -128+16 = -112$.

*Example II:* For `int a = -1 * pow(2,31); int b = 10;` we have $a < b$ but not $a - b < 0$ (for char the behavior is a bit different as up to int there is implicit type conversion.)

**Integer division:** All decimal places are truncated, so

$$5/3 = 1; -5/3 = -1; 5/-3 = -1$$

**Modulo operation:** The modulus in C is defined as $n\%m = n - (n/m)m$ so

$$5\%3 = 2; -5\%3 = -2; 5\% - 3 = -2$$

**Implicit type conversion:** In C, the type can implicitly be converted up to unsigned int, which can avoid overflow, as illustrated in table 2.

| implicit type conversion up to int can be helpful | mind its only up to int | explicit type consversion |
|---|---|---|
| `char a,b;`<br>`a = 100;`<br>`b = 4;`<br>`int c = a * b; //`<br>`↪ 400` | `int a = 2e9;`<br>`int b = 3;`<br>`long long c = a *`<br>`↪ b;`<br>`printf("%llu\n",`<br>`↪ c); //`<br>`↪ 1705032704`<br>As $2^{32} - 1 \approx 4 \cdot 10^9 < 6 \cdot 10^9 < 2^{33} - 1$ (unsigned ranges) we overflow to the 33rd-bit, which is cut-off, giving the unsigned result of $6 \cdot 10^9 - 2^{32} = 1705032704$. | `int a = 2e9;`<br>`int b = 3;`<br>`long long c =`<br>`↪ ((long long) a)`<br>`↪ * ((long long)`<br>`↪ b);`<br>`printf("%llu\n",`<br>`↪ c); //`<br>`↪ 6000000000` |

Table 2: Type conversion and its caveats

### 2.1.6 Can there be integer-overflow in python?

Note that in python3, integers are implemented as "long" integer objects of arbitrary size, overflows are impossible (at the cost of speed; mind that e.g. numpy is based on C code).

## 2.2 Floating Point Arithmetic

In the following, we will encode rational numbers in the form $V = x \cdot 2^y$, with $x, y \in \mathbb{Z}$. Similar to the decimal notation

$$d_m d_{m-1} \dots d_0 . d_{-1} d_{-2} \dots d_{-p} = \sum_{i=-p}^{m} d_i 10^i \tag{6}$$

we can write in binary notation

$$b_m b_{m-1} \dots b_0 . b_{-1} b_{-2} \dots b_{-p} = \sum_{i=-p}^{m} b_i 2^i, \quad 0.01_2 = 0.25_{10} \tag{7}$$

or generally in base $\beta$ in scientific notation

$$\begin{gathered}(-1)^s \cdot \underbrace{b_0 . b_{-1} b_{-2} \dots b_{-p}}_{\text{mantissa M}} \cdot \beta^e = \beta^e \cdot \sum_{i=-p}^{0} b_i \beta^i, \quad \text{exponent } e, \\ \text{precision with implicit first bit } p+1, \quad \text{sign-bit } s \in \{0,1\}\end{gathered} \tag{8}$$

Note that in this notation (in the form $V = x \cdot 2^y$) we cannot exactly represent e.g. 0.1 or 0.2.

$$0.1_{10} = 1.10011[0011] \dots_2 \cdot 2^{-4} \tag{9}$$

**Disastrous historic example:** In the first Gulf War (more specifically on 25th February 1991), a Patriot missile defense battery failed to intercept an incoming Iraqi Scud missile, because it used an internal clock counting up in tenths of a second represented by a 23-bit sequence. Future missile positions are predicted by extrapolating from past position with constant velocity. The Patriot system mixed both this inaccurate internal clock and a more accurate one, leading to the failed interception and 28 deaths among soldiers.

### 2.2.1 IEEE 754 Floating Point Standard

Based on the representation in scientific notation, we can store a floating point number in a bit-vector with

- a sign bit $s$
- an exponent $e$ stored as an unsigned integer $E = e + b$ with bias $b$
- a mantissa $M$

where for single precision (32-bit)

- the exponent is stored in 8 bits, $b = 127$, $e_{min} = -126$, $e_{max} = 127$ with $E = 255$ and $E = 0$ reserved for special cases
- the mantissa is stored in 23 bits, with the first bit being implicitly 1 (**normalization**), which can always be assumed by appropriately choosing the exponent (floating point representations are not unique), so we have $p = 23(+1)$ bits of precision encoding an integer $M$ but need a special representation for 0

where based on the exponent we differentiate between

- **normalized values for** $1 \leq E \leq 254$ with value of the floating point number

$$V = (-1)^s \cdot \left(1 + \frac{M}{2^p}\right) \cdot 2^{E-b}, \quad \text{sign bit } s$$
$$\text{biased exponent } E = e + b, \quad \text{integer representation of mantissa } M \tag{10}$$
$$\text{precision } p = \#\text{mantissa bits, not including the 1 implicit bit}$$

- **denormalized values for** $E = 0$ with value of the floating point number

$$V = (-1)^s \cdot \frac{M}{2^p} \cdot 2^{-b+1}, \quad \text{for } M = 0, V = \pm 0 \text{ depending on } s \tag{11}$$

  The denormalized numbers start just below the normalized ones (by the factor $2^{-b+1}$ with $+1$ as we do not have the implicit leading 1 here) and now no normalization (implicit starting 1-bit) is assumed. This allows to approach zero with gradually decreasing precision (and even spacing) (smaller numbers occupy fewer digits as of the leading zeros) and ensures that for $x \neq y$, $x - y$ is non-zero, so $\frac{1}{x-y}$ is safe for $x \neq y$.

- $\pm\infty$ **for** $E = 255$ **and** $M = 0$
- **NaN (Not a Number) for** $E = 255$ **and** $M \neq 0$

**Why is the exponent stored in a biased way, not two's complement?:** In a two's complement representation of the exponent to compare two numbers, we have to compare the exponents and mantissas separately and the exponent-comparison is a bit more complicated than comparing biased representations. In the biased exponent representation we can just compare the bitvectors of exponent followed by mantissa interpreted as integers (also mind the sign).

The cases are illustrated in figure 6, with a specific example in figure 7.

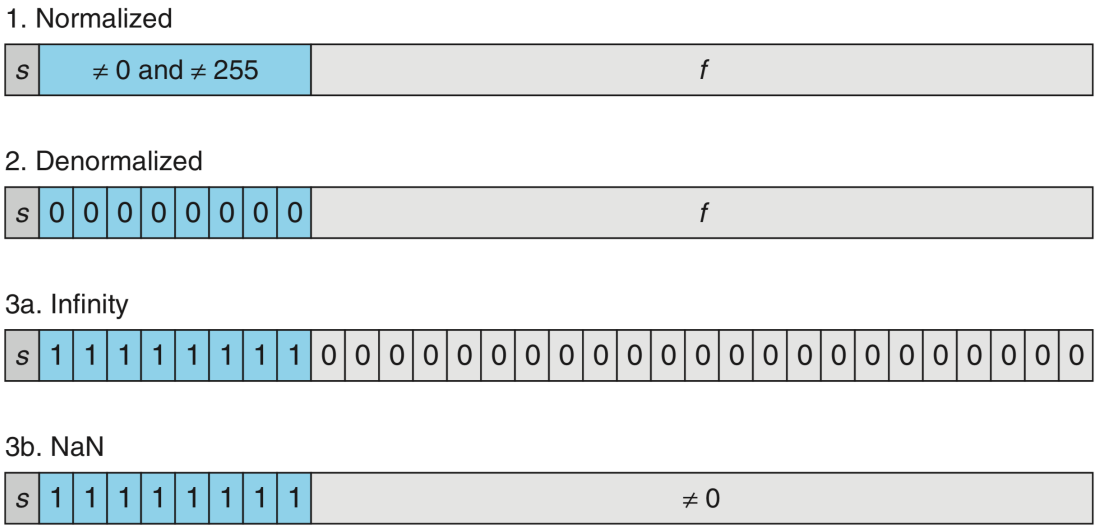



Figure 6: Cases of the IEEE 754 Floating Point Standard.

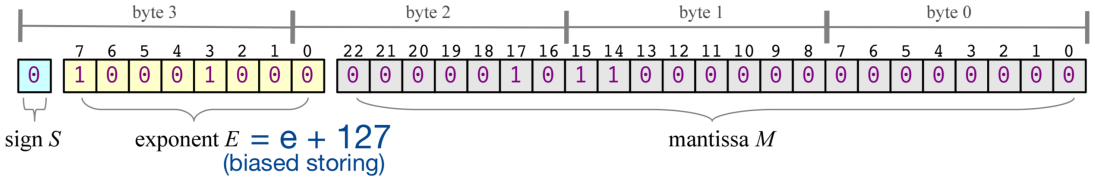



Figure 7: 523 can be written as $1.000001011_2 \cdot 2^9$ so $e = 9, E = e + 127 = 136$. As of the normalization, the leading 1 in the mantissa is implicit. The number is given as a single precision float in big endian.

### 2.2.2 Only a finite set of floating point numbers can be represented exactly

With 32 bits, $2^{32}$ states can be encoded (and mind that here e.g. NaN has multiple representations). In any case, the number of exactly representable numbers is finite, above 0 starting at

$$V_{\text{denorm,min}} = \frac{1}{2^p} \cdot 2^{-b+1} \underbrace{\approx}_{\substack{\text{single} \\ \text{p.}}} 1.4 \cdot 10^{-45} \tag{12}$$

with the smallest normalized number being

$$V_{\text{norm,min}} = (1+0) \cdot 2^{-b} \underbrace{\approx}_{\substack{\text{single} \\ \text{p.}}} 1.2 \cdot 10^{-38} \tag{13}$$

and the largest normalized number being

$$V_{\text{norm,max}} = (1 + \frac{2^p - 1}{2^p}) \cdot 2^{E_{\max} - b} \underbrace{\approx}_{\substack{\text{single} \\ \text{p.}}} 3.4 \cdot 10^{38} \tag{14}$$

### 2.2.3 Machine Precision is finite

The smallest increment in the mantissa, in $1 + \frac{M}{2^p}$, is the **machine precision**

$$\epsilon_{\text{mach}} = \frac{1}{2^p} \underbrace{\approx}_{\substack{\text{single} \\ \text{p.}}} 1.2 \cdot 10^{-7} \tag{15}$$

Consider two floating point numbers $V_1, V_2 > 0$ *next to each other*

$$V_1 = \left(1 + \frac{M}{2^p}\right) \cdot 2^{E-b}, \quad V_2 = \left(1 + \frac{M+1}{2^p}\right) \cdot 2^{E-b} \tag{16}$$

so their relative difference is bound by

$$\frac{V_2 - V_1}{V_2} = \frac{\frac{1}{2^p} \cdot 2^{E-b}}{\left(1 + \frac{M+1}{2^p}\right) \cdot 2^{E-b}} \leq \epsilon_{\text{mach}} \tag{17}$$

### 2.2.4 Rounding and Pitfalls of Floating Point Arithmetic

In the IEEE standard, results of addition, subtraction, multiplication and division must equal to one where the arithmetic operations are assumed to be exact and then there is rounding to the nearest representable number (computation is done at higher (typically double or higher) precision). Therefore (mind the section before)

$$\text{ralative error } \frac{|x - \hat{x}}{|x|} \leq \epsilon_{\text{mach}}, \quad \text{number } x, \text{ number on machine } \hat{x} \tag{18}$$

with the common pitfalls

- **Limitation of machine precision:** $a + b = a$ typically for $|b| < \epsilon_{\text{mach}}|a|$, i.e. when $b$ cannot be resolved by the mantissa of $a$, e.g. $(1 + 0.5\epsilon) - 0.5\epsilon$ in floating point arithmetic yields 0.9999999999999999 not 1.
- **Associativity is not guarantueed:** $(a+b)+c \underbrace{\neq}_{\text{i.A.}} a+(b+c)$, e.g. $(2.3+1e20)-1e20$ yields 0.0 but $2.3 + (1e20 - 1e20)$ yields 2.3

- **Problems of representability:** As e.g. 0.1 is not exactly representable in base $\beta = 2$, $x/10 \neq 0.1 \cdot x$ in general while $x/2.0 = 0.5 \cdot x$ is exact. The compiler may automatically choose the multiplication variant as multiplication is faster than division.
- **Cancellation:** For $x = a - b$ subtractive cancellation causes relative errors already present in $\hat{a}$ and $\hat{b}$ to be (relatively) amplified, when $a$ and $b$ are of similar size. Significant digits are lost and the relative error explodes. Consider e.g. $a = 1.75682, b = 1.75471$ with $\hat{a} = 1.76$ and $\hat{b} = 1.75$. While $\hat{a}, \hat{b}$ have small relative errors (3 digits precision[3] Note that here 0.123 ans 0.127 agree in two significant digits, where one intuitively might say this should be rather 1.), $a - b = 0.00211$ and $\hat{a} - \hat{b} = 0.01$ has a large relative error (no precise digit).
- **NaN and Inf:** All calculations including NaN yield NaN, calculations with inf mostly inf (except of course $1/\text{inf} = 0$, ...)

where we should

- **Rewrite calculations so that errors do not amplify:** For $x = 10^8, y = 10^5, z = -1 - 10^5$ we have $xy + xz = -1.0066 \cdot 10^8$ (problem in resolving the $-1$ in $xz$) but $x(y + z) = -1.0 \cdot 10^8$.
- **Compare floats based on a maximum relative error:** Instead of $x == y$ we should use e.g. $|x - y| \leq \epsilon_{\text{mach}} \cdot \max(|x|, |y|)$.
- **As avoid overflow to inf and nan:** E.g. in `float x = 1e20; float y = x * x; float z = y / x` $z$ will be inf.

### 2.2.5 Rewriting Expressions to Avoid Cancellation I

Consider $f(x) = \frac{1-\cos x}{x^2}$. For $x_e = 10^{-4}$, we have (when we represent 6 figures)

$$c := \cos x_e, \quad \hat{c} = 0.999999, \quad 1 - \hat{c} = 10^{-6}, \quad \text{while } 1 - c \approx 5 \cdot 10^{-9} \tag{20}$$

$1 - c$ has only one significant digit, the relative error is enormously amplified and we get

$$\frac{1 - \hat{c}}{x_e^2} = 100, \quad \text{while in reality for } x \neq 0 : 0 \leq f(x) \leq \frac{1}{2} \tag{21}$$

[3] $\hat{x}$ is said to approximate $x$ to the r-th digit, if the absolute error is at most $\frac{1}{2}$ in the r-th digit, so

$$\text{largest integer s so that } 10^s < |x|, \quad |x - \hat{x}| < 1/2 \cdot 10^{s-r+1} \tag{19}$$

To avoid cancellation we can rewrite using $\cos x = \cos^2 \frac{x}{2} - \sin^2 \frac{x}{2} = 1 - 2\sin^2 \frac{x}{2}$ to

$$f(x) = \frac{1}{2}\left(\frac{\sin \frac{x}{2}}{\frac{x}{2}}\right)^2 \tag{22}$$

A comparison of both versions in python can be found in figure 8, at some point (roughly below $10^{-8}$), $\cos x$ is too close to 1 to be resolved by the mantissa, so the total result is 0, above that the magnified error is visible.

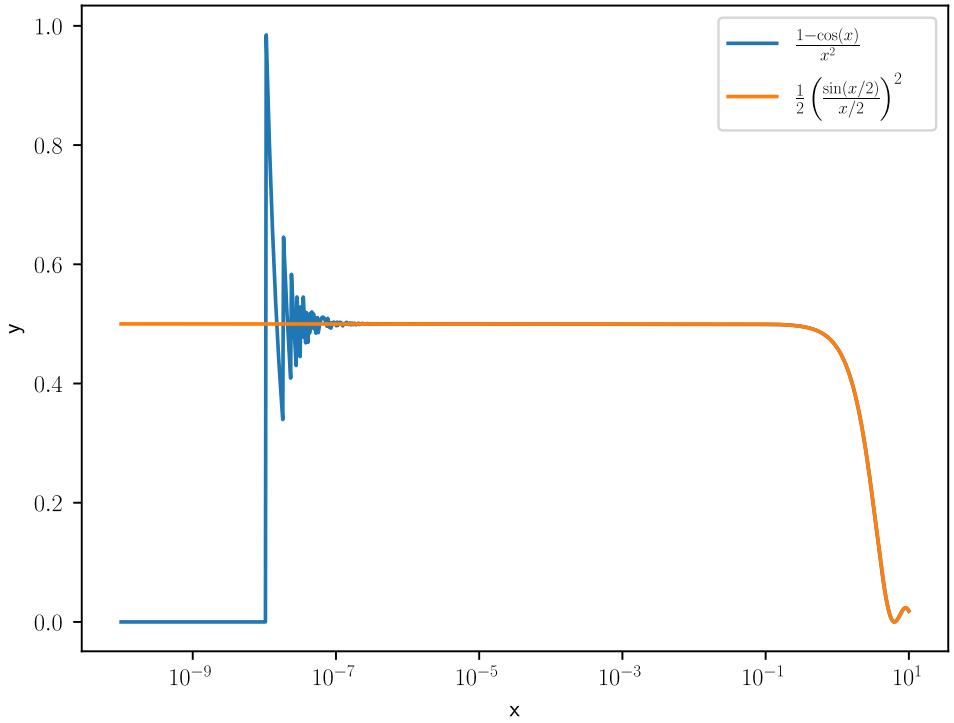



Figure 8: Comparison of the mathematically equivalent expressions $f(x) = \frac{1-\cos x}{x^2}$ and $f(x) = \frac{1}{2}\left(\frac{\sin \frac{x}{2}}{\frac{x}{2}}\right)^2$ in python.

### 2.2.6 Rewriting Expressions to Avoid Cancellation II

Consider the following expressions for the sample variance of $\{x_i\}_{i=1}^N$

$$\begin{aligned} \text{two-pass formula: } \bar{x} = \frac{1}{N}\sum_{i=1}^{N-1} x_i, \quad \sigma_N^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^2 \\ \text{one-pass formula: } \sigma_N^2 = \frac{1}{N}\left(\sum_{i=1}^{N} x_i^2 - \frac{1}{N}\left(\sum_{i=1}^{N} x_i\right)^2\right) \\ \text{as } \sigma^2 = E[(x-\bar{x})^2] = E[x^2] - E[x]^2 \end{aligned} \tag{23}$$

While for the one-pass formula, we can calculate all necessary sums in one pass through the data, it suffers heavily from cancellation: For $\{10000, 10001, 10002\}$, the two-pass formula in single precision correctly gives 1.0 while the one-pass formula yields 0.0 (cancellation) (there are better one-pass formulas though).

### 2.2.7 Accumulation of Round-off Errors

Consider the sum

$$\sum_{k=1}^{\infty} k^{-2} = \frac{\pi^2}{6} \tag{24}$$

which we want to approximate by finitely many summands. If we sum the terms just as the formula suggests from large to small, at some point, the small changes will not be resolved anymore - so better sum up from small to large. Summing up in single precision for $N = 10^7$ terms, one gets

$$\begin{aligned} \text{big to small: } 1.644725323, \quad \text{small to big: } 1.644933939, \\ \text{exact (till 9th digit): } 1.644934058 \end{aligned} \tag{25}$$

As expected the big-to-small summation is too small.

### 2.2.8 Higher Precision

The above pitfalls are less severe in higher precision. For instance in double precision (64-bit)

$$\begin{aligned} p = 52(+1) \text{ mantissa bits}, \quad 11 \text{ exponent bits, with } e_{min} = -1022, e_{max} = 1023, \\ \text{smallest and largest repr. numbers } f_{min} \simeq 2.2 \cdot 10^{-308}, f_{max} \simeq 1.8 \cdot 10^{308}, \\ |e_{min}| < |e_{max}| \to \frac{1}{f_{min}} < f_{max} \end{aligned} \tag{26}$$

with machine precision $\epsilon_{\text{mach}} \approx 2.2 \cdot 10^{-16}$. There is even quad-double precision (128-bit) (not supported on hardware though and therefore relatively slow as it has to be emulated). Packages for nearly arbitrary precision also exist.

## 2.3 A more general view on sources of numerical error

In numerical computation, the typical error sources are

- rounding
- data uncertainty
- truncation (of terms in numerical schemata, e.g. in the approximation of a function by its Taylor series)

where we have now discussed rounding errors and their effects to some extent.

## 2.4 Backward error, forward error and condition number

Consider we approximate $y = f(x)$ as $\hat{y}$ in an arithmetic of limited precision, with $f : \mathbb{R} \to \mathbb{R}$.

- **Forward error:** The absolute or relative error between $\hat{y}$ and $y$ is the forward error (living in the output space)
- **Backward error:** The backward error is the smallest $\Delta x$ (in the input space) so that $f(x + \Delta x) = \hat{y}$, so the smallest perturbation where the exact function gives our approximate result.

If this $\Delta x$ is sufficiently small, e.g. as small as the uncertainty in the data in the first place, we speak of backward stability. A weaker formulation is the mixed forward-backward error

$$\hat{y} + \Delta y = f(x + \Delta x), \quad |\Delta y| \leq \epsilon |y|, \quad |\Delta x| \leq \eta |x| \tag{27}$$

In the context of rounding errors, we call the algorithm numerically stable if $\hat{y}$ is almost the right answer for almost the right data ($\epsilon, \eta$ small).

### 2.4.1 Conditioning

Backward and forward error are connected by the conditioning of a problem, the sensitivity of the solution to perturbations in the data. Assuming $\hat{y} = f(x + \Delta x)$ and $f$ differentiable, we have

$$\hat{y} - y = f(x + \Delta x) - f(x) = f'(x)\Delta x + \mathcal{O}((\Delta x)^2) \tag{28}$$

so the relative error is

$$\frac{\hat{y}-y}{y} = \frac{f'(x)\Delta x}{f(x)} + \mathcal{O}((\Delta x)^2) \tag{29}$$

leading to the relative condition number

$$\kappa(x) \underbrace{=}_{\text{i.A.}} \lim_{\epsilon\to 0^+} \sup_{||\Delta x||\leq\epsilon} \frac{||y-\hat{y}||/||y||}{||\Delta x||/||x||} = \left|\frac{xf'(x)}{f(x)}\right| \tag{30}$$

for small $\Delta x$ measuring the relative change in the output over a relative change in the input. As a rule of thumb

$$\text{forward error} \lesssim \text{condition number} \cdot \text{backward error} \tag{31}$$

so ill-conditioned problems can have large forward errors.

**Application on Matrices**

Consider the linear system $\underline{\underline{A}}\underline{y} = \underline{x}, \underline{y} = \underline{\underline{A}}^{-1}\underline{x}, \underline{\hat{y}} = \underline{\underline{A}}^{-1}(\underline{x}+\underline{\Delta x})$. The condition number follows as

$$\begin{aligned}
\kappa(\underline{\underline{A}}) &= \max_{\underline{x},\underline{\Delta x}\neq 0} \frac{||\underline{\underline{A}}^{-1}\underline{x} - \underline{\underline{A}}^{-1}(\underline{x}+\underline{\Delta x})||/||\underline{\underline{A}}^{-1}\underline{x}||}{||\underline{\Delta x}||/||\underline{x}||} \\
&= \max_{\underline{\Delta x}\neq 0} \frac{||\underline{\underline{A}}^{-1}\underline{\Delta x}||}{||\Delta x||} \max_{\underline{x}\neq 0} \frac{||\underline{x}||}{||\underline{\underline{A}}^{-1}\underline{x}||} \\
&= \max_{\underline{\Delta x}\neq 0} \frac{||\underline{\underline{A}}^{-1}\underline{\Delta x}||}{||\underline{\Delta x}||} \max_{\underline{y}\neq 0} \frac{||\underline{\underline{A}}\underline{y}||}{||\underline{y}||} \\
&= ||\underline{\underline{A}}^{-1}|| \cdot ||\underline{\underline{A}}||
\end{aligned} \tag{32}$$

where we used the definition of the matrix norm $||\underline{\underline{A}}|| = \max_{\underline{x}\neq 0} \frac{||\underline{\underline{A}}\underline{x}||}{||\underline{x}||}$. For large condition numbers, small perturbations in the input $\underline{x}$ lead to large changes in the solution $\underline{y}$.

# 3 Solving Linear Systems

**Relevance of solving linear equations for simulations**: Implicit schemes lead to linear systems of equations which we have to solve, examples already covered are implicit Euler applied to a linear ODE in sec. **??**, or to a non-linear one (the linear equation then follows from doing the implicit step by Newton-Raphson where the Jacobian is to be inverted (sec. **??**)), or just recently in the diffusion equation (based on method of lines and implicit Euler or Crank-Nicolson).

Another example of a linear system we care about is solving a discretized Poisson equation.

Direct inversion or LU or QU decomposition of our linear system (the matrix describing it) are often too costly, so iterative methods - methods where we construct a sequence of approximate solutions that hopefully converge to the exact solution - are used.

In the multigrid-technique, linear equations are solved using a hierarchy of discretizations.

## 3.1 Motivational Example 1: From the Poisson equation we can get a possibly big linear system

Let us discretize the 1D Poisson equation

$$\partial_x^2\phi = 4\pi G\rho \to (\partial_x^2\phi) \approxeq \frac{\phi_{i+1} - 2\phi_i + \phi_{i-1}}{h^2} \approxeq 4\pi G\rho_i \qquad i = 1, \dots, N, \quad \text{spacing } h \tag{33}$$

We write this as a matrix equation

$$\underline{\underline{A}}\underline{\phi} = \underline{b}, \quad \underline{\phi} = \begin{pmatrix} \phi_1 \\ \phi_2 \\ \vdots \\ \phi_N \end{pmatrix}, \quad \underline{b} = 4\pi G\underline{\rho}, \underline{\rho} = \begin{pmatrix} \rho_1 \\ \rho_2 \\ \vdots \\ \rho_N \end{pmatrix} \tag{34}$$

with in the case of periodic boundary conditions

$$\underline{\underline{A}} = \frac{1}{h^2}\begin{pmatrix} -2 & 1 & 0 & \dots & 0 & 1 \\ 1 & -2 & 1 & 0 & \dots & \\ 0 & 1 & -2 & 1 & 0 & \dots \\ 0 & 0 & \ddots & \ddots & \ddots & 0 \\ 0 & \dots & & 1 & -2 & 1 \\ 1 & 0 & 0 & \dots & 1 & -2 \end{pmatrix} \tag{35}$$

**Problem:** LU-decomposition or Gauss elimination with pivoting are generally $\mathcal{O}(N^3)$ (for such a sparse matrix we can do better though). Imagine a discretization in 3D, then for 1000 grid points per dimension $\underline{\underline{A}}$ would be $10^9 \times 10^9$ (mind the discretization in 3D is not *as* simple as in 1D).

## 3.2 Poisson equation and solving a tridiagonal system

### 3.2.1 1D heat Diffusion equation with Dirichlet boundaries in matrix form

Consider simple heat diffusion with constant heating rate $\epsilon$.

$$-D\partial_x^2 T = \epsilon \tag{36}$$

so a 1D Poisson equation with constant source term $\epsilon$.

Discretized we get as before

$$\frac{T_{i+1} - 2T_i + T_{i-1}}{\delta^2} \approxeq -\frac{\epsilon}{D} \tag{37}$$

So for a state vector

$$\underline{T} = \begin{pmatrix} T_1 \\ T_2 \\ \vdots \\ T_N \end{pmatrix} \tag{38}$$

with Dirichlet boundary conditions $T_1 = T_N := T_0$ we get the matrix equation

$$\underline{\underline{A}}\underline{T} = \underline{b}, \quad \underline{b} = \begin{pmatrix} T_0 \\ -\frac{\delta^2\epsilon}{D} \\ \vdots \\ -\frac{\delta^2\epsilon}{D} \\ T_0 \end{pmatrix}, \quad \underline{\underline{A}} = \frac{1}{h^2}\begin{pmatrix} 1 & 0 & 0 & \dots & 0 & 0 \\ 1 & -2 & 1 & 0 & \dots & \\ 0 & 1 & -2 & 1 & 0 & \dots \\ 0 & 0 & \ddots & \ddots & \ddots & 0 \\ 0 & \dots & & 1 & -2 & 1 \\ 0 & 0 & 0 & \dots & 0 & 1 \end{pmatrix} \tag{39}$$

So for a 1D Poisson problem with Dirichlet (constant) boundary conditions we get a tridiagonal matrix.

### 3.2.2 Forward elimination backward substitution method for solving a tridiagonal system

Consider the general tridiagonal matrix

$$\underline{\underline{A}} = \begin{pmatrix} d_1 & u_1 & 0 & \dots & 0 & 0 \\ l_1 & d_2 & u_2 & 0 & \dots & \\ 0 & l_2 & d_3 & u_3 & 0 & \dots \\ 0 & 0 & \ddots & \ddots & \ddots & 0 \\ 0 & \dots & & l_{N-2} & d_{N-1} & u_{N-1} \\ 0 & 0 & \dots & 0 & l_{N-1} & d_N \end{pmatrix} \tag{40}$$

where we want to solve

$$\underline{\underline{A}}\underline{x} = \underline{b} \tag{41}$$

for $\underline{x}$. We use elementary operations $\underline{\underline{E}}_i$ to add multiples of rows in $\underline{\underline{A}}$ to other rows in $\underline{\underline{A}}$.

1. (Forward elimination) Using those operations, we succesively (top-down) eliminate the lower diagonal $\underline{l}$ of $\underline{\underline{A}}$.

$$\underline{\underline{E}}_1\underline{\underline{E}}_2 \dots \underline{\underline{E}}_{N-1}\underline{\underline{A}}\underline{x} = \begin{pmatrix} d_1 & u_1 & 0 & \dots & 0 & 0 \\ 0 & \tilde{d}_2 & u_2 & 0 & \dots & \\ 0 & 0 & \tilde{d}_3 & u_3 & 0 & \dots \\ 0 & 0 & \ddots & \ddots & \ddots & 0 \\ 0 & \dots & & 0 & \tilde{d}_{N-1} & u_{N-1} \\ 0 & 0 & \dots & 0 & 0 & \tilde{d}_N \end{pmatrix} \underline{x} = \underline{\underline{E}}_1\underline{\underline{E}}_2 \dots \underline{\underline{E}}_{N-1}\underline{b} \tag{42}$$

   with $\tilde{d}_2 = d_2 - \frac{l_1}{d_1}u_1$ and $\tilde{d}_i = d_i - \frac{l_{i-1}}{\tilde{d}_{i-1}}u_{i-1}$ for $i = 3, \dots, N$.

2. (Backward substitution) We can now solve the system starting from the last row.

## 3.3 Classical Exact Solution: LU-decomposition*

**Why is it not a good idea to numerically invert the matrix?:** While $\underline{\underline{A}} \in \mathbb{R}^{N\times N}$ might often be sparse (e.g. a sparse Jacobian), $\underline{\underline{A}}^{-1}$ is dense in general and our storage might not be able to handle $N^2$ terms of a dense inverse. This is not a problem of stability - the inverse can be computed as numerically stable as other methods are. Matrix inversion can even by done in $\mathcal{O}(N^{\log_2 7})$ (Strassen algorithm, *Gaussian elimination is not optimal*).

Let us discuss LU-decomposition (also called LR-decomposition).

Consider $\underline{\underline{L}}, \underline{\underline{R}} \in \mathbb{R}^{N\times N}$ and $\underline{\underline{A}} \in \mathbb{R}^{N\times N}$ invertible with

$$\underline{\underline{A}} = \underline{\underline{L}}\,\underline{\underline{R}}, \quad \text{lower-left triangular matrix } \underline{\underline{L}}, \text{ upper-right triangular matrix } \underline{\underline{R}} \tag{43}$$

**Note:** More generally, $\underline{\underline{P}}\,\underline{\underline{A}} = \underline{\underline{L}}\,\underline{\underline{R}}$ with $\underline{\underline{P}}$ a permutation matrix is possible for every invertible matrix $\underline{\underline{A}}$. Otherwise the decomposition is not always possible, e.g. for $a_{11} = 0 \to r_{11}$ or $l_{11} = 0$, so $\underline{\underline{L}}$ or $\underline{\underline{R}}$ would be singular (not full rank), so $\underline{\underline{A}}$ would not be invertible (contradiction). So $\underline{\underline{P}}$ is used to switch rows of $\underline{\underline{A}}$.

### 3.3.1 Solving the linear system for $\underline{x}$ when we already know the LU-decomposition

We can split

$$\underline{\underline{A}}\,\underline{x} = \underline{\underline{L}}\,\underline{\underline{R}}\,\underline{x} \quad \leftrightarrow \quad \underline{\underline{L}}\,\underline{y} = \underline{b}, \quad \underline{\underline{R}}\,\underline{x} = \underline{y} \tag{44}$$

so into solving two triangular systems.

1. (Forward substitution) Solve $\underline{\underline{L}}\,\underline{y} = \underline{b}$ for $\underline{y}$. We can write the system as

$$\begin{pmatrix} l_{11} & \\ \underline{L}_{*1} & \underline{\underline{L}}_{**} \end{pmatrix} \begin{pmatrix} y_1 \\ \underline{y}_* \end{pmatrix} = \begin{pmatrix} b_1 \\ \underline{b}_* \end{pmatrix} \Rightarrow l_{11}y_1 = b_1, \quad \underline{L}_{*1}y_1 + \underline{\underline{L}}_{**}\underline{y}_* = b_* \tag{45}$$

   from which we can solve for $y_1 = \frac{b_1}{l_{11}}$. We can then construct a new triangular system for $\underline{y}_*$

$$\underline{\underline{L}}_{**}\underline{y}_* = \underline{b}_* - \underline{L}_{*1}y_1 \tag{46}$$

   so $\underline{y}$ can be solved recursively, yielding the explicit formula

$$y_i = \frac{1}{l_{ii}}\left(b_i - \sum_{k=1}^{i-1} l_{ik}y_k\right) \tag{47}$$

   taking $\mathcal{O}(N^2)$ operations.

2. (Backward substitution) Then $\underline{\underline{R}}\,\underline{x} = \underline{y}$ for $\underline{x}$. The logic is the same as for forward substitution, but bottom-up. We can write the system as

$$\begin{pmatrix} \underline{\underline{R}}_{**} & \underline{R}_{*n} \\ & r_{nn} \end{pmatrix} \begin{pmatrix} \underline{x}_* \\ x_1 \end{pmatrix} = \begin{pmatrix} \underline{y}_* \\ y_n \end{pmatrix} \tag{48}$$

   yielding the explicit formula

$$a_i = \frac{1}{r_{ii}}\left(y_i - \sum_{k=i+1}^{N} r_{ik}a_k\right) \tag{49}$$

also taking $\mathcal{O}(N^2)$ operations.

### 3.3.2 Calculating the LU-decomposition in $\mathcal{O}(N^3)$

From

$$\begin{pmatrix} a_{11} & \underline{A}_{1*}^T \\ \underline{A}_{*1} & \underline{\underline{A}}_{**} \end{pmatrix} = \begin{pmatrix} l_{11} & \\ \underline{L}_{*1} & \underline{\underline{L}}_{**} \end{pmatrix} \begin{pmatrix} r_{11} & \underline{R}_{1*}^T \\ & \underline{\underline{R}}_{**} \end{pmatrix}, \quad \text{set diagonal of } \underline{\underline{L}} \text{ to } 1 \rightarrow \text{ unique decomp} \tag{50}$$

we can devise an $\mathcal{O}(N^3)$ algorithm to calculate $\underline{\underline{L}}$ and $\underline{\underline{R}}$, see code 2.

```
# in-place decomposition of A into L and R, the upper-right part of A is overwritten by R,
# the lower-left part of A is overwritten by L without diagonal (set to 1)
function lu_decomposition!(A::Matrix{T}) where T <: Number
    N = size(A, 1)
    for i in 1:N
        for j in i+1:N
            A[j, i] /= A[i, i]
            for k in i+1:N
                A[j, k] -= A[j, i] * A[i, k]
            end
        end
    end
end
# one can then extract L = tril(A, -1) + I
# and U = triu(A) via the LinearAlgebra package
```

Code-Snippet 2: LU-decomposition of a matrix $\underline{\underline{A}} \in \mathbb{R}^{N \times N}$ in $\mathcal{O}(N^3)$.

**Note:** Blocked versions can be parallelized, QR-decomposition retains the condition number of $\underline{\underline{A}}$.

**Problem:** LU-decomposition is $\mathcal{O}(N^3)$.

## 3.4 Jacobi iteration | a splitting method

**Aim:** We want quicker than $\mathcal{O}(N^3)$, approximate solutions to linear systems.

Consider the linear system

$$\underline{\underline{A}}\,\underline{x} = \underline{b}, \quad \underline{\underline{A}} \in \mathbb{R}^{N \times N}, \underline{x}, \underline{b} \in \mathbb{R}^N \tag{51}$$

For Jacobi iteration, we start with the trivial decomposition

$$\underline{\underline{A}} = \underline{\underline{D}} - (\underline{\underline{L}} + \underline{\underline{U}}), \quad \text{diagonal part } \underline{\underline{D}}, \quad \text{negative left-below-diagonal part} \underline{\underline{L}} \quad \text{negative right-above-diagonal part} \underline{\underline{U}} \tag{52}$$

so

$$\underline{\underline{A}}\underline{x} = \left[\underline{\underline{D}} - (\underline{\underline{L}} + \underline{\underline{U}})\right] \underline{x} = \underline{b} \quad \rightarrow \quad \underline{x} = \underline{\underline{D}}^{-1}\underline{b} + \underline{\underline{D}}^{-1}(\underline{\underline{L}} + \underline{\underline{U}})\underline{x} \tag{53}$$

so $\underline{x}$ is a fixed point of the RHS, leading to fixed-point, here Jacobi iteration

$$\boxed{\underline{x}^{(n+1)} = \underline{\underline{D}}^{-1}\underline{b} + \underline{\underline{D}}^{-1}(\underline{\underline{L}} + \underline{\underline{U}})\underline{x}^{(n)}, \quad (\underline{\underline{D}}^{-1})_{ii} = \frac{1}{A_{ii}}} \tag{54}$$

### 3.4.1 When does the Jacobi iteration converge?

The Jacobi iteration converges if and only if all eigenvalues $\lambda_i$ of the convergence matrix

$$\underline{\underline{M}} := \underline{\underline{D}}^{-1}(\underline{\underline{L}} + \underline{\underline{U}}) \tag{55}$$

are smaller than one.

$$\text{spectral radius } \rho_s(\underline{\underline{M}}) \equiv \max_i |\lambda_i| < 1 \tag{56}$$

The smaller the spectral radius, the faster the convergence.

#### 3.4.1.1 Derivation of the convergence criterion

Consider the error at step $n+1$

$$\underline{e}^{(n+1)} = \underline{x}_{\text{exact}} - \underline{x}^{(n+1)} \underbrace{=}_{\text{Jacobi-Iteration}} \underline{x}_{\text{exact}} - \left(\underline{\underline{D}}^{-1}\underline{b} + \underline{\underline{M}}\underline{x}^{(n)}\right)$$
$$\underbrace{=}_{\underline{x}_{\text{exact}} = \underline{\underline{D}}^{-1}\underline{b} + \underline{\underline{M}}\underline{x}_{\text{exact}}} \underline{\underline{M}}(\underline{x}_{\text{exact}} - \underline{x}^{(n)}) = \underline{\underline{M}}\underline{e}^{(n)} \tag{57}$$

so

$$\underline{e}^{(n)} = \underline{\underline{M}}^n \underline{e}^{(0)} \tag{58}$$

which scales with $\rho_s^n(\underline{\underline{M}})$ for $n$ sufficiently large (follows from decomposing $\underline{e}^{(0)}$ into eigenvectors of $\underline{\underline{M}}$) so the convergence criterion follows.

**Problem:** The convergence is usually slow, better use Gauss-Seidel iteration.

### 3.4.2 Example Jacobi Step

Consider

$$\underline{\underline{A}}\underline{x} = \begin{pmatrix} 10 & -5 & 0 \\ -5 & 10 & -5 \\ 0 & -5 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \underline{b} = \begin{pmatrix} 0 \\ 0 \\ 15 \end{pmatrix} \tag{59}$$

then

$$\underline{\underline{D}} = \begin{pmatrix} 10 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 5 \end{pmatrix}, \quad \underline{\underline{L}} = \begin{pmatrix} 0 & 0 & 0 \\ 5 & 0 & 0 \\ 0 & 5 & 0 \end{pmatrix}, \quad \underline{\underline{R}} = \begin{pmatrix} 0 & 5 & 0 \\ 0 & 0 & 5 \\ 0 & 0 & 0 \end{pmatrix} \tag{60}$$

so

$$\underline{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \quad \rightarrow \quad \underline{x}^{(1)} = \begin{pmatrix} \frac{1}{10}b_0 + \frac{1}{10} \cdot 5x_2^{(0)} \\ \frac{1}{10}b_1 + \frac{1}{10} \cdot \left(5x_1^{(0)} + 5x_3^{(0)}\right) \\ \frac{1}{5}b_1 + \frac{1}{5} \cdot 5x_2^0 \end{pmatrix} \tag{61}$$

## 3.5 Gauss-Seidel iteration | better splitting method

**Idea:** In each step $\underline{x}^{(n)} \rightarrow \underline{x}^{(n+1)}$ ($\underline{x} \in \mathbb{R}^N$) consists of $N$ scalar updates. If we do these $N$ scalar updates iteratively (not in parallel) we can use the results from previous updates.

### 3.5.1 Motivational Example

For instance in the example above

$$\underline{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \quad \rightarrow \quad \underline{x}^{(1)} = \begin{pmatrix} \frac{1}{10}b_0 + \frac{1}{10} \cdot 5x_2^{(0)} \\ \frac{1}{10}b_1 + \frac{1}{10} \cdot \left(5x_1^{(1)} + 5x_3^{(0)}\right) \\ \frac{1}{5}b_1 + \frac{1}{5} \cdot 5x_2^1 \end{pmatrix} \tag{62}$$

### 3.5.2 Gauss-Seidel update

Let us devise a different fix-point equation

$$(\underline{\underline{D}} - \underline{\underline{L}})\underline{x} = \underline{\underline{U}}\underline{x} + \underline{b} \rightarrow \underline{x} = (\underline{\underline{D}} - \underline{\underline{L}})^{-1}\underline{\underline{U}}\underline{x} + (\underline{\underline{D}} - \underline{\underline{L}})^{-1}\underline{b} \tag{63}$$

from which we follow the fix-point iteration

$$\underline{x}^{(n+1)} = (\underline{\underline{D}} - \underline{\underline{L}})^{-1}\underline{\underline{U}}\underline{x}^{(n)} + (\underline{\underline{D}} - \underline{\underline{L}})^{-1}\underline{b} \tag{64}$$

**Problem:** We cannot easily compute $(\underline{\underline{D}} - \underline{\underline{L}})^{-1}$.

Therefore, multiply with $(\underline{\underline{D}} - \underline{\underline{L}})$.

$$(\underline{\underline{D}} - \underline{\underline{L}})\underline{x}^{(n+1)} = \underline{\underline{U}}\underline{x}^{(n)} + \underline{b} \quad \rightarrow \quad \underline{x}^{(n+1)} = \underline{\underline{D}}^{-1}\underline{\underline{L}}\underline{x}^{(n+1)} + \underline{\underline{D}}^{-1}\underline{\underline{U}}\underline{x}^{(n)} + \underline{\underline{D}}^{-1}\underline{b} \tag{65}$$

**But isn't this implicit now with $\underline{x}^{(n+1)}$ on both sides?**: $\underline{\underline{L}}$ is a lower diagonal matrix (diagonal and everything above zero), so in $\underline{\underline{L}}\underline{x}^{(n+1)}$ we always only need values we have already calculated if we solve consecutively ($\rightarrow$ problem for parallelization). This is illustrated in fig. 9.

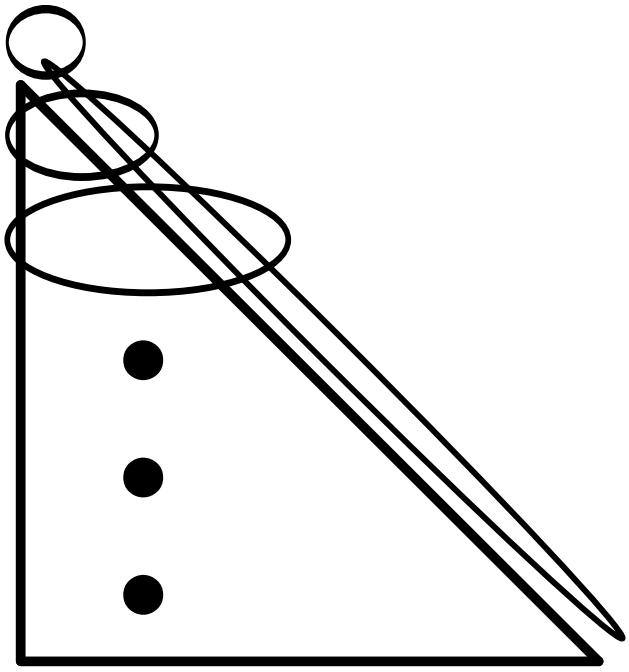


Figure 9: As of $\underline{\underline{L}}\underline{x}^{(n+1)}$ to calculate $x_i^{(n+1)}$ we only need $x_0^{(n+1)}, \ldots, x_{i-1}^{(n+1)}$ which we have already calculated.

**Advantage of Gauss-Seidel:** Convergence is sped up (often by a factor of 2) compared to Jacobi iteration. Convergence is guaranteed if $\underline{\underline{A}}$ is strictly diagonally dominant $|A_{ii}| > \sum_{j \neq i} |A_{ij}| \quad \forall i$ or symmetric and positive definite.

### 3.5.3 The problem of parallelization in Gauss-Seidel and red-black ordering

**Problem:** In Gauss-Seidel, the equations must be solved in sequential order - this cannot be parallelized. A general problem of using information from the same step is that the overall result is order dependent (which element is selected first).

**Idea:** Do not do a scheme with sequential dependence (where information is used as soon as available) but if possible e.g. rather split the $N$ updates into two groups where all updates within a group are independent, but the updates of the second group depend on the ones from first.

For instance for the 2d-Poisson equation, *a red-black ordering* scheme can be derived, see figure 10 (details follow).

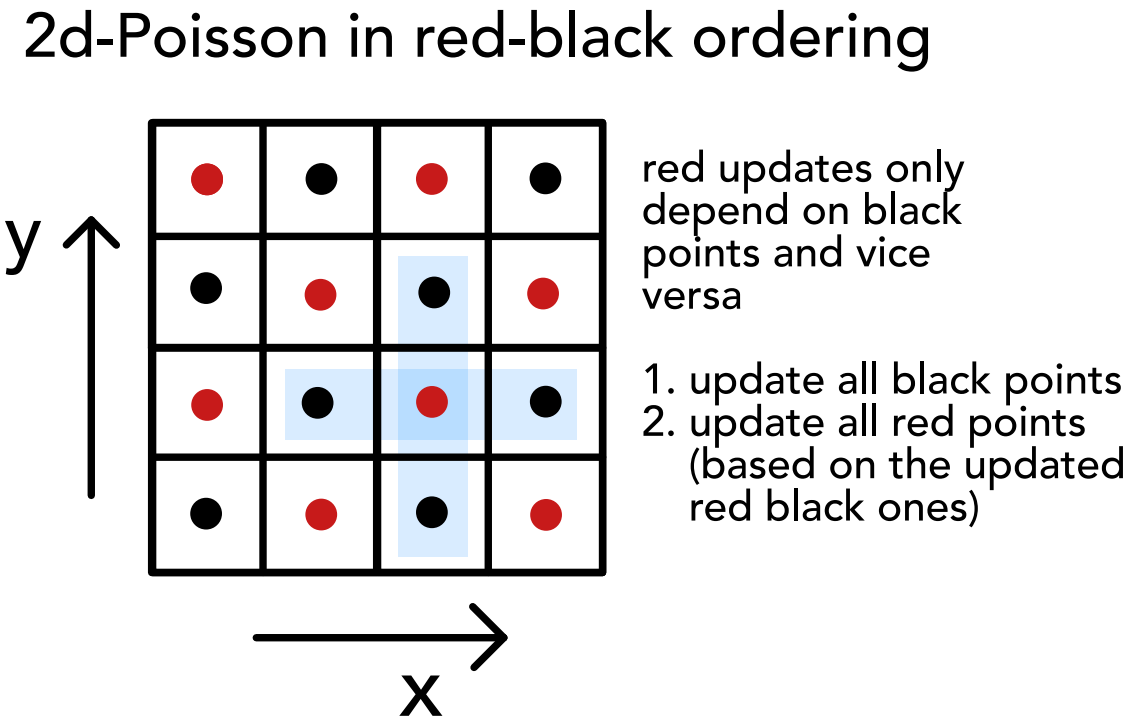



Figure 10: Red-black ordering for the 2D Poisson equation.

## 3.6 Relaxation problem | Poisson equation in red-black ordering

We can physically understand why applying an iterative scheme to a stationary (elliptic) problem makes sense in terms of relaxation of a dynamical system.

### 3.6.1 Formulating an elliptic equation as an equilibrium of a relaxation problem

Consider the elliptic equation

$$Lx = b \tag{66}$$

for instance for the gravitational 2D-Poisson problem $(\partial_x^2 + \partial_y^2)\phi = 4\pi G\rho$.

The elliptic problem can be written as the equilibrium of the relaxation problem

$$\frac{1}{K}\partial_t x = Lx - b, \quad \text{for } \partial_t x = 0 \text{ for } t \to \infty, \quad \text{some factor } K \text{ in } \frac{\mathrm{m}^2}{s} \tag{67}$$

### 3.6.2 Red-black ordering for the 2D Poisson equation

Consider for instance the 2D Poisson equation, formulated as a relaxation problem, here central in space, forward in time (here assume $\Delta y = \Delta x$)

$$\frac{1}{K}\frac{\phi_{i,j}^{(n+1)} - \phi_{i,j}^{(n)}}{\Delta t} = \frac{\phi_{i+1,j}^{(n)} - 2\phi_{i,j}^{(n)} + \phi_{i-1,j}^{(n)}}{\Delta x^2} + \frac{\phi_{i,j+1}^{(n)} - 2\phi_{i,j}^{(n)} + \phi_{i,j-1}^{(n)}}{\Delta x^2} - 4\pi G\rho_{ij} \tag{68}$$

so

$$\phi_{i,j}^{(n+1)} = \frac{K\Delta t}{\Delta x^2}\left(\phi_{i+1,j}^{(n)} + \phi_{i-1,j}^{(n)} + \phi_{i,j+1}^{(n)} + \phi_{i,j-1}^{(n)}\right) + \underbrace{\left(1 - 4\frac{K\Delta t}{\Delta x^2}\right)}_{=0 \text{ by choice } \Delta t = \frac{1}{4}\frac{\Delta x^2}{K}} \phi_{i,j}^{(n)} - 4\pi G K \rho_{ij}\Delta t \tag{69}$$

where the time-step choice is akin to the CFL criterion in diffusion.

The discretized Poisson-relaxation update lends itself naturally to a red-black ordering scheme.

## 3.7 Multigrid technique

We have now gained an understanding of iteratively solving a linear equation as a relaxation problem.

**Note:** In each update step information travels as given by the stencil, for the Poisson equation only between nearest neighbors. This also limits the largest possible *timestep* to the CFL criterion.

**Problem:** As only cells in the stencil (often only neighboring cells) communicate per step in Jacobi and Gauss-Seidel iteration, we have to make a compromise between speed of convergence and resolution:

- On a coarse grid, information travels quicker through space (in less steps) but the resolution is bad
- On a fine grid, resolution is good, but long range interactions take lots of computational steps and long-wavelength errors (in the error vector $\underline{e}$) die out only very slowly

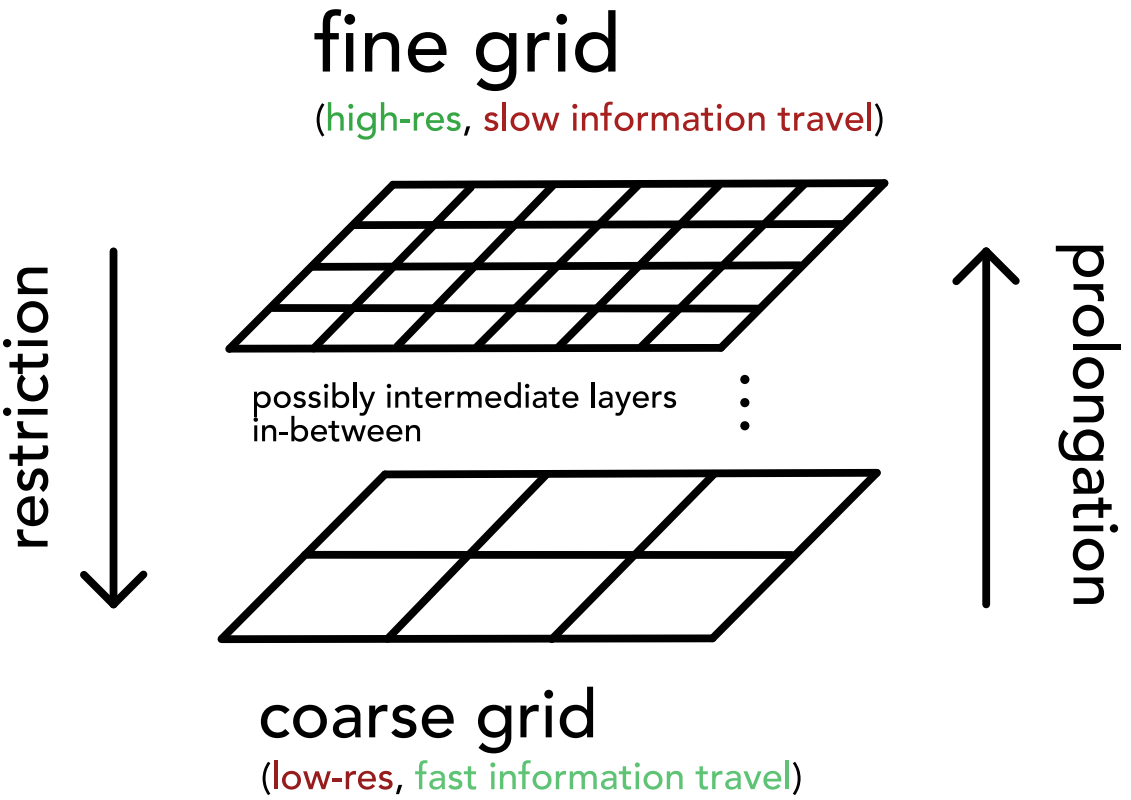



Figure 11: Grids of different resolution.

**Idea:** Start on a coarse grid, where information travels quickly, long range correlations are taken care of and we have fast convergence, then get the information onto a fine grid and resolve the details.

1. But how can we map from a coarser to a finer grid and vice versa?
2. How can we solve $\underline{\underline{A}}\underline{x} = \underline{b}$ on the coarser grid?

**Note:** Coarse-to-fine is called prolongation, fine-to-coarse restriction (see figure 11).

### 3.7.1 Getting finer and coarser | prolongation and restriction

**Note:** In the following we will only consider the case of a 1D grid (not 2D as illustrated before).

Consider meshes

1. Let $\Omega^{(h)}$ denote a 1D mesh with $N$ cells $i = 1, \ldots, N$ with spacing $h$.
2. Let $\Omega^{(2h)}$ denote a 1D mesh with $N/2$ cells $i = 1, \ldots, N/2$ with spacing $2h$.

and let

1. $\underline{x}_i^{(h)} \in \mathbb{R}^N$ denote the solution on $\Omega^{(h)}$, so $x_i^{(h)}$ is the solution on cell $i$ of $\Omega^{(h)}$ (e.g. a density $\rho_i^{(h)}$).
2. $\underline{x}_i^{(2h)} \in \mathbb{R}^{N/2}$ denote the solution on $\Omega^{(2h)}$, so $x_i^{(2h)}$ is the solution on cell $i$ of $\Omega^{(2h)}$.

Restriction and prolongation in 1D are illustrated in figure 12.

Prolongation and restriction are linear operators written as matrices $\underset{=\text{from this spacing}}{I^{\text{to this spacing}}}$.

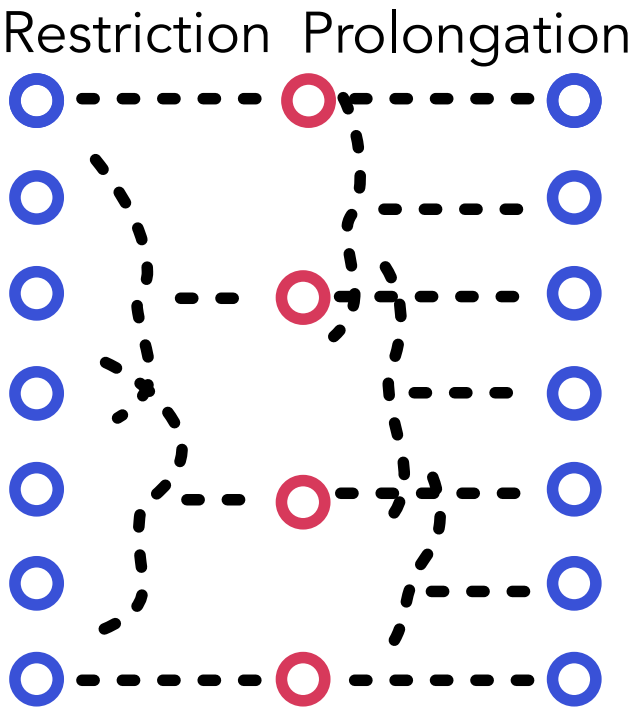



Figure 12: Illustration of restriction and prolongation.

#### 3.7.1.1 Coarse-to-fine | interpolation called prolongation

We map from coarse to fine using

$$\underset{=2h}{I^h} \underline{x}^{(2h)} = \underline{x}^{(h)} \tag{70}$$

for instance by

$$\underset{=2h}{I^h} \in \mathbb{R}^{N \times \frac{N}{2}} : \text{for } 0 \leq i < \frac{N}{2} : x_{2i}^{(h)} = x_i^{(2h)}, \quad x_{2i+1}^{(h)} = \frac{1}{2}\left(x_i^{(2h)} + x_{i+1}^{(2h)}\right) \tag{71}$$

as shown in figure 12 and furthermore in figure 13.

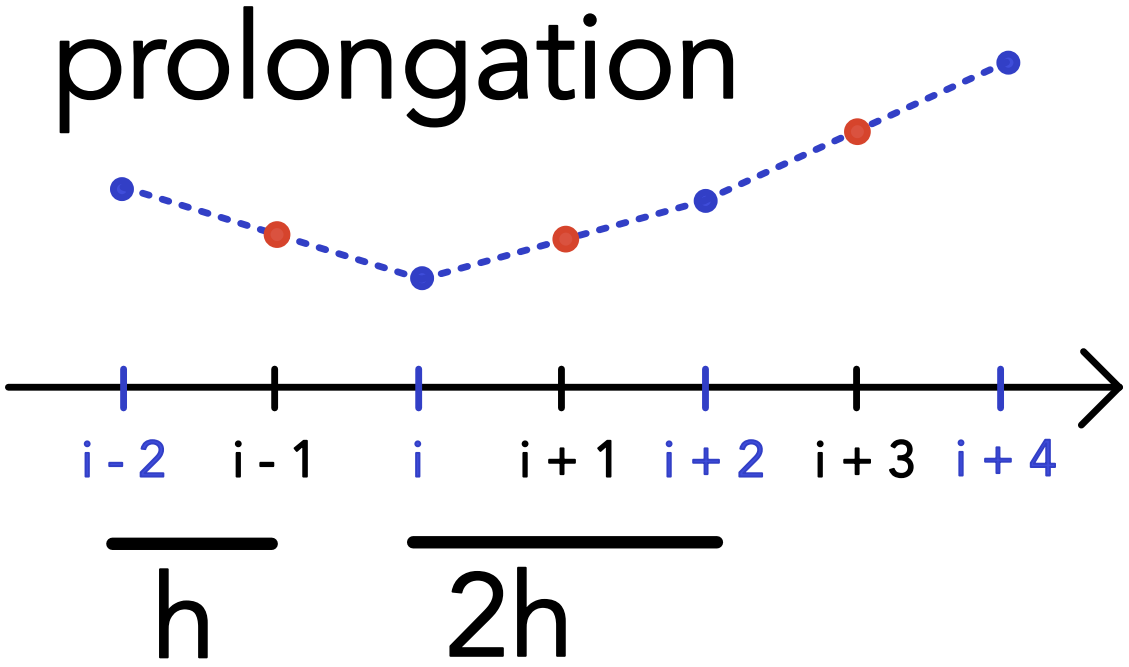



Figure 13: 1D-Prolongation from coarse to fine grid.

#### 3.7.1.2 Fine-to-coarse | restriction

We now convert

$$\underset{=h}{I}^{2h}\underline{x}^{(h)} = \underline{x}^{(2h)} \tag{72}$$

with a simple example being

$$\underset{=h}{I}^{2h} \in \mathbb{R}^{\frac{N}{2}\times N} : \text{for } 0 \leq i < \frac{N}{2} : x_i^{(2h)} = \frac{x_{2i-1}^{(h)} + 2x_{2i}^{(h)} + x_{2i+1}^{(h)}}{4} \tag{73}$$

#### 3.7.1.3 Relation of restriction and prolongation

Prolongation and restriction matrices are oftentimes constructed to be scaled transposes of each other, so

$$\underset{=h}{I}^{2h} = c[\underset{=2h}{I}^{h}]^T \tag{74}$$

Consider for instance

$$\begin{aligned}\text{prolongation: } \underset{=2h}{I}^{h} &= \frac{1}{2}\begin{pmatrix} 1 & & \\ 2 & & \\ 1 & 1 & \\ & 2 & \\ & 1 & 1 \\ & & 2 \\ & & 1 \end{pmatrix} \\ \text{restriction: } \underset{=h}{I}^{2h} &= \frac{1}{2}\left(\underset{=2h}{I}^{h}\right)^{T} = \frac{1}{4}\begin{pmatrix} 1 & 2 & 1 & & & & \\ & & 1 & 2 & 1 & & \\ & & & & 1 & 2 & 1 \end{pmatrix}\end{aligned} \tag{75}$$

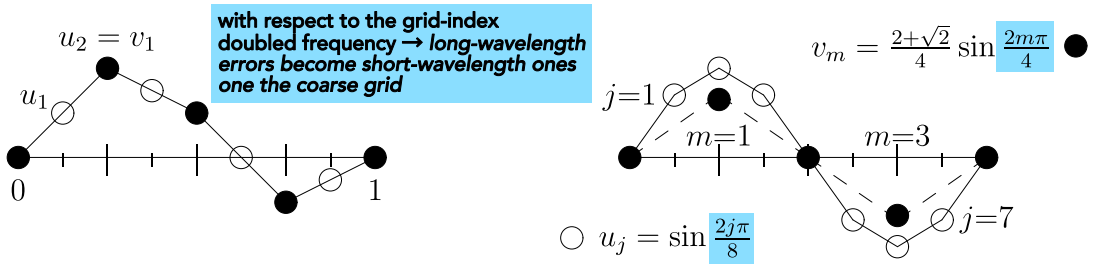
so the application of **prolongation** means

$$\frac{1}{2}\begin{pmatrix} 1 & & \\ 2 & & \\ 1 & 1 & \\ & 2 & \\ & 1 & 1 \\ & & 2 \\ & & 1 \end{pmatrix}\begin{pmatrix} x_1^{(2h)} \\ x_2^{(2h)} \\ x_3^{(2h)} \end{pmatrix} = \begin{pmatrix} x_1^{(2h)}/2 \\ x_1^{(2h)} \\ x_1^{(2h)}/2 + x_2^{(2h)}/2 \\ x_2^{(2h)} \\ x_2^{(2h)}/2 + x_3^{(2h)}/2 \\ x_3^{(2h)} \\ x_3^{(2h)}/2 \end{pmatrix} = \begin{pmatrix} x_1^{(h)} \\ x_2^{(h)} \\ x_3^{(h)} \\ x_4^{(h)} \\ x_5^{(h)} \\ x_6^{(h)} \\ x_7^{(h)} \end{pmatrix} \tag{76}$$

and **restriction**

$$\frac{1}{4}\begin{pmatrix} 1 & 2 & 1 & & & & \\ & & 1 & 2 & 1 & & \\ & & & & 1 & 2 & 1 \end{pmatrix}\begin{pmatrix} x_1^{(h)} \\ x_2^{(h)} \\ x_3^{(h)} \\ x_4^{(h)} \\ x_5^{(h)} \\ x_6^{(h)} \\ x_7^{(h)} \end{pmatrix} = \begin{pmatrix} x_1^{(2h)} \\ x_2^{(2h)} \\ x_3^{(2h)} \end{pmatrix} \tag{77}$$

Both are illustrated in figure 14, also illustrating that with respect to the grid index, low-frequency errors on the fine grid have double (so higher) frequency on the coarse grid.




(a) Linear interpolation by $u = I_{2h}^h v$ (b) Restriction by $R_h^{2h} u = \frac{1}{2}(I_{2h}^h)^{\mathrm{T}} u$

Figure 14: Prolongation and restriction of a sine wave (figure by Gilbert Strang).

**Note:** Taking prolongation and restriction as transposes of each others breaks at the boundaries, if we do not as in figure 14 have zeroes on the boundaries.

#### 3.7.1.4 Short-hand stencil notation

Short-hand notations for prolongation and restriction are given in table 3.

| **Prolongation** | **Restriction** |
|---|---|
| We can write prolongation as<br>1D prolongation: $\underset{=2h}{I}^h : \begin{bmatrix} \frac{1}{2} & 1 & \frac{1}{2} \end{bmatrix}$ (78)<br>meaning that every coarse point is added to three fine points with these respective weights (see previous example). | We can write restriction as<br>1D restriction: $\underset{=h}{I}^{2h} : \begin{bmatrix} \frac{1}{4} & \frac{1}{2} & \frac{1}{4} \end{bmatrix}$ (79)<br>- the coarse point is the weighted sum of the three fine points. |

Table 3: Short-hand notations for prolongation and restriction.

Similar short-hand notations can be devised for 2D and 3D.

### 3.7.2 Multigrid V-cycle

Our aim is still to solve $\underline{\underline{A}}\underline{x} = \underline{b}$ smartly, by - in the view of a relaxation problem - doing the rough large-scale work quickly on a coarse and the fine details on a fine grid afterwards. We still have to anwer

- How can such a procedure look like?
- How do we convert $\underline{\underline{A}}$ to coarser grids?

#### 3.7.2.1 Initial definitions

The error is defined as

$$\underline{e} \equiv \underline{x}_{\text{exact}} - \underline{\tilde{x}}, \quad \text{current approximate solution } \underline{\tilde{x}} \tag{80}$$

**Note:** The residual is the error in the solution, not $\underline{x}$. The error and residual are related linearly by $\underline{\underline{A}}$.

$$\underline{\underline{A}}\underline{x}_{\text{exact}} = \underline{b} \quad \rightarrow \quad \underline{\underline{A}}(\underline{e} + \underline{\tilde{x}}) = \underline{b} \quad \rightarrow \quad \underline{\underline{A}}\underline{e} = \underline{r} \tag{81}$$

**Idea:** Consider we have an initial guess $\underline{\tilde{x}}$ on the fine grid. We can easily calculate the residual $\underline{r}$. We can then restrict $\underline{r}$ to a coarse grid, solve for the error there ($\underline{\underline{A}}\underline{e} = \underline{r}$) (given we can transform $\underline{\underline{A}}$ to the coarse grid) and then prolong the error to the fine grid and make the correction $\underline{x}_{\text{exact}} = \underline{\tilde{x}} + \underline{e}$.

Let us formalize this idea.

#### 3.7.2.2 Coarse-grid correction scheme

We use the error calculated on the coarse grid to correct on the fine grid.

Let us start with a (fine-grid) guess $\underline{\tilde{x}}^{(h)}$ for the problem

$$\underline{\underline{A}}^{(h)}\underline{x}^{(h)} = \underline{b}^{(h)} \tag{82}$$

and we want to obtain an improvement $\underline{\tilde{x}}'^{(h)} = CG(\underline{\tilde{x}}^{(h)}, \underline{b}^{(h)})$.

$CG$ consists of the following steps

1. Perform one iterative *relaxation* step on the current grid $h$, e.g. Jacobi or Gauss-Seidel
2. Compute the residual
$$\underline{r}^{(h)} = \underline{b} - \underline{\underline{A}}\underline{\tilde{x}}^{(h)} \tag{83}$$
3. Restrict the residual to the coarser mesh
$$\underline{r}^{(2h)} = \underline{\underline{I}}_h^{2h}\underline{r}^{(h)} \tag{84}$$
4. Solve for the error on the coarser mesh
$$\underline{\underline{A}}^{(2h)}\underline{e}^{(2h)} = \underline{r}^{(2h)}, \quad \text{starting guess } \underline{\tilde{e}}^{(2h)} = \underline{0} \tag{85}$$

5. Correct $\underline{\tilde{x}}^{(h)}$ on the finer mesh using the prolonged error $\underline{e}^{(h)} = \underline{\underline{I}}^{h}_{2h}\underline{e}^{2h}$

$$\underline{\tilde{x}}'^{(h)} = \underline{\tilde{x}}^{(h)} + \underline{e}^{(h)} \tag{86}$$

6. Further iterative *relaxation* step on the fine mesh

**How to solve for the error on the coarse mesh?:** Step 4 can be performed by recursively calling $CG(\underline{\tilde{x}}^{(2^i h)}, \underline{b}^{(2^i h)}), i \geq 1$ until a coarseness is reached where one can easily exactly solve or solve by multiple relaxation steps.

**How to find $\underline{\underline{A}}^{(2h)}$ on the coarse mesh?:** Generally in the **Galerkin coarse grid approximation** one defines

$$\underline{\underline{A}}^{(2h)} = \underline{\underline{I}}^{2h}_{h}\underline{\underline{A}}^{(h)}\underline{\underline{I}}^{h}_{2h} \tag{87}$$

which is an additional matrix operation that might enlarge the stencil (and so the computational cost), depending on the interpolation operator.

When $\underline{\underline{A}}$ was brought forth by formulating a discretization in matrix form, we can use the same discrete equations as one the fine grid (**direct method**). In other words, the same stencil (*Schablone*) is used to go over the points and collect for the linear relations, e.g. for the 2D-Poisson - no matter the coarseness - we would construct $\underline{\underline{A}}$ based on eq. 69.

#### 3.7.2.3 V-cycle

The 6-step iteration process with a recursion call in step 4 leads to a cycle as illustrated in figure 15.

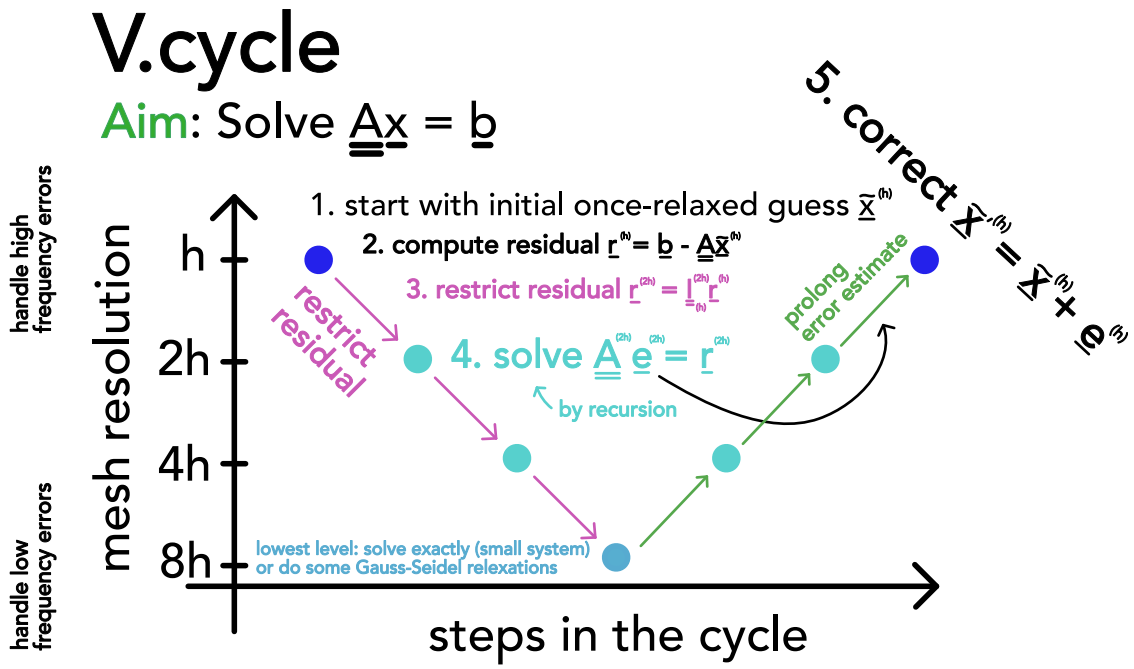



Figure 15: V-cycle.

The V-cycle for the Poisson equation has the same cost as FFT-based methods but requires less thread communication when parallelized.

**Computational costs**

$$\text{computational cost per V-cycle:} \mathcal{O}(N_{\text{grid}})$$
$$\text{computational cost until convergence to machine error (mult. cycles) } \mathcal{O}(N_{\text{grid}} \log N_{\text{grid}})$$
$$\text{with } N_{\text{grid}} \text{ being the number of grid-cells on the fine grid} \tag{88}$$

#### 3.7.2.4 Full multigrid method

**Problem:** How to make a good initial guess $\tilde{\underline{x}}^{(h)}$ (if not available e.g. by taking the result from the last time-step in a simulation)?

**Idea:** First solve on the coarsest grid, then interpolate up to get a good initial guess. Based on this idea, the full multigrid cycle is constructed.

The multigrid cycle is illustrated in figure 16.

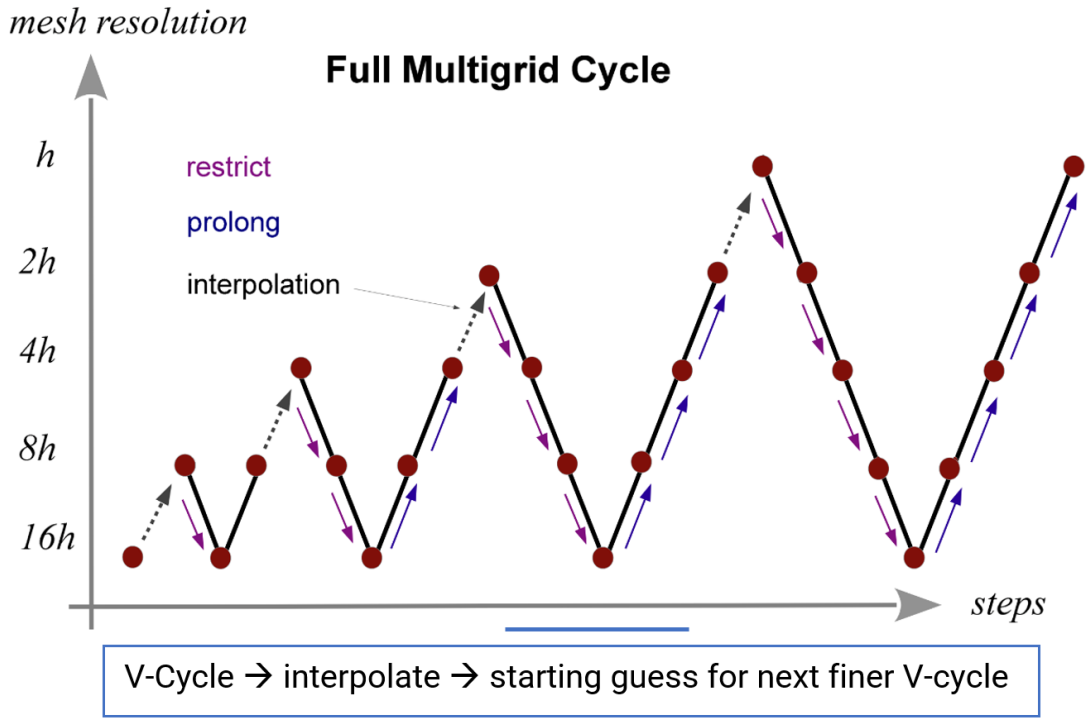



Figure 16: Full multigrid cycle.

The steps are

1. Initialize the righthand side on all grid levels $\underline{b}^{(h)}, \underline{b}^{(2h)}, \underline{b}^{(H)}$.
2. Solve $A^{(H)}\underline{x}^{(H)} = \underline{b}^{(H)}$ exactly on the coarsest level $H$.
3. Obtain an initial guess for a finer grid by interpolating the solution from the next coarser level below, i.e. $\underline{\tilde{x}}^{(h)} = \underline{\underline{I}}^{h}_{=2h}\underline{\tilde{x}}^{(2h)}$
4. Solve the problem in the finer level using this starting guess $\underline{\tilde{x}}^{(h)}$ and one v-cycle.
5. repeat step 3 until finest level is reached

**Computational cost of one multigrid cycle:** As the V-cycle it has $\mathcal{O}(N_{\text{grid}})$ as $N_{\text{grid}}$ is large compared to the cost of the V-cycle hierarchy.

## 3.8 Krylow subspace methods

Our aim still is to solve a linear system $\underline{\underline{A}}\underline{x} = \underline{b}$.

**Problem:** LU-decomposition has cubic runtime ($\sim \frac{2}{3}n^3$) and the previously discussed iterative methods might also not always be optimal.

### 3.8.1 Motivation for the need of Krylow subspace methods in the context of non-linear root-finding of big systems*

Consider a non-linear equation

$$\underline{0} = \underline{g}(\underline{\xi}) \tag{89}$$

which we want to solve for $\underline{\xi}$.[4] A common root-finding approach is Newton-Iteration[5]

$$\underline{\xi}_{k+1} = \underline{\xi}_k - \underline{\underline{J}}_{\underline{g}}^{-1}(\underline{\xi}_k)\underline{g}(\underline{\xi}_k), \quad \text{Jacobian } \underline{\underline{J}}_{\underline{g}} \tag{91}$$

where at the lowest level we have to solve the linear equation

$$\underline{b} := \underline{g}(\underline{\xi}_k) = \underline{\underline{J}}_{\underline{g}}(\underline{\xi}_k - \underline{\xi}_{k+1}) = \underline{\underline{J}}_{\underline{g}}\underline{a} \tag{92}$$

for $\underline{a}$.

**Problem:** But what if the resulting Jacobian is too large to be stored?

**Idea:** A directional derivative (a Jacobian vector product[a]) can be calculated in one (forward) automatic differentiation pass - cheaply and storage efficient. So what if we could solve $\underline{\underline{J}}\underline{a} = \underline{b}$ only based on Jacobian vector products?

[a]

$$\underline{\underline{J}}\underline{v} = (\underline{\nabla} \cdot \underline{g})\underline{v} = \lim_{h \to 0} \frac{\underline{g}(\underline{x} + \underline{v}h) - \underline{g}(\underline{x})}{h}$$
$$\left( \text{as } (\underline{\underline{J}}\underline{v})_i = \sum_{j=1}^{n} \left(\underline{\underline{J}}_{\underline{g}}\right)_{ij} v_j = \sum_{j=1}^{n} \frac{\partial g_i}{\partial x_j} v_j = ((\underline{\nabla} \cdot \underline{g})\underline{v})_i \right) \tag{93}$$

### 3.8.2 Motivation | solving a system of linear equations using a gradient method

Consider $\underline{\underline{A}} \in \mathbb{R}^{N \times N}$ symmetric ($\underline{\underline{A}}^T = \underline{\underline{A}}$) and positive definite, so $\underline{x}^T \underline{\underline{A}} \underline{x} > 0 \forall \underline{x} \in \mathbb{K}^N \setminus \{\underline{0}\}$ (only positive eigenvalues). Then the following holds

$$\underline{\underline{A}}\underline{x} = \underline{b} \leftrightarrow \text{quadratic form } f(x) := \frac{1}{2}\underline{x}^T \underline{\underline{A}} \underline{x} - \underline{x}^T \underline{b} \text{ is minimal} \tag{94}$$

[4] For instance an implicit Euler step can be formulated as such a root finding problem.

$$\begin{aligned} &\text{implicit step } \underline{y}^{(n+1)} = \underline{y}^{(n)} + \Delta t \underline{f}\left(\underline{y}^{(n+1)}\right) \\ \rightarrow \quad &\text{root-finding-problem } \underline{0} = \underline{y}^{(n+1)} - \underline{y}^{(n)} - \Delta t \underline{f}\left(\underline{y}^{(n+1)}\right) =: \underline{g}(\underline{\xi}) \end{aligned} \tag{90}$$

[5] Which has quadratic convergence - as the method converges on the root, the difference between the root and the approximation is squared in each step.

(intuitively as the quadratic form for a positive definite matrix if parabolic) where we can now apply standard gradient descent (**steepest descent method**)

$$\underline{x}^{(n+1)} = \underline{x}^{(n)} + \alpha^{(n)}\underline{p}^{(n)}, \quad \underline{p}^{(n)} = -\underline{\nabla} f(x^{(n)}) = -(\underline{b} - \underline{\underline{A}}\underline{x}^{(n)}) \tag{95}$$

so we take steps along the steepest descent with some *learning rate* $\alpha^{(n)}$

**Note:** Quadratic forms for different kinds of matrices are illustrated in figure 17. For an indifinite matrix the steepest descent method or the later-introduces conjugate gradient method will fail, because the solution is a saddle point. For the more general case of even non-symmetric matrices, GMRES can be used.

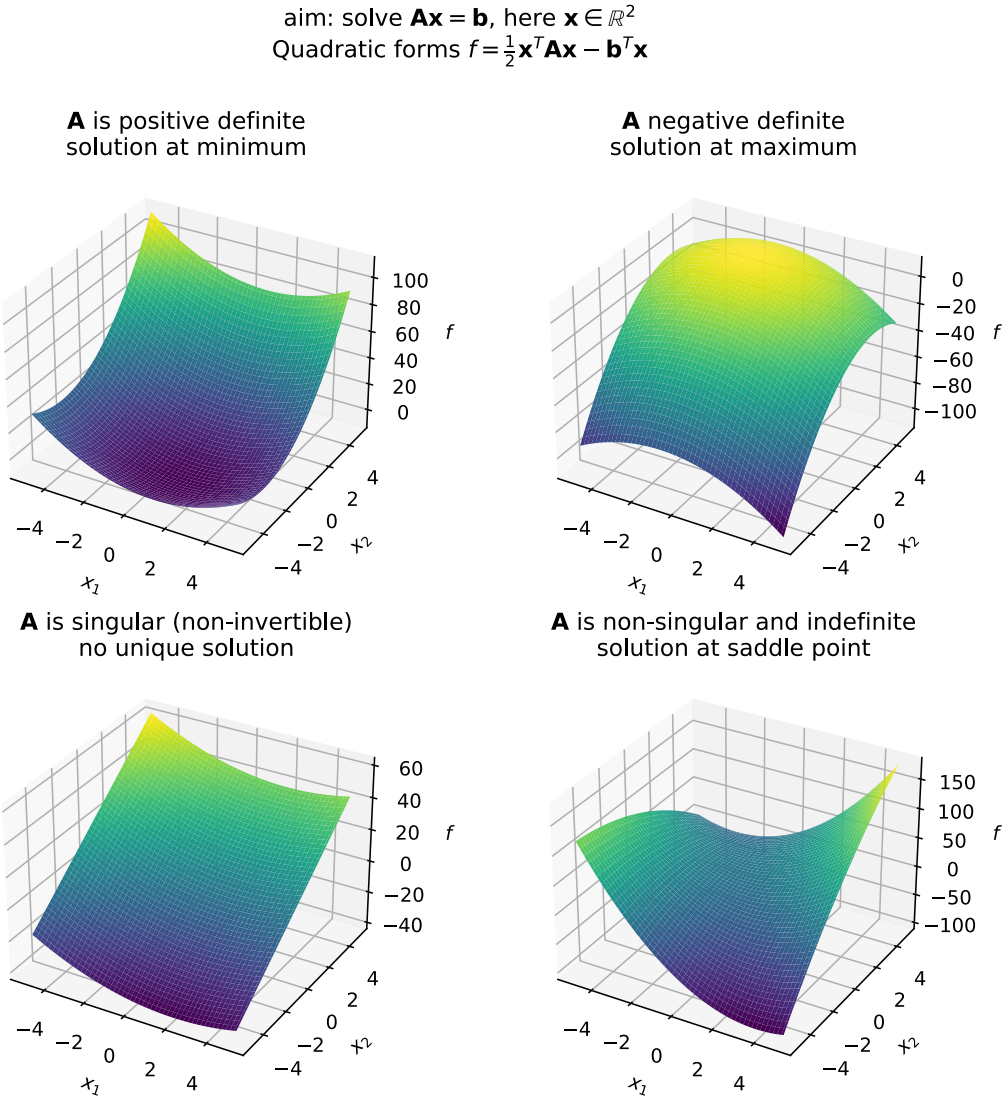



Figure 17: Quadratic forms for different kinds of matrices.

**Problem:** Steepest descent often takes steps in directions already taken. Steepest descent is good if our parabolic quadratic form $f$ is nearly circular but the more elliptic $f$ is, the more problematic steepest descent becomes, see figure 18.

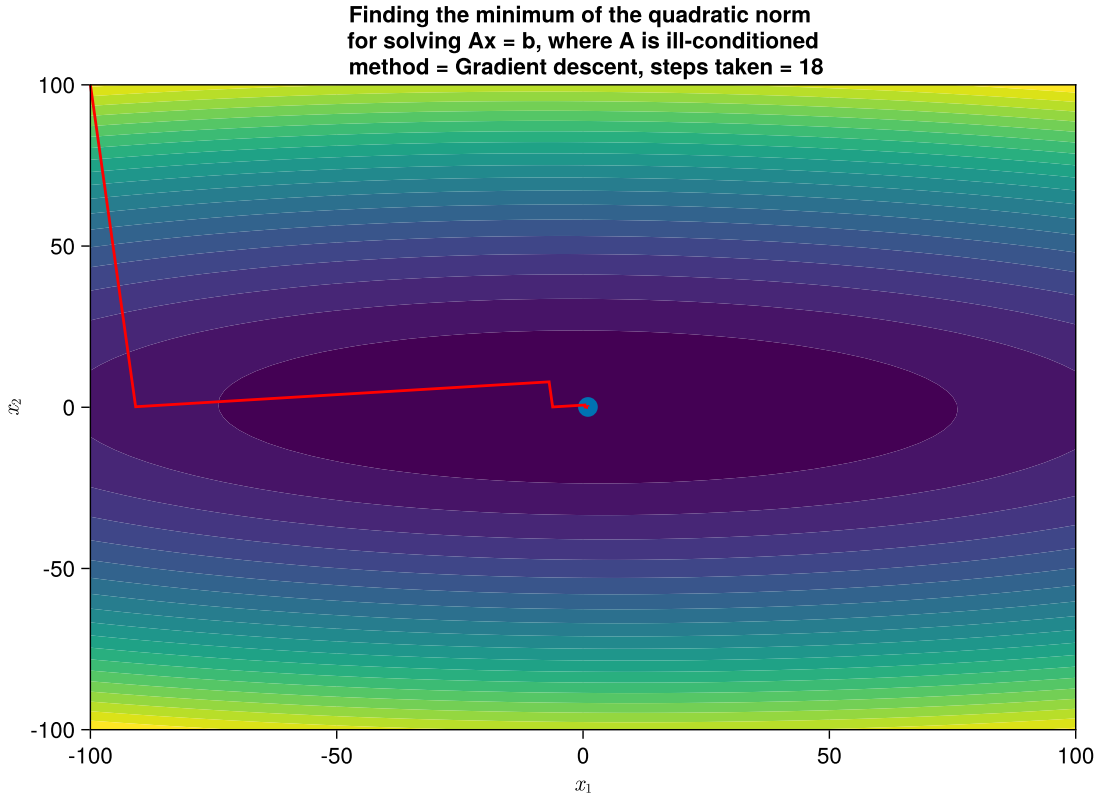



Figure 18: Steepest descent for soling a linear system $\underline{\underline{A}}\underline{x} = \underline{b}$ with positive definite matrix $\underline{\underline{A}}$.

**Idea:** Is there maybe a smarter way to choose directions $\underline{p}^{(n)}$ so that we do not go into the same direction multiple times and still find the solution?

We will introduce the conjugate gradient method, which takes at most $N$ (size of the linear system) steps to converge to the solution, see figure 19.

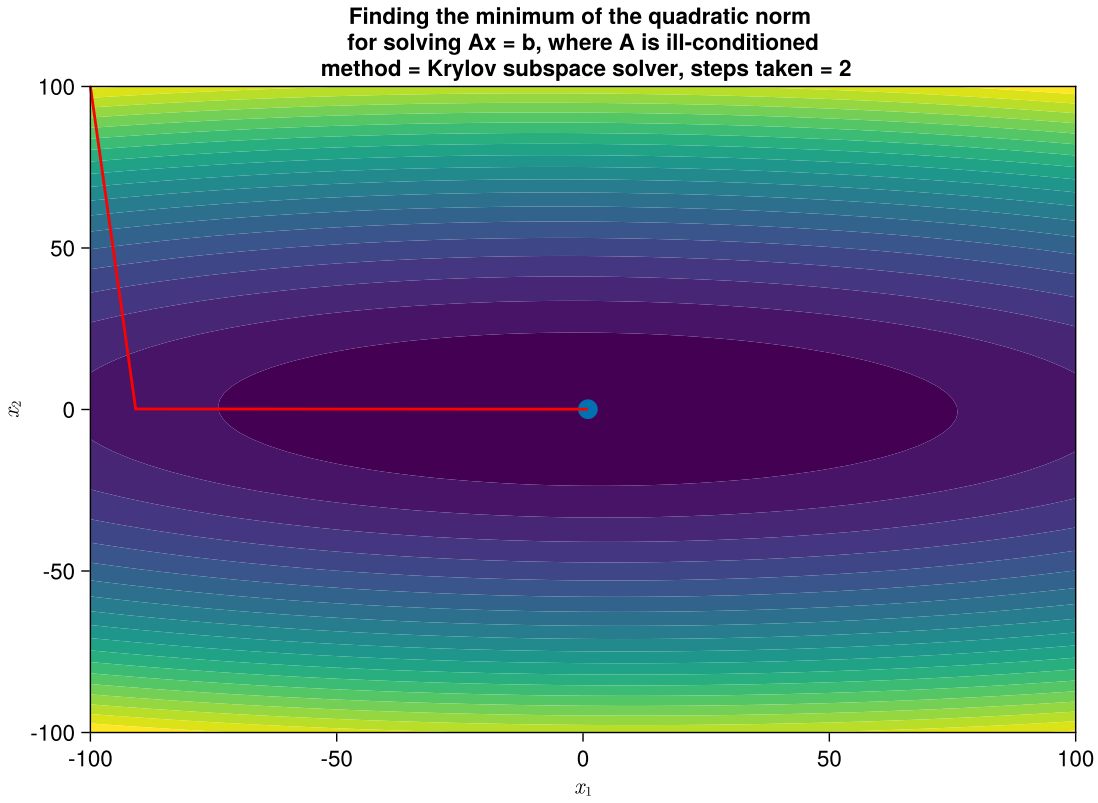



Figure 19: Conjugate gradient method for soling a linear system $\underline{\underline{A}}\underline{x} = \underline{b}$ with positive definite matrix $\underline{\underline{A}}$.

### 3.8.3 Krylov subspace and construction of iterative methods for solving $\underline{\underline{A}}\underline{x} = \underline{b}$

Consider the simple fixed-point iteration

$$(\underline{\underline{A}} + \underline{\underline{1}} - \underline{\underline{1}})\underline{x} = b \rightarrow \underline{x}^{(n+1)} = (\underline{\underline{1}} - \underline{\underline{A}})x^{(n)} + \underline{b} \tag{96}$$

then for $x^{(0)} = \underline{b}$

$$\underline{x}^{(1)} = 2\underline{b} - \underline{\underline{A}}\underline{b}, \quad \underline{x}^{(2)} = 3\underline{b} - 3\underline{\underline{A}}\underline{b} + \underline{\underline{A}}^2\underline{b}, \quad \dots \tag{97}$$

- it is only based on simple matrix vector products.

The Krylov subspace of order $r$ is based on

$$\underline{b}, \underline{\underline{A}}\underline{b}, \underline{\underline{A}}^2\underline{b}, \dots, \underline{\underline{A}}^{r-1}\underline{b} \tag{98}$$

forming a linear vector space

$$\mathcal{K}_r := \mathcal{K}_r(\underline{\underline{A}}, \underline{b}) = \text{span}(\underline{b}, \underline{\underline{A}}\underline{b}, \underline{\underline{A}}^2\underline{b}, \dots, \underline{\underline{A}}^{r-1}\underline{b}) \tag{99}$$

note that

- $\underline{\underline{A}}^i \underline{b}$ can be calculated by $i$ matrix-vector products (very efficient)
- $\underline{\underline{A}}^i \underline{b}$ are linearly independent for each $i < N$, the dimension of $\mathcal{K}_r$ increases with $r$ up to $N$, the dimension of $\underline{\underline{A}}$

How can we smartly (smarter than in the simple scheme above) choose $\underline{x}^{(n)} \in \mathcal{K}_n$ in an iterative scheme for solving $\underline{\underline{A}}\underline{x} = \underline{b}$?

- so that the residual $\underline{r}^{(n)} = \underline{b} - \underline{\underline{A}}\underline{x}^{(n)}$ is orthogonal to $\mathcal{K}_n$ (so $\in \mathcal{K}_{n+1}$) (**conjugate gradients**)
- so that the residual has minimum norm for $\underline{x}^{(n)} \in \mathcal{K}_n$ (GMRES)
- ...

**Note:** The best basis for the Krylov subspace $\mathcal{K}_r$ is orthonormal and can be computed using Arnoldi's method (essentialy Gram-Schmidt). Since $\underline{r}^{(n)} \perp \mathcal{K}_n$ it will be a scaled version of the last orthonormal base vector, one needs to get from $\mathcal{K}_n$ to $\mathcal{K}_{n+1}$

### 3.8.4 Conjugate gradients method

The sequence

$$\underline{x}^{(n+1)} = \underline{x}^{(n)} + \alpha^{(n)} \underline{p}^{(n)}, \quad \text{residual } \underline{r}^{(n)} = \underline{b} - \underline{\underline{A}}\underline{x}^{(n)} \tag{100}$$

with (without proof)

$$\underline{p}^{(n)} = \underline{r}^{(n)} + \frac{\underline{r}^{(n)} \cdot \underline{r}^{(n)}}{\underline{r}^{(n-1)} \underline{r}^{(n-1)}} \underline{p}^{(n-1)} \tag{101}$$

and *learning rate*[6]

$$\alpha^{(n)} = -\frac{\left(\underline{p}^{(n)}\right)^T \underline{r}^{(n)}}{\left(\underline{p}^{(n)}\right)^T \underline{\underline{A}}\underline{p}^{(n)}} \tag{102}$$

solves $\underline{\underline{A}}\underline{x} = \underline{b}, \underline{\underline{A}} \in \mathbb{R}^{N \times N}$, $\underline{\underline{A}}$ symmetric and positive definite, in at most $N$ steps, so $\underline{r}^{(N)} = \underline{0}$ (here stated without proof).

The directions taken are constructed so that

- the residuals are orthogonal, $\underline{r}^{(i)} \cdot \underline{r}^{(j)} = 0$ for $i \neq j$
- the step-directions are conjugate, $\left(\underline{p}^{(j)}\right)^T \underline{\underline{A}}\underline{p}^{(j)} = 0$ for $i \neq j$
- the residuals and step-directions are mutually orthogonal, $\underline{p}^{(i)} \cdot \underline{r}^{(j)} = 0$ for $i \neq j$

[6] This follows simply from $\partial_x f|_{\underline{x}^{(n)} + \alpha^{(n)} \underline{p}^{(n)}} \underset{!}{=} 0$, where $f$ is the quadratic form of $\underline{\underline{A}}\underline{x} = \underline{b}$.

# 4 Numerical Derivatives and Interpolation

We are interested in differentiating simulation outputs $\underline{U}(\underline{\theta})$ or an objective function $J(\underline{\theta})$ of these simulation outputs with respect to the simulation parameters $\underline{\theta}$, e.g. for sensitivity analysis or optimization tasks. These parameters might include the weights and biases of neural networks situated inside the simulator for error correction (Um et al., 2021) or capturing physical effects (e.g. Wang et al., 2023). Generally speaking, our simulator is the implementation of a numerical method for solving an ordinary differential equation (ODE) or partial differential equation (PDE).

There are essentially four ways for computing these derivatives:

- **finite differencing:** We can approximate derivatives based on finite differences, e.g. the forward difference

$$\partial_{\theta_i} U_j = \frac{U_j(\underline{\theta} + h\underline{e}_i) - U_j(\underline{\theta})}{h} + \mathcal{O}(h) \tag{103}$$

  or central difference

$$\partial_{\theta_i} U_j = \frac{U_j(\underline{\theta} + h\underline{e}_i) - U_j(\underline{\theta} - h\underline{e}_i)}{2h} + \mathcal{O}(h^2) \tag{104}$$

  with some $h > 0$. To calculate these numerical derivatives we only need to evaluate our simulator for different parameter choices. Finite differencing, however, has multiple problems. While we would like to make $h$ small to reduce the truncation error, e.g. $\mathcal{O}(h^2)$ in the central difference scheme, small $h$ will come at the cost of round-off errors due to limited floating point precision - so we have to strike a balance (see e.g. Fig. 3 in Baydin et al., 2018). While we only have to evaluate our simulator twice for forward or central differences of all simulation outputs with respect to one simulation parameter, computing each additional derivative with respect to a parameter requires one (forward differencing) or two (central differencing) additional simulations. This will be unfeasible in many-parameter settings such as fitting a neural network inside the simulator.

- **automatic differentiation (AD):** If the building blocks of our simulator are differentiable, we can differentiate through our simulator to machine precision by applying the chain rule, more specifically forward- or reverse-mode automatic differentiation (Baydin et al., 2018). These building blocks might be the primitive operations in a numerical library like `JAX` (Bradbury et al., 2018) or more monolithic numerical operators with hand-calculated derivatives (e.g. of an advection step in Thuerey et al., 2025, Sec. 13). While calculating the derivatives of a simulation outcome with respect to

multiple parameters is accurate, fast and simple using automatic differentiation, memory demand can be an issue (Kidger, 2021, Sec. 5.1.1.2). We will discuss automatic differentiation in greater detail below.

- **solving an adjoint problem:** The derivatives of interest can also be obtained by solving an adjoint equation of the PDE or ODE of the physical state. The state and adjoint system will generally differ, e.g. to calculate the derivative of the solution to an initial value problem to a PDE with a first order derivative in time, solving the adjoint problem amounts to integrating a final value problem backwards in time (Wilcox et al., 2013). The adjoint equation might be discretized independently of the state equation or derived based on the discretized state equation. While *adjoint methods* do not have the memory problems of automatic differentiation, deriving the adjoint system might be tedious, and numerically solving the adjoint system comes at additional computational cost and also introduces numerical errors to the derivatives calculated (see Sec. 5.1.2.3 and Example 5.6 in Kidger, 2021). The problem of inaccurate derivatives can be amplified if different discretizations of the state and adjoint system are chosen (Wilcox et al., 2013). In the following we will limit ourselves to the continuous adjoint method for ODEs, where an adjoint system is derived from the ODE system itself, not a discretization thereof. We limit ourselves to ODEs for two reasons: First, to make the introduction more approachable and second because PDEs with dependencies on space and time can be turned into a system of ODEs in time by discretizing over space (*method of lines*, Bradley, 2024, Sec. 2.1). For specific coverage on adjoints for linear hyperbolic PDEs (solved with a discontinuous Galerkin method), we refer to Sec. 2 in Wilcox et al., 2013.
- **Reversible solvers**: If the numerical solver is reversible, we can faithfully reconstruct the information necessary for the backward pass during the backward pass, but current reversible solvers are low-order and have poor stability properties (Kidger, 2021, Sec. 5.1.3).

Our fluid simulator, `jf1uids`, is differentiable based on automatic differentiation in `JAX`. We will therefore introduce automatic differentiation in the following. For reference, we will also present the continuous adjoint method and compare both at the example of optimizing parameters in an ODE.

## 4.1 Introducing Automatic Differentiation

Consider we are interested in the derivatives of a function $f : \mathbb{R}^n \to \mathbb{R}^m$, e.g. the simulation output as a function of the parameters.

Based on the chain rule and known derivatives of a set of primitive operations, we can efficiently and accurately (to machine precision) calculate these derivatives.

Intuitively

- forward-mode automatic differentiation forward propagates a perturbation $\underline{v} \in \mathbb{R}^n$ at $\underline{x}$ (with $\underline{v}$ sufficiently small so we can linearize $\underline{f}$ around $\underline{x}$) in the input space through the computational graph of the function $\underline{f}$ to a resulting perturbation $\underline{\underline{J}}_{\underline{f}}\Big|_{\underline{x}} \underline{v}$ in the output space
- backward-mode automatic differentiation relates output perturbations back to input perturbations, projecting perturbations in the output space $\underline{p} \in \mathbb{R}^m$ onto the output perturbations caused by the base vectors of the input space

$$\underline{\underline{J}}^T_{\underline{f}}\Big|_{\underline{x}} \underline{p} = \begin{pmatrix} - & (\partial_{x_1}\underline{f})^T & - \\ - & (\partial_{x_2}\underline{f})^T & - \\ & \vdots & \\ - & (\partial_{x_n}\underline{f})^T & - \end{pmatrix} \underline{p} = \begin{pmatrix} (\partial_{x_1}\underline{f})^T\underline{p} \\ (\partial_{x_2}\underline{f})^T\underline{p} \\ \vdots \\ (\partial_{x_n}\underline{f})^T\underline{p} \end{pmatrix} \tag{105}$$

for a scalar output and $p = 1$ this amounts to the derivatives of this output with respect to all inputs.

### 4.1.1 Forward-mode automatic differentiation

In a simulation we typically start at initial conditions $\underline{x} \in \mathbb{R}^N$ on which we apply a chain of transformations, e.g.

$$\underline{f} = \underline{g}_m \circ \cdots \circ \underline{g}_2 \circ \underline{g}_1, \quad \underline{f}(\underline{x}) = \underline{g}_m(\ldots \underline{g}_2(\underline{g}_1(\underline{x}))) \tag{106}$$

with for simplicity $\underline{f}, \underline{g}_m, \cdots, \underline{g}_1 : \mathbb{R}^N \to \mathbb{R}^N$. We might be interested in the sensitivity of the simulation output to a wiggle $\underline{v}$ in the initial conditions as captured by the Jacobian vector product (JVP) $\underline{\underline{J}}_{\underline{f}}(\underline{x})\underline{v}$ with $\underline{\underline{J}}_{\underline{f}}(\underline{x}) \in \mathbb{R}^{N\times N}$ if we linearize $\underline{f}$ around $\underline{x}$.

By the chain rule

$$\underline{\underline{J}}_{\underline{f}}(\underline{x}) = \underline{\underline{J}}_{\underline{g}_m}(\underline{g}_{m-1}, \cdots, g_1(\underline{x})) \cdots \underline{\underline{J}}_{\underline{g}_2}(\underline{g}_1(\underline{x}))\underline{\underline{J}}_{\underline{g}_1}(\underline{x}) \tag{107}$$

which we can calculate in $m-1$ matrix-matrix multiplications, each scaling with $\mathcal{O}(N^3)$ (naively) (or e.g. $\mathcal{O}(N^{\log_2 7})$ with Strassen's algorithm (Strassen, 1969)), leading to an overall

scaling of calculating the Jacobian of $\mathcal{O}(mN^3)$. But for a JVP we don't actually need to compute the Jacobian, we can just evaluate

$$\underline{\underline{J}}_{\underline{f}}(\underline{x})\underline{v} = \underline{\underline{J}}_{\underline{g}_m}(\underline{g}_{m-1}(\cdots(\underline{g}_1(\underline{x})))) \cdots \underline{\underline{J}}_{\underline{g}_2}(\underline{g}_1(\underline{x}))\underline{\underline{J}}_{\underline{g}_1}(\underline{x})\underline{v} \tag{108}$$

from right to left with $m-1$ matrix-vector products, each costing only $\mathcal{O}(N^2)$!

If we capture the dependency structure of a composition of functions as a computational graph, with functions as nodes and dependencies as arrows, calculating the JVP amounts to

- starting from the inputs evaluate the nodes to calculate the inputs to the next nodes (*primals*) in a forward pass
- in the same forward pass, starting at a *tangent* $\underline{v}$ in the input space calculate the JVPs of the nodes (evaluated at the *primals*), giving the *tangents* to the following nodes

Forward differentiating through computational graphs is commonly called forward-mode automatic differentiation[7], forwardpropagation, or, drawing from differential geometry terminology, pushforward.[8]

Partial derivatives $\partial_{x_i}\underline{f}$ are JVPs with the basis vectors, $\partial_{x_i}\underline{f} = \underline{\underline{J}}_{\underline{f}}\hat{e}_i$.

#### 4.1.1.1 Example of forward differentiation through a computational graph

Consider the following composition of functions

$$\underline{f}(\underline{x}) = \begin{pmatrix} g(h(x_1, x_2)) \\ u(h(x_1, x_2), x_2) \end{pmatrix} \tag{109}$$

with some functions $g : \mathbb{R} \to \mathbb{R}, h : \mathbb{R}^2 \to \mathbb{R}, u : \mathbb{R}^2 \to \mathbb{R}$. As $\underline{f}$ is a composition of functions, a chain of dependencies, it makes sense to illustrate it in a computational graph with functions as nodes and dependencies as arrows, as done in Fig. 20.

Consider a Jacobian-vector-product (JVP), here for our example function $\underline{f}$

[7] We will also use *autodiff* to refer to automatic differentiation.

[8] More details regarding the relation to differential geometry can be found under https://juliadiff.org/ChainRulesCore.jl/stable/maths/propagators.html.

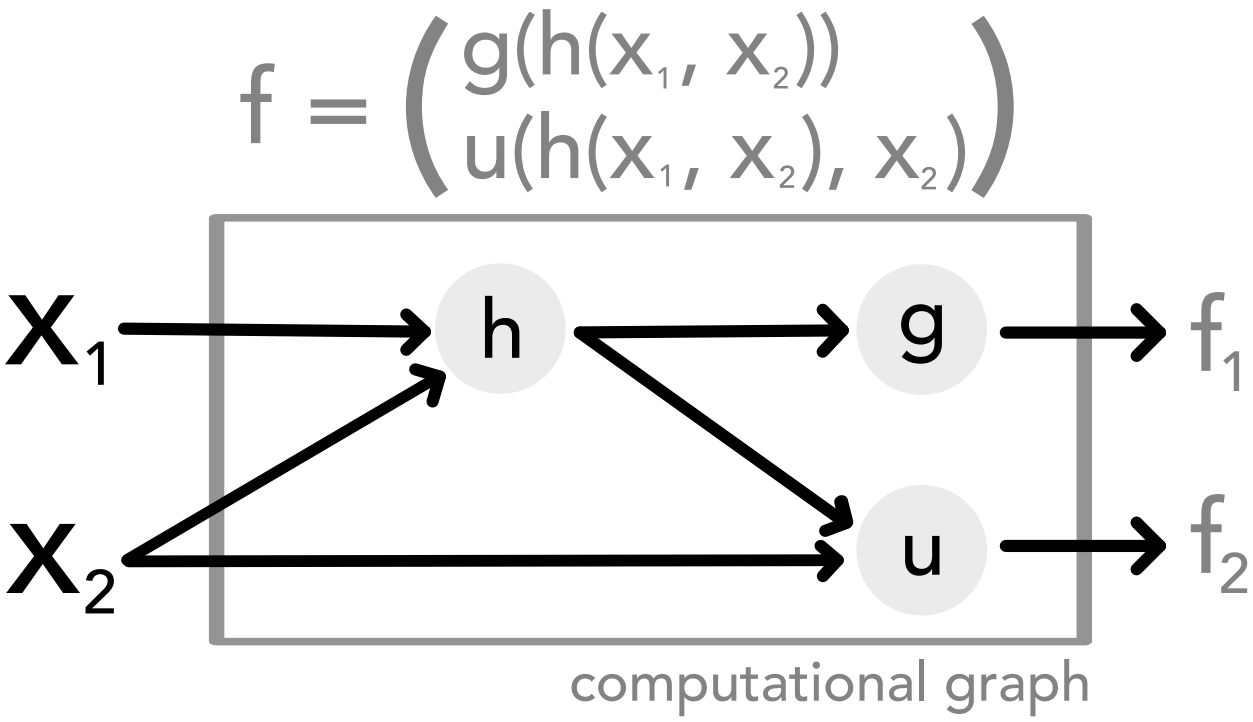



Figure 20: Illustrated is the computational graph of a function $\underline{f} : \mathbb{R}^2 \to \mathbb{R}^2$ (given in Eq. 109) which is composed of functions $g : \mathbb{R} \to \mathbb{R}, h : \mathbb{R}^2 \to \mathbb{R}, u : \mathbb{R}^2 \to \mathbb{R}$ depicted as nodes in the computational graph. Input relations are depicted as arrows.

$$\underline{\underline{J}}_{\underline{f}}\,\underline{v} = \begin{pmatrix} (\partial_1 h)\,(\partial_1 g) & (\partial_2 h)\,(\partial_1 g) \\ (\partial_1 h)\,(\partial_1 u) & (\partial_2 u) + (\partial_2 h)\,(\partial_1 u) \end{pmatrix} \underline{v} = \begin{pmatrix} (v_1\,\partial_1 h + v_2\,\partial_2 h)\,\partial_1 g \\ (v_1\,\partial_1 h + v_2\,\partial_2 h)\,\partial_1 u + v_2\,\partial_2 u \end{pmatrix} \tag{110}$$

where $\partial_i$ denotes the partial derivative of a function with respect to its $i$-th argument.

Calculating the JVP through the computational graph is illustrated in Fig. 21.

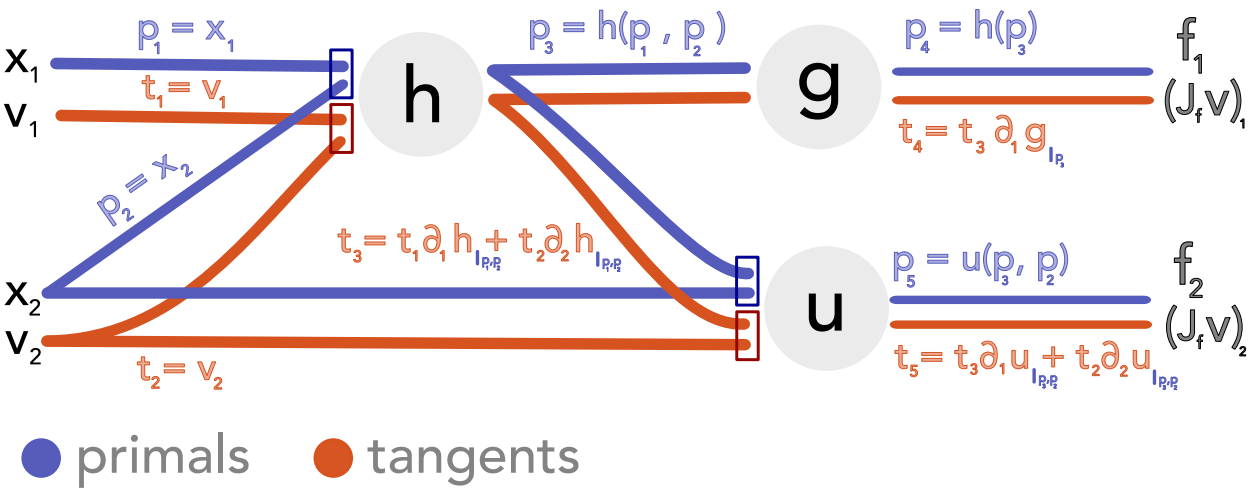



Figure 21: Illustrated is the forward differentiation through the function $\underline{f}$ (given in Eq. 109). The computational graph is evaluated from left to right, the tangent expressions correspond to the terms of the Jacobian-vector product as given in Eq. 110.

### 4.1.2 Reverse-mode automatic differentiation

Let us reconsider our simulation $\underline{f} = \underline{g}_m \circ \cdots \circ \underline{g}_2 \circ \underline{g}_1$ from before but we apply a scalar function $L : \mathbb{R}^N \to \mathbb{R}$ to the simulation output $\underline{f}(\underline{x})$, e.g. a loss $L(\underline{f}(\underline{x})) = ||\underline{f}(\underline{x}) - \underline{y}_{exp}||^2$ to an experimental measurement $\underline{y}_{exp} \in \mathbb{R}^N$.

To optimize $L$, we are interested in the gradient $\underline{\nabla}_{\underline{x}} L$, which we could calculate in $N$ forward differentiations (for each base vector $\hat{\underline{e}}_i, i = 1, ..., N$).

Let us write out this gradient

$$\underline{\nabla}_{\underline{x}} L(\underline{f}(\underline{x})) = \underset{=\underline{f}}{J^T} \underline{\nabla}_{\underline{f}} L = \underset{=\underline{g}_1}{J^T}(\underline{x}) \underset{=\underline{g}_2}{J^T}(\underline{g}_1(\underline{x})) \cdots \underset{=\underline{g}_m}{J^T}(\underline{g}_{m-1}(\cdots(g_1(\underline{x})))) \underline{\nabla}_{\underline{f}} L \tag{111}$$

We see that calculating the gradient amounts to a Jacobian transpose vector product (J'VP). Similar to the forward-mode automatic differentiation rather than calculating the Jacobian matrix explicitly by matrix multiplication, we perform a chain of matrix vector products, here J'VPs. Note that the transpose has permuted the order of the Jacobians - we now move backwards through our computational graph.

To evaluate the Jacobians, we need the evaluations $\underline{g}_1, ...\underline{g}_{m-1}$ obtained in a *forward pass* through the computational graph. Therefore, **reverse-mode autodiff has memory complexity scaling linearly with the number of algorithm iterations**, which can become a bottleneck.

We can calculate the gradient of a scalar function in a single backward pass through the computational graph (which must be preceded by a forward pass).

If we capture the dependency structure of a composition of functions as a computational graph, with functions as nodes and dependencies as arrows, calculating the J'VP amounts to

- in a forward pass, evaluate all nodes, possibly precalculate the Jacobians of the nodes with respect to their inputs
- starting from the output space calculate the J'VP backwards through the computational graph

Backward differentiation through a computational graph like this is commonly called reverse-mode automatic differentiation, backpropagation, or, again drawing from differential geometry terminology, pullback (of a differential form).

#### 4.1.2.1 Example of backward differentiation through a computational graph

We consider the same example as before (Eq. 109). The J'VP reads

$$\underset{=}{J}_{\underline{f}}^{T}\underline{s} = \begin{pmatrix} (\partial_1 h)\,(\partial_1 g) & (\partial_2 h)\,(\partial_1 g) \\ (\partial_1 h)\,(\partial_1 u) & (\partial_2 u) + (\partial_2 h)\,(\partial_1 u) \end{pmatrix}^{T} \underline{s} = \begin{pmatrix} (\partial_1 h)\,((\partial_1 g)s_1 + (\partial_1 u)s_2) \\ (\partial_2 h)\,((\partial_1 g)s_1 + (\partial_1 u)s_2) + (\partial_2 u)s_2 \end{pmatrix} \tag{112}$$

which is illustrated on the computational graph in Fig. 22. Often, one considers the transpose of the J'VP $(\underset{=}{J}_{\underline{f}}^{T}\underline{s})^T = \underline{s}^T \underset{=}{J}_{\underline{f}}$ and calls it Vector-Jacobian Product (VJP).

#### 4.1.2.2 Checkpointing for memory efficiency

Consider we have created a computational graph of our simulation with primitive operations as nodes in the graph. To calculate the J'VPs in the backward pass, the *linearization points*, e.g. all inputs to (elementwise nonlinear) primitive operations, must be known - this saves space compared to the naive approach of storing the Jacobians of the primitive operations.

It can, however, be too costly memorywise to store or too costly computationally (with regards to the computational cost of memory access) to retrieve all these linearization points.

Alternatively, we might only store certain *checkpoints* and recompute intermediate linearization points, e.g. for a larger function we might only store its inputs and calculate the linearization points of its primitive operations by reevaluating the function in the backward

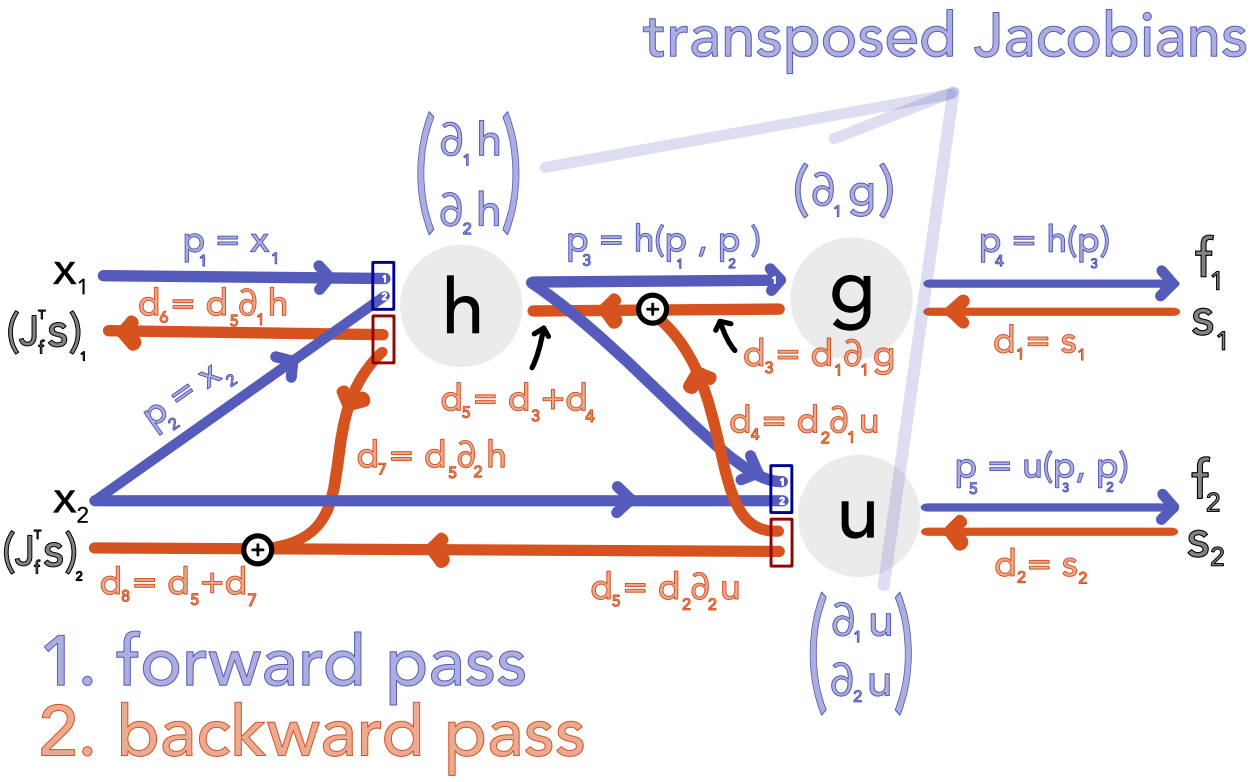



Figure 22: Backward differentiation illustrated for Eq. 109, the result from the backward pass corresponds to the Jacobian transpose vector product in Eq. 112.

pass as necessary.

**A note on optimal checkpointing:** Let us consider a simulation of $l = 6$ timesteps with states $x_0 \to x_1 \to \cdots \to x_6$ and assume we are able to store $c = 2$ checkpoints at a time. We might reasonably start with $x_0$ and $x_3$ checkpointed, but in the backward pass might want to change which state is checkpointed for the optimal checkpointing strategy. This is illustrated in Fig. 23, an adaptation of Fig. 2.1 in Stumm and Walther, 2010. But how do we choose initial checkpoints when the number of simulation steps is unknown a priori, i.e. in adaptive time-stepping with a while-loop? For this there exist optimal *online* checkpointing strategies, e.g. Algorithm 1 from Stumm and Walther, 2010.[a] The algorithmic details are beyond the scope of this thesis, but in Fig. 24 adapted from Fig. 2.2 in Stumm and Walther, 2010 we provide intuition on how one adapts which states are checkpointed while moving forward in the loop.

[a] An implementation in `JAX` by Patrick Kidger is available under `https://github.com/patrick-kidger/equinox/blob/main/equinox/internal/_loop/checkpointed.py`

More information on checkpointing in the context of `JAX` can be found under `https://docs.jax.dev/en/latest/_autosummary/jax.checkpoint.html`.

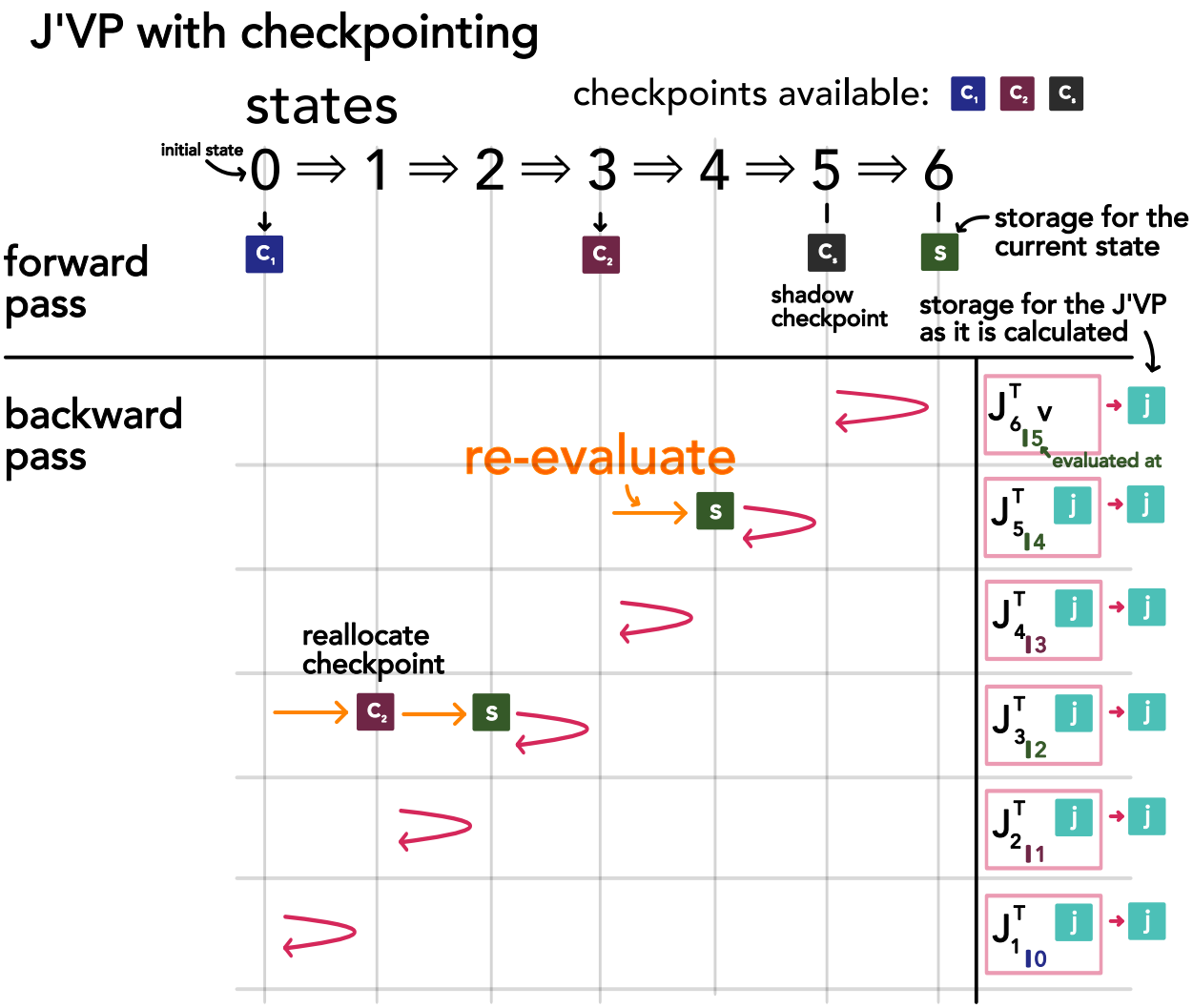



Figure 23: Illustrated is the calculation of a Jacobian transpose vector product (J'VP) using checkpointing through a loop with six iterations and with two checkpoints available. The illustration is based on Fig. 2.1 in Stumm and Walther, 2010. To calculate the derivative of a state in the sequence with respect to its inputs (i.e. the previous state), these inputs must be recalculated from checkpoints. We start out with state 0 and 3 checkpointed and state 5 stored as a *shadow checkpoint* (we assume that in the iteration we store the current and last state). With state 5 known we can differentiate state 6. In the next step we have to reconstruct state 4 from the checkpoint at state 3 to differentiate state 5 and so forth. Checkpoints can be reallocated to reduce the number of necessary recomputations.

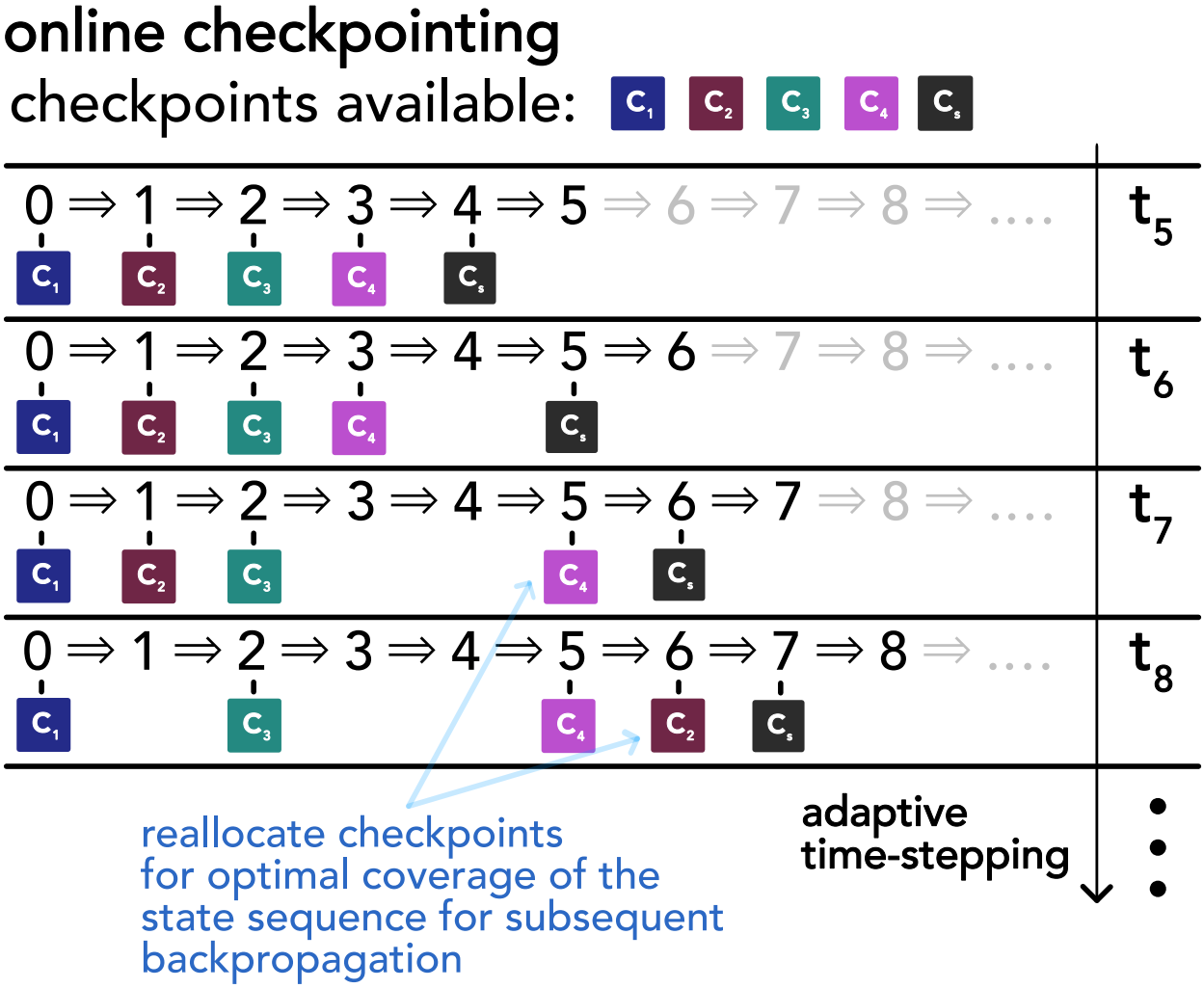



Figure 24: Illustrated is *online checkpointing* with four checkpoints available, i.e. in a loop with unknown number of iterations how do we reallocate the checkpoints as we go for optimal coverage of the state sequence for subsequent backpropagation. This illustration is based on Fig. 2.2 in Stumm and Walther, 2010. A shadow checkpoint $c_s$ always trails behind the current state. The figure is only meant to give a rough idea, for the details see Algorithm 1 in Stumm and Walther, 2010.

### 4.1.3 Example of custom differentiation rules: Implicit differentiation through a fixed-point iteration

There might be parts of our simulator for which we know differentiation rules so that we can avoid the memory and computational cost of differentiating through the primitive operations making up these parts.

To equip a function with custom rules for forward and backward differentiation, we need to define a custom JVP and J'VP, here its transpose the VJP, respectively.

As an example, consider our simulation contains a fixed point iteration, approximating $\underline{x}^\star(\underline{\theta})$ implicitly defined by

$$\underline{x}^\star(\underline{\theta}) = \underline{f}(\underline{x}^\star(\underline{\theta}), \underline{\theta}) \tag{113}$$

by iterating $\underline{x}_{t+1} = \underline{f}(\underline{x}_t, \underline{\theta})$ (assuming there exists one attracting fixed-point, which we usually show using Banach's fixed-point theorem).

By the chain rule (assuming $\underline{x}^\star(\underline{\theta}) = \underline{f}(\underline{x}^\star(\underline{\theta}), \underline{\theta})$ holds in a neighborhood around $\underline{\theta}_{eval}$ around which we want to differentiate)[9]

$$\underline{\underline{J}} = \partial_{\underline{\theta}} \underline{x}^\star(\underline{\theta}) = \underbrace{\partial_{\underline{\theta}} \underline{x}^\star(\underline{\theta})}_{=\underline{\underline{J}}} \underbrace{\partial_1 \underline{f}(\underline{x}^\star(\underline{\theta}), \underline{\theta})}_{=:\underline{\underline{A}}} + \underbrace{\partial_2 \underline{f}(\underline{x}^\star(\underline{\theta}), \underline{\theta})}_{=:\underline{\underline{B}}} \tag{114}$$

(where we have implicitly applied the implicit function theorem, see Blondel et al., 2022 for more details on this).

We can rewrite Eq. 114 to give an explicit expression

$$\underline{\underline{J}} = (\underline{\underline{1}} - \underline{\underline{A}})^{-1} \underline{\underline{B}} \tag{115}$$

The JVP can therefore be calculated by solving the linear system

$$(\underline{\underline{1}} - \underline{\underline{A}})(\underline{\underline{J}}\underline{v}) = \underline{\underline{B}}\underline{v} \tag{116}$$

and the VJP by

[9] The derivation follows `https://docs.jax.dev/en/latest/advanced-autodiff.html#example-implicit-function-differentiation-of-iterative-implementations`.

$$(\underline{\underline{1}} - \underline{\underline{A}})^T \underline{u} = \underline{v} \quad \rightarrow \quad \underline{v}^T \underline{\underline{J}} = \underbrace{\underline{v}^T (\underline{\underline{1}} - \underline{\underline{A}})^{-1}}_{=\underline{u}^T} \underline{\underline{B}} \tag{117}$$

where solving the linear systems might be done using GMRES (Saad and Schultz, 1986). While there are sophisticated techniques for inverting a matrix, the inverse can potentially be very dense even if the matrix itself is not, which would interfere with smart memory optimizations (Rackauckas et al., 2022).

For the VJP, we might also solve

$$(\underline{\underline{1}} - \underline{\underline{A}})^T \underline{u} = \underline{v} \quad \iff \quad \underline{u}^T = \underline{v}^T + \underline{u}^T \underline{\underline{A}} \tag{118}$$

using a fixed-point iteration.

As a simple example, based on Heron's root finding algorithm, we construct

$$\underline{f}(\underline{x}, \underline{\theta}) = \alpha \underline{x} + (1 - \alpha) \begin{pmatrix} \frac{1}{x_1} \exp(\theta_1^2 + 2\theta_2) \\ \frac{1}{x_2} \exp(2\theta_1 + \theta_2^2) \end{pmatrix} \tag{119}$$

with analytical solution

$$\underline{x}^\star(\underline{\theta}) = \begin{pmatrix} \exp(\frac{1}{2}\theta_1^2 + \theta_2) \\ \exp(\theta_1 + \frac{1}{2}\theta_2^2) \end{pmatrix} \tag{120}$$

where we use $\alpha = 0.8$ for slower convergence.[10]

[10]The fixed point method is based on

$$x = \alpha x + (1 - \alpha)\frac{S}{x} \iff x = \sqrt{S} \tag{121}$$

with $S > 0$, known as Heron's method for $\alpha = 0.5$. It converges with

$$e_{n+1} = x_{n+1} - \sqrt{S} \underbrace{=}_{x_n = \sqrt{S} + e_n} \alpha(\sqrt{S} + e_n) + (1 - \alpha)\frac{\sqrt{S}}{1 + \frac{e_n}{\sqrt{S}}} - \sqrt{S} \tag{122}$$

where approximating $\frac{1}{1+\frac{e_n}{\sqrt{S}}}$ to 2nd order around $e_n = 0$ to $1 - \frac{e_n}{\sqrt{S}} + \frac{e_n^2}{S}$ yields

$$e_{n+1} \approx (2\alpha - 1)e_n + (1 - \alpha)\frac{e_n^2}{\sqrt{S}} \tag{123}$$

so only for $\alpha = 0.5$ do we get quadratic convergence, for $\alpha \neq 0.5$ the linear term dominates and the error decreases by a factor $2\alpha - 1$ per iteration.

For $v = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ and $\underline{\theta}_{eval} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$ we compare the accuracy of the JVP and VJP compared to the ground thruth between a direct differentiation through the fixed-point iteration and the implicit differentiation for different numbers of iterations in Fig. 25 with the source code for the custom JVP given in Code-Snippet 3, and the custom VJP in Code-Snippet 4. The error of the JVP and VJP from direct automatic differentiation through the fixed-point iteration decreases with the same scaling as the fixed-point error itself, in the example around two orders of magnitude per 10 iterations. This is expected, the automatic differentiation calculates the accurate derivatives of our approximation of the fixed point to the parameters. Interestingly, the implicit differentiation, derived for the *true* fixed-points, has better error scaling behavior (roughly 4 orders of magnitude per 10 iterations). By design the reverse-mode automatic differentiation has a memory overhead compared to the implicit differentiation (scaling with the number of iterations) while the linear solve (or fixed-point iteration if one follows Eq. 118) in the implicit differentiation is a computational overhead compared to the backmode automatic differentiation (not scaling with the iterations but the number of parameters). We leave more in-depth profiling to future work.

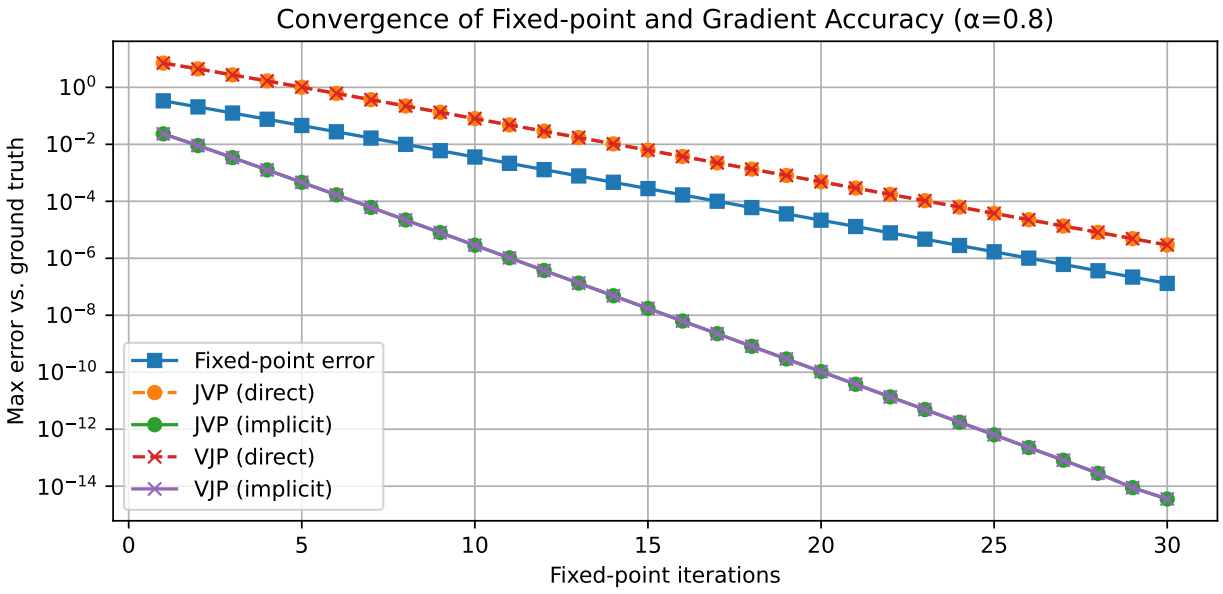



Figure 25: Our aim is to differentiate the fixed-point of Eq. 119 with respect to the parameters $\theta_1, \theta_2$ or rather calculate JVPs and VJPs. We compare direct automatic differentiation through the fixed-point iteration and implicit differentiation (evaluated at the current estimate of the fixed-point) to the ground truth (based on the analytical expression for the fixed-point, Eq. 120).

```
    @partial(custom_jvp, nondiff_argnums=(0, 3))
    def fixed_point_jvp(f, x_guess, theta, num_iters=100):
        def body(_, x): return f(x, theta)
        return fori_loop(0, num_iters, body, x_guess)

    @fixed_point_jvp.defjvp
    def fixed_point_jvp_rule(f, num_iters, primals, tangents):
        x_guess, theta = primals
        dx_guess, dtheta = tangents
        x_star = fixed_point_jvp(f, x_guess, theta, num_iters)
        A = jacobian(f, argnums=0)(x_star, theta)
        B = jacobian(f, argnums=1)(x_star, theta)
        dx = jnp.linalg.solve(jnp.eye(x_star.shape[0]) - A, B @ dtheta)
        # primal output, tangent output
        return x_star, dx
```

Code-Snippet 3: Implementation of a custom Jacobian vector product for the results of fixed-point iterations in JAX following Eq. 116

```
    @partial(custom_vjp, nondiff_argnums=(0, 3))
    def fixed_point_vjp(f, x_guess, theta, num_iters=100):
        def body(_, x): return f(x, theta)
        return fori_loop(0, num_iters, body, x_guess)

    def fixed_point_vjp_fwd(f, x_guess, theta, num_iters):
        x_star = fixed_point_vjp(f, x_guess, theta, num_iters)
        # primal output, stored variables for backward pass
        return x_star, (x_star, theta)

    def fixed_point_vjp_bwd(f, num_iters, res, v):
        x_star, theta = res
        A = jacobian(f, argnums=0)(x_star, theta)
        B = jacobian(f, argnums=1)(x_star, theta)
        u = jnp.linalg.solve((jnp.eye(x_star.shape[0]) - A).T, v)
        return jnp.zeros_like(x_star), u @ B

    fixed_point_vjp.defvjp(fixed_point_vjp_fwd, fixed_point_vjp_bwd)
```

Code-Snippet 4: Implementation of a custom vector Jacobian product for the results of fixed-point iterations in JAX following Eq. 117

## 4.2 Introducing the Continuous Adjoint Method for ODEs

We will now introduce the continuous adjoint method for differentiating the solution of ODEs, or rather a scalar function $L$ of the solution like a loss, and compare it to direct differentiation through the numerical solver in Sec. 4.3. Our treatment here follows Kidger, 2021.

Consider the initial value problem (IVP)

$$\underline{y}(0) = \underline{y}_0, \quad \frac{d\underline{y}}{dt} = \underline{f}_{\underline{\theta}}(t, \underline{y}(t)) \tag{124}$$

with

$$\underline{y}(0) \in \mathbb{R}^d, \quad \underline{\theta} \in \mathbb{R}^m, \quad \underline{f}_{\underline{\theta}} : [0,T] \times \mathbb{R}^d \to \mathbb{R}^d \tag{125}$$

where $\underline{y} : [0,T] \to \mathbb{R}^d$ is the solution to the IVP and $\underline{f}_{\underline{\theta}}$ is continuous in $t$ and uniformly Lipschitz and continuously differentiable in $\underline{y}$.

Our aim is to calculate $\frac{dL(\underline{y}(T))}{d\underline{\theta}}$, where $L : \mathbb{R}^d \to \mathbb{R}$ and $T$ is the time point of interest.

This can be achieved by solving the adjoint system (here of the *continuous* original problem, not its discretization)

$$\begin{aligned} \underline{a}_{\underline{y}}(T) &= \frac{dL}{d\underline{y}(T)}, \quad \frac{d\underline{a}_{\underline{y}}}{dt}(t) = -\underline{a}_{\underline{y}}(t)^T \underbrace{\frac{\partial \underline{f}_{\underline{\theta}}}{\partial \underline{y}}(t, \underline{y}(t))}_{\in \mathbb{R}^{d \times d}} \\ \underline{a}_{\underline{\theta}}(T) &= 0, \quad \frac{d\underline{a}_{\underline{\theta}}}{dt}(t) = -\underline{a}_{\underline{y}}(t)^T \underbrace{\frac{\partial \underline{f}_{\underline{\theta}}}{\partial \underline{\theta}}(t, \underline{y}(t))}_{\in \mathbb{R}^{d \times m}} \end{aligned} \tag{126}$$

with $\underline{a}_{\underline{y}} : [0,T] \to \mathbb{R}^d, \underline{a}_{\underline{\theta}} : [0,T] \to \mathbb{R}^m$ from $t = T$ to $t = 0$ backwards in time where

$$\boxed{\frac{dL(\underline{y}(T))}{d\underline{\theta}} = \underline{a}_{\underline{\theta}}(0)} \tag{127}$$

Note that to evaluate the Jacobians in Eq. 126 we need to know $\underline{y}(t)$ but to have a memory advantage over direct autodiff through the numerical solver, we cannot save the whole forward trajectory in the *forward pass*. Therefore, as we integrate $\underline{a}_{\underline{y}}$ and $\underline{a}_{\underline{\theta}}$ backwards in time starting from $t = T$, we obtain $\underline{y}(t)$ by integrating back from $y(T)$ (obtained in the forward pass). **Note, however, that the numerical integration backwards in time will not match the forward trajectory in general, except when the numerical solver itself is reversed and floating point errors are insignificant.** Also, the backward problem might generally be harder than the forward problem.

For a proof of the continuous adjoint method, we refer to Kidger, 2021, Sec. 5.1.2.1.

## 4.3 Application Example: AD through the solver vs. continuous adjoint method on exponential decay

Consider the ODE for exponential decay

$$\frac{dy}{dt}(t) = \lambda y(t), \quad \lambda < 0, \quad y(0) = y_0, \tag{128}$$

with analytical solution $y(t) = y_0 \exp(\lambda t)$ and analytical derivative $\partial_\lambda y = t y_0 \exp(\lambda t)$.

The adjoint system reads

$$\begin{aligned} a_y(T) &= 1, & \frac{da_y}{dt}(t) &= -a_y(t)\lambda \\ a_\lambda(T) &= 0, & \frac{da_\lambda}{dt}(t) &= -a_y(t)y(t) \end{aligned} \tag{129}$$

We will compare the following approaches for calculating $\partial_\lambda y$:

- direct automatic differentiation through the implicit Euler method with $y_{n+1} = y_n + \Delta t \lambda y_{n+1} \rightarrow y_{n+1} = \frac{y_n}{1-\Delta t\lambda}$
- direct automatic differentiation through dopri5[11]
- the continuous adjoint method with implicit Euler for the forward- and backsolve
- a discrete adjoint system of the implicit Euler method which we derive based on the

[11] All tests involving dopri5 were carried out using `diffrax`, `https://docs.kidger.site/diffrax/`.

backpropagation idea as illustrated in Fig. 26:

$$
\begin{aligned}
g(y,\lambda) = \frac{y}{1-\Delta t\lambda} \rightarrow y_N = \underbrace{(g \circ g \circ \cdots \circ g)}_{\times N}(y_0) \\
\rightarrow \partial_\lambda y_N = \sum_{n=0}^{N-1} \partial_\lambda g|_{y_n} \, a_{y_{n+1}} \text{ with } a_{y_n} = \partial_y g|_{y_n} \, a_{y_{n+1}}, a_{y_N} = 1 \\
\partial_\lambda g = \Delta t \frac{y}{(1-\Delta t\lambda)^2}, \quad \partial_y g = \frac{1}{1-\Delta t\lambda}
\end{aligned}
\tag{130}
$$

Let us introduce

$$
a_{\lambda_n} = a_{\lambda_{n+1}} + \partial_\lambda g|_{y_n} \, a_{y_{n+1}}, \quad a_{\lambda_N} = 0 \tag{131}
$$

such that $\partial_\lambda y_N = a_{\lambda_0}$. By manually reverting the implicit Euler step for this specific problem, we find the discrete adjoint system

$$
\begin{aligned}
y_n = y_{n+1} \cdot (1-\Delta t\lambda), \quad y_N \text{ from forward pass} \\
a_{y_n} = \frac{1}{1-\Delta t\lambda} a_{y_{n+1}}, a_{y_N} = 1 \\
a_{\lambda_n} = a_{\lambda_{n+1}} + \Delta t \frac{y_n}{(1-\Delta t\lambda)^2} a_{y_{n+1}} = a_{\lambda_{n+1}} + \Delta t a_{y_n} y_{n+1}, a_{\lambda_N} = 0
\end{aligned}
\tag{132}
$$

which we can recognize to be a specific discretization of the continuous adjoint equations, connecting the continuous adjoint method, backpropagation and reversible solvers.

- the continuous adjoint method with dopri5 for the forward- and backsolve

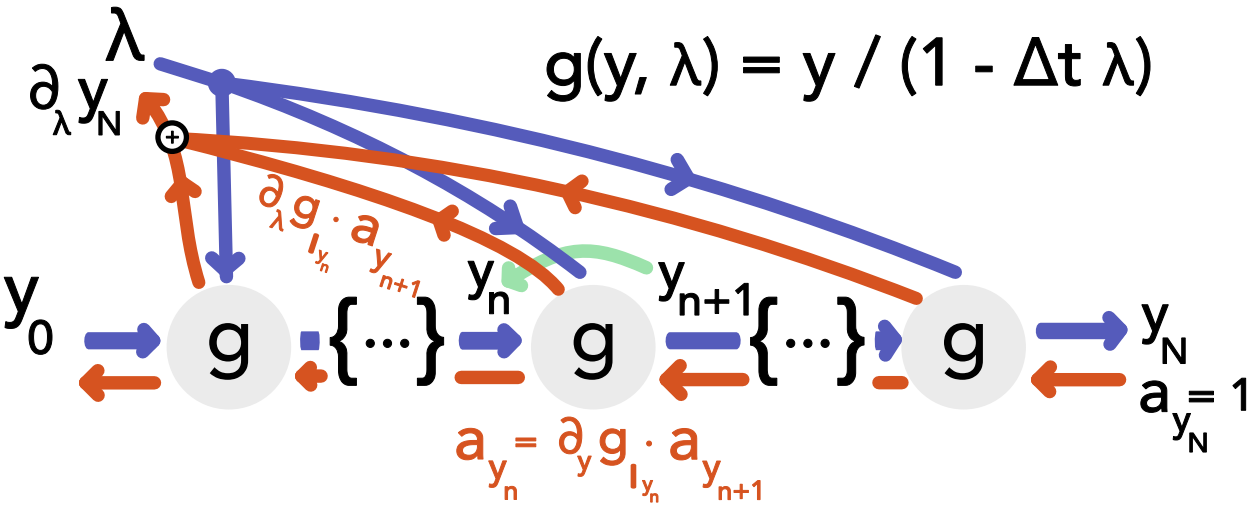



Figure 26: Illustration of backpropagation through the computational graph of an implicit Euler iteration to solve $\frac{dy}{dt}(t) = \lambda y(t), \lambda < 0, y(0) = y_0$, corresponding to Eq. 132.

An exemplary comparison is shown in Fig. 27 for $\Delta t = 0.3$ and $\lambda = -0.5$. The core

observations are:

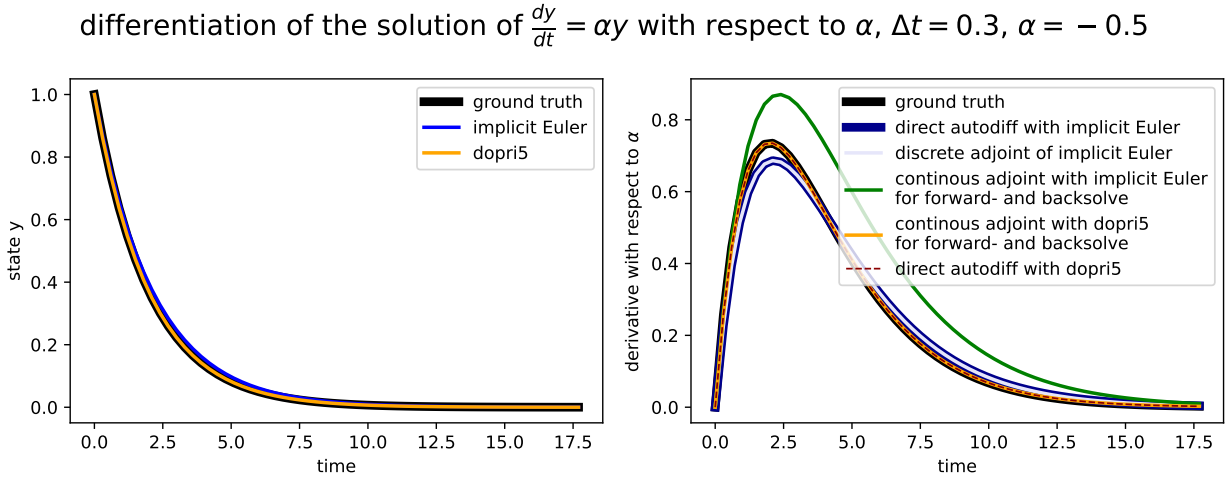



Figure 27: Comparison of different techniques for taking the derivative of the solution to an ODE with respect to its parameters for a simple exponential decay problem.

- the discrete adjoint and autodiff results match as expected
- the continuous adjoint method on implicit Euler, where the backward discretization is not an appropriate adjoint system to the forward discretization, yields the worst results
- dopri5 is sufficiently accurate such that both the direct autodiff and continuous adjoint nicely match the ground truth

# 5 Optimization

## 5.1 Overview on numerical methods for minimizing the loss (in the MLE setting)

The basic methods are to either numerically minimize the loss function directly, e.g.

- gradient descent

or to apply root-finding methods to the gradient of the loss function, in the MLE setting the gradient of the log-likelihood, **the score**

$$\underline{s}(\underline{\theta}) = \nabla_{\underline{\theta}} \log \mathcal{L}(\underline{\theta}) \tag{133}$$

with methods like

- Newton-Raphson
- Fisher scoring[12]

## 5.2 Gradient Descent

Here we just walk down the gradient of the loss function, starting at some initial guess $\underline{\theta}_0$, so

$$\begin{gathered} \underline{\hat{\theta}}^{(n+1)} = \underline{\hat{\theta}}^{(n)} - \gamma \underline{\nabla}_{\underline{\theta}} L \Big|_{\underline{\theta}=\underline{\hat{\theta}}^{(n)}}, \quad \text{learning rate } \gamma \\ \text{until } \left| L\left(\underline{\hat{\theta}}^{(n+1)}\right) - L\left(\underline{\hat{\theta}}^{(n)}\right) \right| < \epsilon \text{ or } \#\text{ steps } > N_{\max} \end{gathered} \tag{135}$$

where $\epsilon$ is a small number and $N_{\max}$ is the maximum number of steps, set by the user. Alternatively, one can stop when $||\underline{\theta}^{(n+1)} - \underline{\theta}^{(n)}|| < \epsilon$.

**Problems of standard gradient descent:**

- finds local minima
- how to choose the learning rate
- gets stuck on plateaus

[12] Like Newton-Raphson but with the Fisher information matrix in place of the Hessian.

$$\underline{\underline{I}}(\underline{\theta}) = E(\underline{s}\underline{s}^T) = -E\left(\frac{\partial^2 \log \mathcal{L}}{\partial \underline{\theta} \partial \underline{\theta}^T}\right) \tag{134}$$

(the negative expectation of the Hessian of the log-likelihood) (which is not a function of any particular observation as the random variable $X$ was averaged out).

### 5.2.1 Improvements to standard gradient descent

- to combat local minima, we can start from multiple initial conditions and take the best final result
- we prefer an approximate global optimum over an exact local one → use stochasticity, (e.g. injected noise → jump out of local but not global minima); simulated annealing (probabilistically move to some state or stay in the current one, even locally suboptimal steps are allowed)
- **stochastic gradient descent**: consider we want to minimize an additive function

$$Q(\underline{\theta}) = \frac{1}{N}\sum_{i=1}^{N} Q_i(\underline{\theta}), \quad \text{e.g.} L(\underline{\theta}) = -\mathcal{LL}(\underline{\theta}) = \sum_{i=1}^{N} -\log p(\underline{x}_i|\underline{\theta}) \tag{136}$$

  We do the gradient descent steps by

$$\underline{\hat{\theta}}^{(n+1)} = \underline{\hat{\theta}}^{(n)} - \frac{\gamma}{k}\underline{\nabla}_{\underline{\theta}} \sum_{\substack{\text{random subset} \\ \text{of size } k}} Q_k(\underline{\theta}) \tag{137}$$

  so we only consider the loss following from a subset of data points - which can avoid local minima (noise in the data itself is used)
- adaptive learning rage $\gamma$ (reduce for steep gradients) (e.g. AdaGrad, RMSProp, Adam)

## 5.3 Newton-Raphson

### 5.3.1 Standard Newton iteration for root finding

Consider we want to find the root of a function $g(\xi)$. We linearly approximate $g$ around the current guess $\xi_k$ and solve for the root.

$$\begin{aligned} g(\xi_{k+1}) &\approx g(\xi_k) + (\xi_{k+1} - \xi_k)\,\partial_\xi g|_{\xi=\xi_k} = 0 \\ &\rightarrow \xi_{k+1} = \xi_k - \frac{g(\xi_k)}{\partial_\xi g|_{\xi=\xi_k}} \end{aligned} \tag{138}$$

so in the multivariate case, $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$, we have

$$\underline{\xi}_{k+1} = \underline{\xi}_k - \underline{\underline{J}}_{\underline{g}}^{-1}\left(\underline{\xi}_k\right)\underline{g}\left(\underline{\xi}_k\right), \quad \underline{\underline{J}}\underline{g} = \begin{pmatrix} - & \underline{\nabla}_{\underline{\xi}} g_1 & - \\ \vdots & \vdots & \vdots \\ - & \underline{\nabla}_{\underline{\xi}} g_n & - \end{pmatrix} \tag{139}$$

Which is illustrated in figure 28.

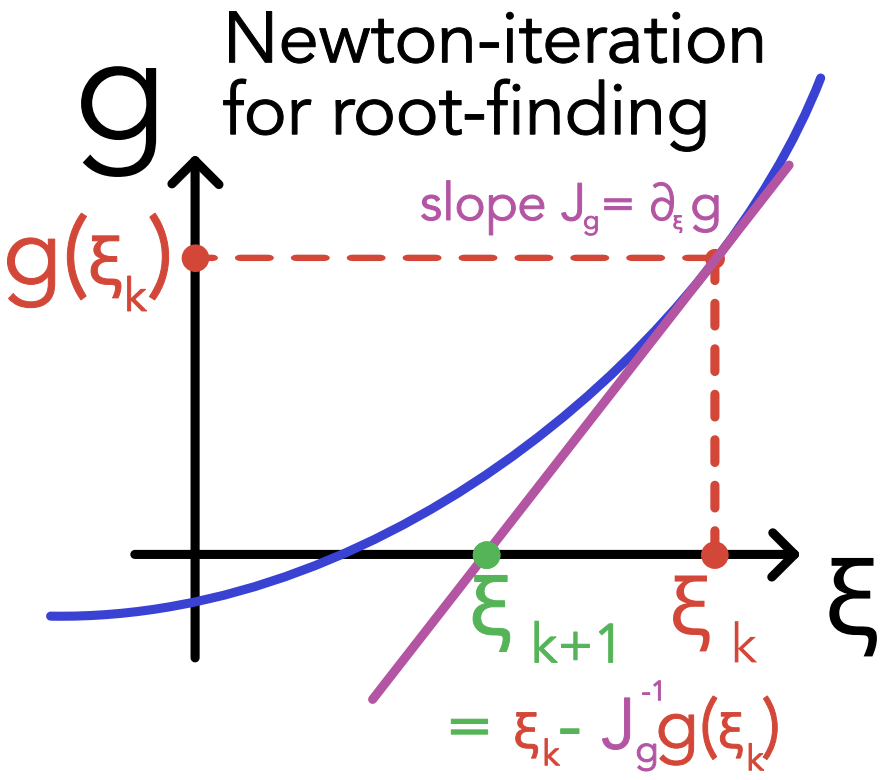



Figure 28: Illustration of the Newton-Raphson method.

### 5.3.2 Newton's method in optimization

We now want to find critical values $\xi_{\text{crit}}$ of a function $g$, where

$$\partial_\xi g|_{\xi_{\text{crit}}} = 0 \tag{140}$$

These can be minima, maxima or saddle points.

Using Newton iteration, we get

$$\xi_{k+1} = \xi_k - \frac{\partial_\xi g|_{\xi=\xi_k}}{\partial_\xi^2 g|_{\xi=\xi_k}} \tag{141}$$

**Intuition of Newton-optimization:** At $\xi_k$, a parabola is fitted to the function $g$ and we move the the extremum (in higher dimension this can be a saddle point) of it.

In higher dimensions $g : \mathbb{R}^n \to \mathbb{R}$, we get

$$\underline{\xi}_{k+1} = \underline{\xi}_k - \underline{\underline{H}}_g^{-1}(\underline{\xi}_k) \underline{\nabla}_{\underline{\xi}} g(\underline{\xi}_k) \tag{142}$$

with the numerically costly Hessian

$$\underline{\underline{H}}_g = \underline{\underline{J}}_{\underline{\nabla}_{\underline{\xi}} g} \tag{143}$$

best analytically found or calculated based on Automatic Differentiation (AD).

In our loss-optimization we simply have $g(\underline{\xi}) = L(\underline{\theta})$.

#### 5.3.2.1 Advantages of Newton optimization

- In the convex / concave setting the usage of curvature information by Newton iteration leads to faster convergation than gradient descent, as illustrated in figure 29.
- step-size is automatically adaptive
  - *small Hessian* $\rightarrow$ automatically larger steps
  - *large Hessian* $\rightarrow$ automatically smaller steps

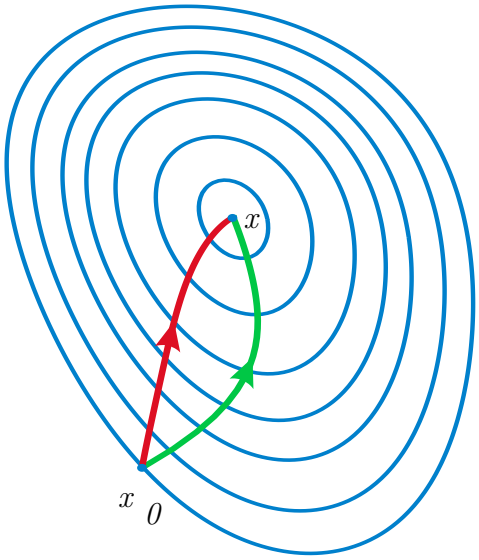



Figure 29: Newton optimization (green), gradient descent (red).

#### 5.3.2.2 Disadvantages of Newton optimization

- strictly only applies to convex / concave functions $g$
- at the inflection point of a non-convex function, the Hessian is indefinite so in the 1D case $\partial^2_\xi g = 0$ and Newton optimization breaks down
- it searches critical points, not minima, if $\partial^2_\xi g < 0$ and $\partial_\xi g > 0$, we go uphill, see figure 30.

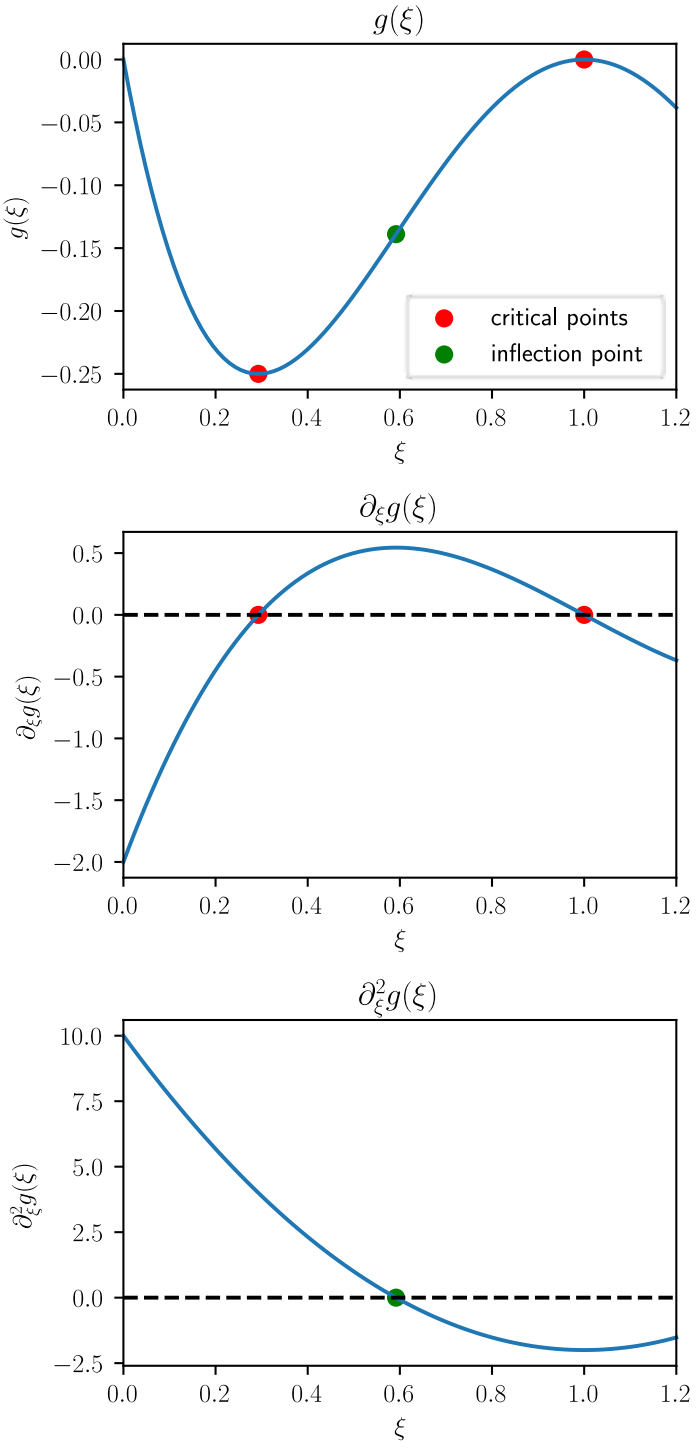



Figure 30: Newton optimization breaks down at the inflection point and goes uphill right of it.

# 6 Integration

## 6.1 Monte Carlo Integration

Classical numerical methods for integration (e.g. Simpson's rule) become infeasible for high-dimensional integrals, essentially, as the errors scale with the number of function-evaluation-points per dimension (so in high dimensions, we need to perform lots of function evaluations).

### 6.1.1 Intuition for Monte Carlo Integration

Consider we want to calculate the integral

$$I := \int_V g(\underline{x}) d^d \underline{x} \tag{144}$$

where $V$ is a $d$-dimensional volume and $g$ is a function $g : V \to \mathbb{R}$. Then it makes intuitive sense to approximate this integral by the mean of the function $g$ on the domain of interest, as found as the mean of $g$ evaluated at $N$ random points $\underline{x}_i$ in $V$, multiplied by the volume of $V$ (also denoted by $V$)

$$\hat{I}_N := \frac{V}{N} \sum_{i=1}^{N} g(\underline{x}_i) \xrightarrow[N\to\infty]{a.s.} I \tag{145}$$

**Monte Carlo Integration**: Let $V$ for simplicity be the hypercube $[0,1]^d$ (by change of variables, the domain of interest can be mapped to this hypercube). Then Monte Carlo integration to approximate $I = \int_{[0,1]^d} g(\underline{x}) d^d \underline{x}, \underline{x} \in \mathbb{R}^d$ is given by

- Generate $N$ random vectors $\underline{x}$ where each component is drawn uniformly from $[0,1]$ (so we need $N \times d$ random numbers)
- Evaluate

$$\hat{I}_N = V \frac{1}{N} \sum_{i=1}^{N} g(\underline{x}_i), \quad \text{here } V = 1 \tag{146}$$

**Scaling of the error**: The error of the Monte Carlo integrator scales with $1/\sqrt{N}$, independent of the number of dimensions of the integration $d$.

### 6.1.2 Connection to the Monte Carlo Estimator

We can also obtain the result from above from the Monte Carlo Estimator (eq. **??**) by using the uniform distribution

$$f(\underline{x}) = \begin{cases} \frac{1}{V} & \text{if } \underline{x} \in V \\ 0 & \text{else} \end{cases} \tag{147}$$

and rewriting the integral as

$$I = \int_V g(\underline{x}) d^d\underline{x} = V \int_V g(\underline{x}) \cdot \frac{1}{V} d^d\underline{x} = V \int_V g(\underline{x}) \cdot f(\underline{x}) d^d\underline{x} \tag{148}$$

### 6.1.3 Intuition from calculating the area of a shape

Consider an irregular shape with area $A_{\text{shape}}$ within a rectangle with area $A_{\text{rect}}$. The probability that a point drawn uniformly from the rectangle lies within the shape is

$$p_{\text{shape}} = \frac{A_{\text{shape}}}{A_{\text{rect}}} \tag{149}$$

so we can approximate

$$A_{\text{shape}} \approxeq A_{\text{rect}} \frac{\#\text{ hits in shape}}{\#\text{ hits in rectangle}} \tag{150}$$

which is the same as the Monte Carlo Integration introduced, where $g(\underline{x}) = 1_{\text{shape}}(\underline{x})$ and $V = A_{\text{rect}}$.

### 6.1.4 Error in Monte Carlo Integration - scaling with $\frac{1}{\sqrt{N}}$

Based on the connection to the Monte Carlo Estimator, the variance of our estimator for the integral is given by

$$\text{Var}[\hat{I}_N] = \frac{V^2}{N} \text{Var}[g(x)], \quad \text{estimate Var}[g(x)] \text{ by } S^2_{g;N} := \frac{1}{N-1} \sum_{i=1}^{N} \left( g(x_i) - \frac{\hat{I}_N}{V} \right)^2 \tag{151}$$

So the standard error of our Monte Carlo estimator

$$\hat{I}_N = \frac{V}{N}\sum_{i=1}^{N} g(\underline{x}_i) \tag{152}$$

that is if we calculate $\hat{I}_N$ multiple times with different random samples, the standard deviation of these different estimates is

$$\boxed{\sigma_{\hat{I}_N} = V\sqrt{\frac{\sigma_g^2}{N}} \propto \frac{1}{\sqrt{N}}} \tag{153}$$

which is illustrated in figure 31. This follows directly from the basic properties of the variance

$$\text{for } X, Y \text{ independent} \quad \mathrm{Var}[X+Y] = \mathrm{Var}[X] + \mathrm{Var}[Y], \quad \mathrm{Var}[aX] = a^2\,\mathrm{Var}[X] \tag{154}$$

with

$$\mathrm{Var}[\hat{I}_N] = \mathrm{Var}\left[\frac{V}{N}\sum_{i=1}^{N} g(\underline{x}_i)\right] = \frac{V^2}{N^2}\sum_{i=1}^{N}\mathrm{Var}[g(\underline{x}_i)] \underset{\underline{x}_i \text{ i.i.d.}}{=} \frac{V^2}{N}\mathrm{Var}[g(\underline{x})] \tag{155}$$

### 6.1.5 Distribution of $\hat{I}_N$ and derivation of the central limit theorem*

The Monte Carlo Estimator is

$$\hat{I}_N = \frac{V}{N}\sum_{i=1}^{N} g(\underline{x}_i) = \frac{1}{N}\sum_{i=1}^{N} y_i = \sum_{i=1}^{N} s_i, \quad y_i = V g(\underline{x}_i), \quad s_i = \frac{y_i}{N} \tag{156}$$

by basic properties of variance and mean

$$\begin{aligned} \langle \hat{I}_N \rangle &= N\langle s \rangle = \langle y \rangle \\ \mathrm{Var}[\hat{I}_N] &= N\,\mathrm{Var}[s] = \frac{1}{N}\mathrm{Var}[y] = \frac{V^2}{N}\mathrm{Var}[g(\underline{x})] \end{aligned} \tag{157}$$

**Note:** $p(y)$ is the distribution of $y$ over $V$ along the $y$-axis with

$$\langle y \rangle = \int y p(y)\,dy = \int_V g(\underline{x})\,d^d\underline{x}, \quad \int p(y)\,dy = 1 \tag{158}$$

$p(y)$ **might be any distribution.**

Applying the Central Limit Theorem (eq. **??**) to the Monte Carlo Estimator (assuming the

moments of $p(y)$ exist and are finite), we obtain

$$\begin{aligned} P_N(\hat{I}_N) &= \frac{1}{\sqrt{2\pi \operatorname{Var}[\hat{I}_N]}} \exp\left(-\frac{(\hat{I}_N - \langle \hat{I}_N \rangle)^2}{2 \operatorname{Var}[\hat{I}_N]}\right) \\ &= \frac{\sqrt{N}}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{N(\hat{I}_N - \langle y \rangle)^2}{2\sigma_y^2}\right) \end{aligned} \tag{159}$$

independent of the shape of $p(y)$ for $N$ sufficiently large.

**Derivation of the Central Limit Theorem by Fourier Transform** The central limit theorem can be derived by considering $P_N$ in Fourier space, where the central ingredient will simply be

$$\exp x = \lim_{N\to\infty} \left(1 + \frac{x}{N}\right)^N \tag{160}$$

which is also approximately true for large $N$.

**Distribution of the sum of independent random variables**: Consider **independent** variables $X, Y$ with PDFs $f_X$ and $f_Y$. The PDF of the sum $Z = X + Y$ is naturally the convolution

$$f_Z(z) = \int_{-\infty}^{\infty} \underbrace{f_X(x) f_Y(z-x)}_{x+(z-x)=z} \, dx \tag{161}$$

Another way to write this is

$$f_Z(z) = \int f_X(x) f_Y(y) \delta(z - (x+y)) \, dx \, dy \tag{162}$$

which one can easily see is equivalent to the convolution and also makes intuitive sense - the $\delta$-function ensures that the sum of $x$ and $y$ is $z$.

Analogously to the two-variable case, we write

$$P_N\left(\hat{I}_N\right) = \int \delta\left(\hat{I}_N - \sum_i \frac{y_i}{N}\right) p(y_1)\, p(y_2) \cdots p(y_N) \, \mathrm{d}y_1 \, \mathrm{d}y_1 \cdots \mathrm{d}y_N \tag{163}$$

Based on the Fourier transform and expansion of $p(y)$

$$\begin{aligned} \hat{p}(k) &= \int p(y) \exp\left(ik(y - \langle y \rangle)\right) \mathrm{d}y \\ &\underset{\text{Series Expansion}}{=} \int p(y) \left[1 + ik(y - \langle y \rangle) - \frac{k^2}{2}(y - \langle y \rangle)^2 + \cdots\right] \mathrm{d}y \\ &= 1 - \frac{k^2}{2}\sigma_y^2 + \cdots \end{aligned} \tag{164}$$

we find the Fourier transform of $P_N$ to be

$$\begin{aligned}
\hat{P}_N(k) &= \int P_N\left(\hat{I}_N\right) \exp\left(ik\left(I_N - \langle I_N\rangle\right)\right) \mathrm{d}I_N \\
&\underset{\langle \hat{I}_N\rangle = \langle y\rangle}{=} \int p\left(y_1\right)\cdots p\left(y_N\right) \exp\left(i\frac{k}{N}\left(y_1 - \langle y_1\rangle + y_2 - \langle y_2\rangle + \ldots + y_N - \langle y_N\rangle\right)\right) \mathrm{d}y_1 \cdots \mathrm{d}y_N \\
&= \left[\hat{p}\left(\frac{k}{N}\right)\right]^N \\
&\underset{\text{series expansion of } \hat{p}}{=} \left(1 - \frac{k^2\sigma_y^2}{2N^2}\right)^N \\
&\underset{N \text{ large}}{\approxeq} \exp\left(-\frac{k^2\sigma_y^2}{2N}\right)
\end{aligned} \tag{165}$$

where the inverse Fourier transform of $\hat{P}_N(k)$

$$P_N\left(\hat{I}_N\right) = \frac{1}{2\pi}\int \exp\left(-ik\left(I_N - \langle I_N\rangle\right)\right)\hat{P}_N(k)\mathrm{d}k \tag{166}$$

yields the previously stated result for $P_N\left(\hat{I}_N\right)$[13].

### 6.1.6 Illustrative Example of Monte Carlo Integration

Consider the integral

$$I = \int_0^1 g(x)dx, \quad g(x) = \exp\left(-\frac{x^2}{2}\right) \tag{167}$$

Our Monte Carlo estimator for this integral is given by

$$\hat{I}_N = \frac{1}{N}\sum_{i=1}^{N} g(x_i), \quad x_i \sim \mathcal{U}(0,1) \tag{168}$$

which is illustrated in figure 31. The higher the number of sampled points $N$ the lower the variance of our estimator; the lower the variance of $g$ along the $y$ axis, the lower the variance of our estimator.

[13]Insert the result for $\hat{P}_N(k)$, complete the square and use the normalization of the normal distribution or the Gaussian integral $\int \exp\left(-\alpha x^2\right)\mathrm{d}x = \sqrt{\frac{\pi}{\alpha}}$.

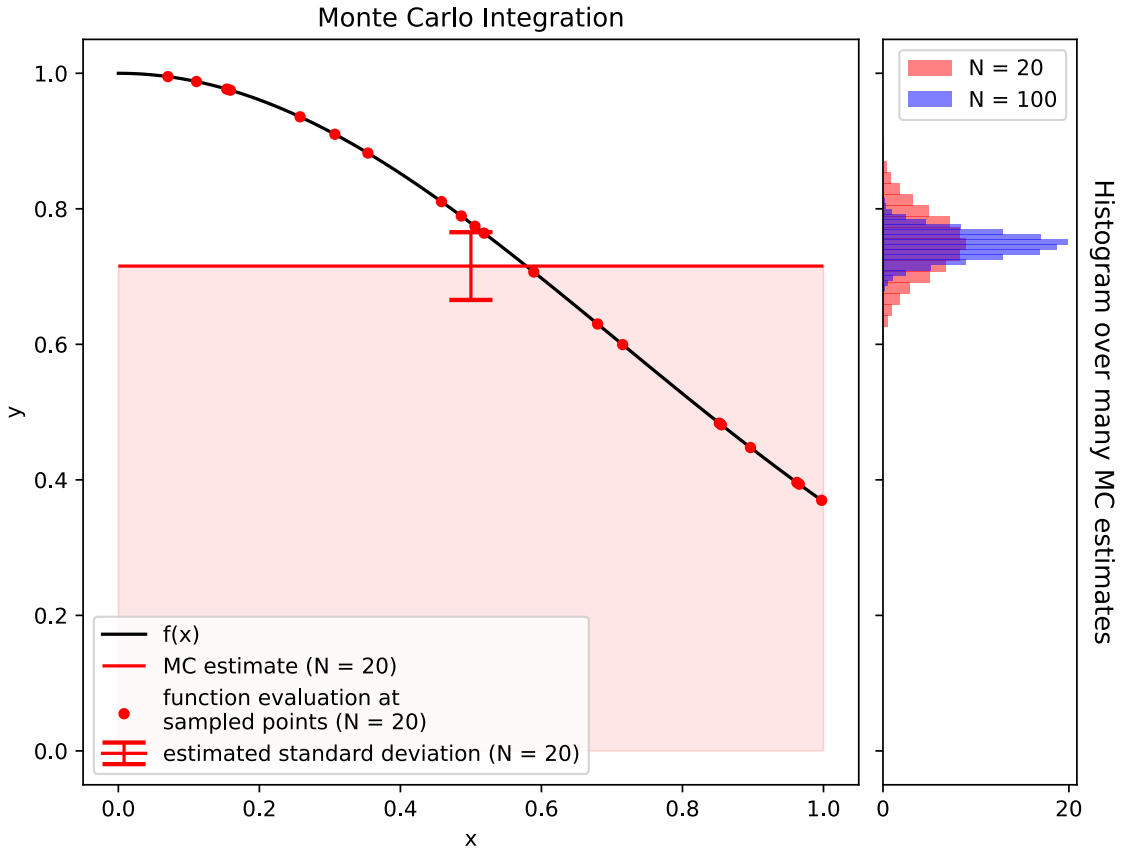



Figure 31: Illustration of Monte Carlo Integration for $I = \int_0^1 \exp\left(-\frac{x^2}{2}\right) dx$

### 6.1.7 Comparison to other techniques - when to use Monte Carlo integration?

**Error of standard methods** In a standard integration scheme like the midpoint rule or Simpsons rule[14] each dimension is divided into $n$ regularly space points, so that we have $N = n^d$ points in total. The error scales with

$$\text{midpoint or trapezoidal rule } \propto \frac{1}{n^2}, \quad \text{Simpson's rule } \propto \frac{1}{n^4} = \frac{1}{N^{\frac{4}{d}}} \tag{169}$$

In these *regular* methods, adding more points to the integration (increasing $N$) helps less and less the higher we go in dimension - e.g. for Simpson's rule err $\propto \frac{1}{N^{\frac{4}{d}}}$. For a standard method if we want 10 points per dimension with $d = 10$ we already need $10^{10}$ function evaluations - infeasible (e.g. high-dimensional integrations in Machine Learning).

[14] $\int_a^b f(x)dx \approx \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right]$ which can be derived by approximating the function $f$ by a quadratic polynomial and integrating this polynomial.

On the other hand in Monte Carlo integration the error scales with $\frac{1}{N^{\frac{1}{2}}}$ independent of the dimensionality $d$.

**Starting at what dimension of the integral $d$ will MC integration have a more favorable error scaling compared to Simpson's rule?** Setting $\frac{1}{N^{\frac{4}{d}}} = \frac{1}{N^{\frac{1}{2}}}$ yields that starting at $d = 8$ Monte Carlo Integration's error will reduce more quickly if we increase $N$ the number of points (/ function evaluations) used for the integration.

**Intuition for the better scaling** One intuition for this advantage of Monte Carlo is that the higher the dimension, the more evenly pairs of random points are spread with respect to their pair-wise distance - normally known as the curse of dimensionality, illustrated in figure 32.

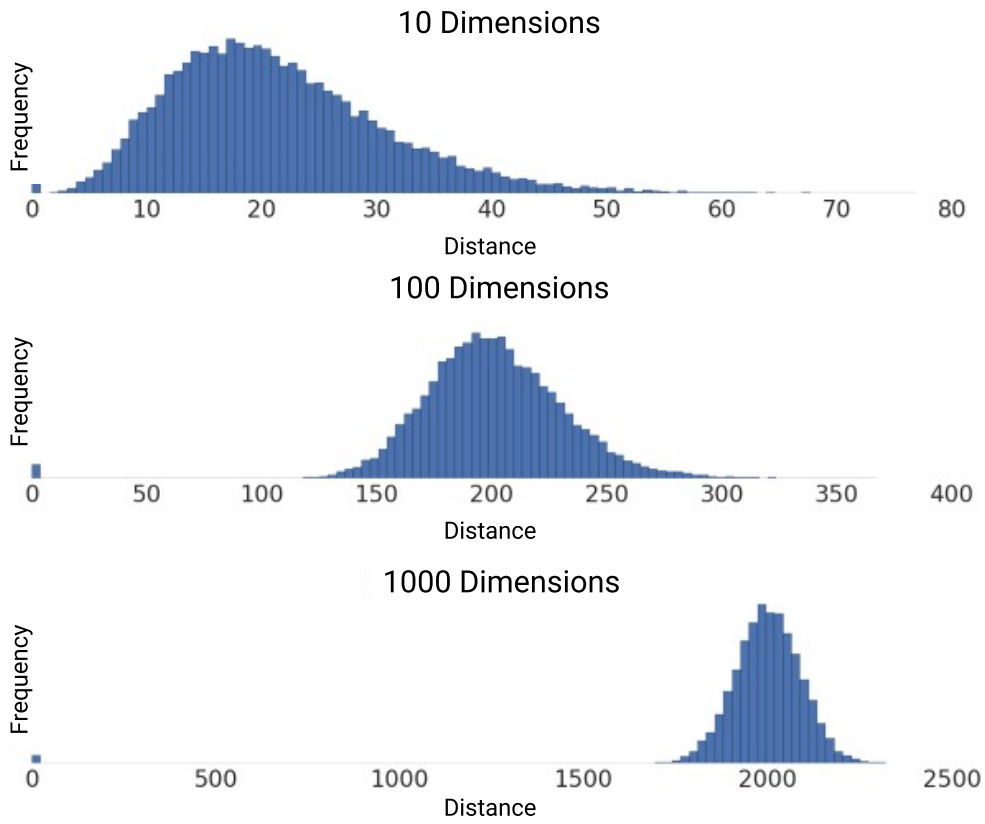



Figure 32: In higher dimensions, pairwise distances between random points show a sharper distribution around the mean distance (a more even distribution in space)

### 6.1.8 Reducing the variance of the Monte Carlo Estimator

#### 6.1.8.1 Antithetic Estimators*

Two random variables with the same distribution on the same probability space are called *antithetic*, if their covariance is negative. We can use such antithetic variables to reduce the variance of a Monte Carlo estimator.

This motivates the following ansatz

$$\begin{aligned} I &:= \int_0^1 g(x)dx, \quad \hat{I}_{anti} = \frac{\hat{I}_{N/2} + \tilde{I}_{N/2}}{2} \\ \hat{I}_{N/2} &= \frac{1}{N/2}\sum_{i=1}^{N/2} g(u_i), \quad \tilde{I}_{N/2} = \frac{1}{N/2}\sum_{i=1}^{N/2} g(1-u_i), \quad u_i \sim \mathcal{U}(0,1) \end{aligned} \tag{170}$$

with

$$\mathrm{Var}[\hat{I}_{anti}] = \frac{1}{4}\left(\mathrm{Var}[\hat{I}_{N/2}] + \mathrm{Var}[\tilde{I}_{N/2}] + 2\,\mathrm{Cov}[\hat{I}_{N/2}, \tilde{I}_{N/2}]\right) \underbrace{=}_{\text{see eq. 151}} \mathrm{Var}[\hat{I}_N] + \frac{1}{2}\,\mathrm{Cov}[\hat{I}_{N/2}, \tilde{I}_{N/2}] \tag{171}$$

where for $g$ continuous and monotonic, $\mathrm{Cov}[\hat{I}_{N/2}, \tilde{I}_{N/2}]$ is negative, as $U$ and $1-U$ with $U \sim \mathcal{U}(0,1)$ are negatively correlated. So we can reduce the variance compared to an estimator $\hat{I}_N$ using the same number of function evaluation and only half the number of random numbers.

Let us give an intuition why using antithetic evaluation points makes sense for estimating an integral of a monotonic function from 0 to 1. Using uniform antithetic variables we create pairs of points $(x_i, 1-x_i)$, which are symmetric around $x = 0.5$ and as of the monotony of the function $g$ we want to integrate, we obtain a better balance between sampling points where $g$ is large and where $g$ is small, reducing the variance of the estimate.

**Example of using Antithetic Estimators**

Consider the integral

$$I = \int_0^2 \frac{1}{1+x^2}dx \underbrace{=}_{\text{exact solution}} \arctan(2) \approx 1.107149 \tag{172}$$

Estimates based on the standard Monte Carlo estimator and the antithetic estimator are given in table 4 and illustrated in figure 33. A relative reduction in standard deviation of roughly 80% is achieved using the antithetic estimator.

| Estimator | Estimate | Standard Deviation | 95% Confidence Interval |
|---|---|---|---|
| Standard Monte Carlo | 1.096 | 0.017 | [1.0627 1.128] |
| Antithetic Monte Carlo | 1.106 | 0.0033 | [1.099 1.112] |

Table 4: Rounded estimates for the integral $\int_0^2 \frac{1}{1+x^2} dx$ using the standard Monte Carlo estimator and the antithetic Monte Carlo estimator for $N = 100$ samples with seed 1234

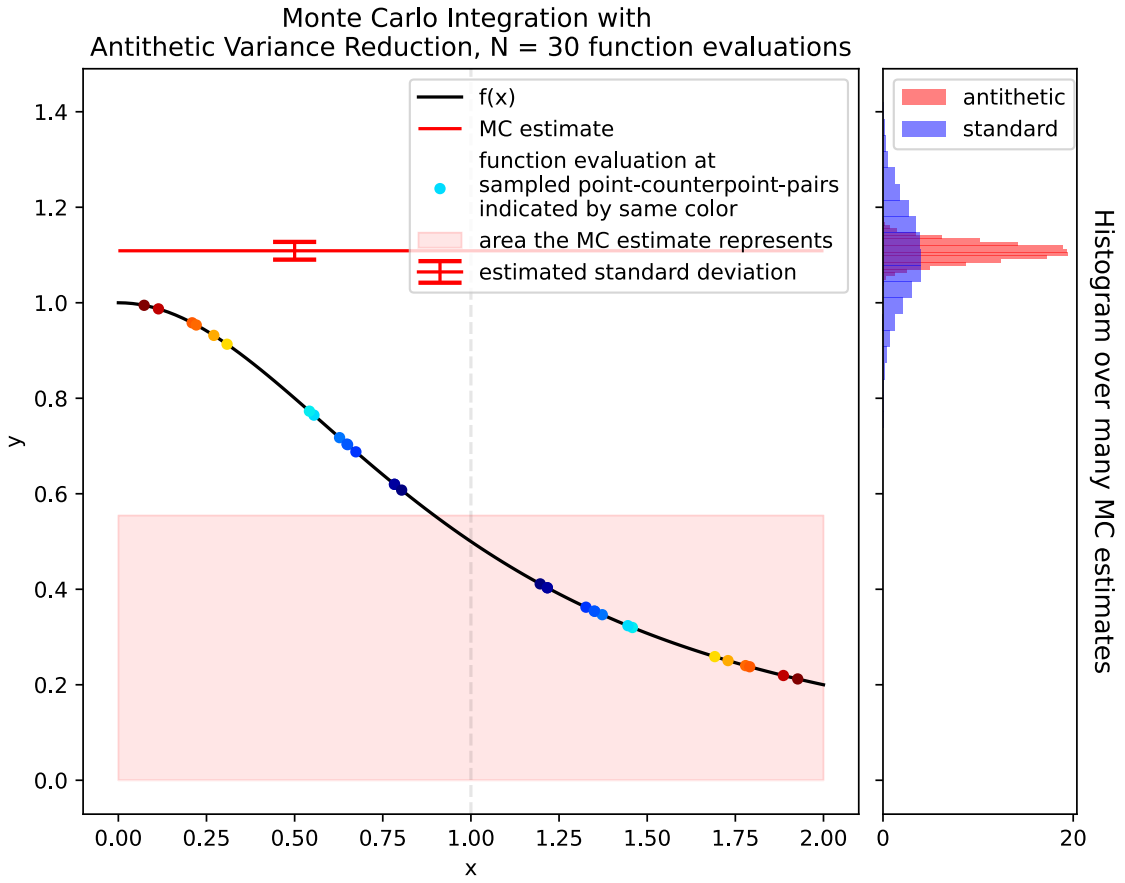



Figure 33: Illustration of Monte Carlo Integration with Antithetic Variables. Notice the even distribution of points around $\frac{a+b}{2} = 1$ by construction yielding an advantage over the standard Monte Carlo estimator.

#### 6.1.8.2 Importance Sampling

**Problem:** Consider we have a sharply peaked function $g(x)$ over the domain of integration. In this case using evaluation points sampled from the uniform will be inefficient.

**Importance sampling idea:** Preferably choose points where the integrand is large. This is done by sampling from a distribution $f(x)$ similarly shaped as $g(x)$ but from which we can easily sample (e.g. by direct inversion[a] or the rejection method).

[a]Note that for the inverse transform we need the inverted CDF, so we need to integrate the PDF. So choosing $f(x)$ itself as $g(x)$ is non-sense as we would need to integrate $f(x)$ to obtain the CDF.

The higher the variance of the output of the function $g(x)$ over the domain of integration, the higher the variance in our Monte Carlo Estimate for our integral.

$$I := \int_V g(x)dx = \int_V \frac{g(x)}{f(x)} f(x)dx, \quad I_N = \frac{1}{N}\sum_{i=1}^{N} \frac{g(x_i)}{f(x_i)} \tag{173}$$
$$\text{sample } \{x_i\}_{i=1,\dots,N} \text{ drawn from } f(x) \text{ limited to } V$$

Let us as an example tackle the integral

$$I = \int_0^1 \frac{4}{1+x^2}dx \underbrace{=}_{\text{exact solution}} \pi \tag{174}$$

using the importance sampling function $f(x) = \frac{4-2x}{3}$ on $0 \leq x \leq 1$. We can sample from $f(x)$ using the inverse transform method using the CDF $F(x) = \frac{4x-x^2}{3}$ (with $F(1) = 1$ as we want) so $F^{-1}(u) = 2 - \sqrt{4-3u}$. We can then simply use equation 173 to obtain an estimate.

The importance sampling approach compared to the standard Monte Carlo approach is illustrated in figure 34 and 35. There it is easy to see how effectively integrating a flatter function is to our advantage.

**Advantage of Importance Sampling**: As we can see in figure 34 the flatter $h(x) := \frac{g(x)}{f(x)}$ (best $h \propto g$ as achievable in lattice Monte Carlo) (mind the different naming in the figure) the sharper the probability distribution $p(h)$ (a limited range of $h$ values with higher probabilities), the smaller the Monte Carlo error $\sigma_{\hat{I}_N} = \sqrt{\frac{\sigma_h^2}{N}}$.

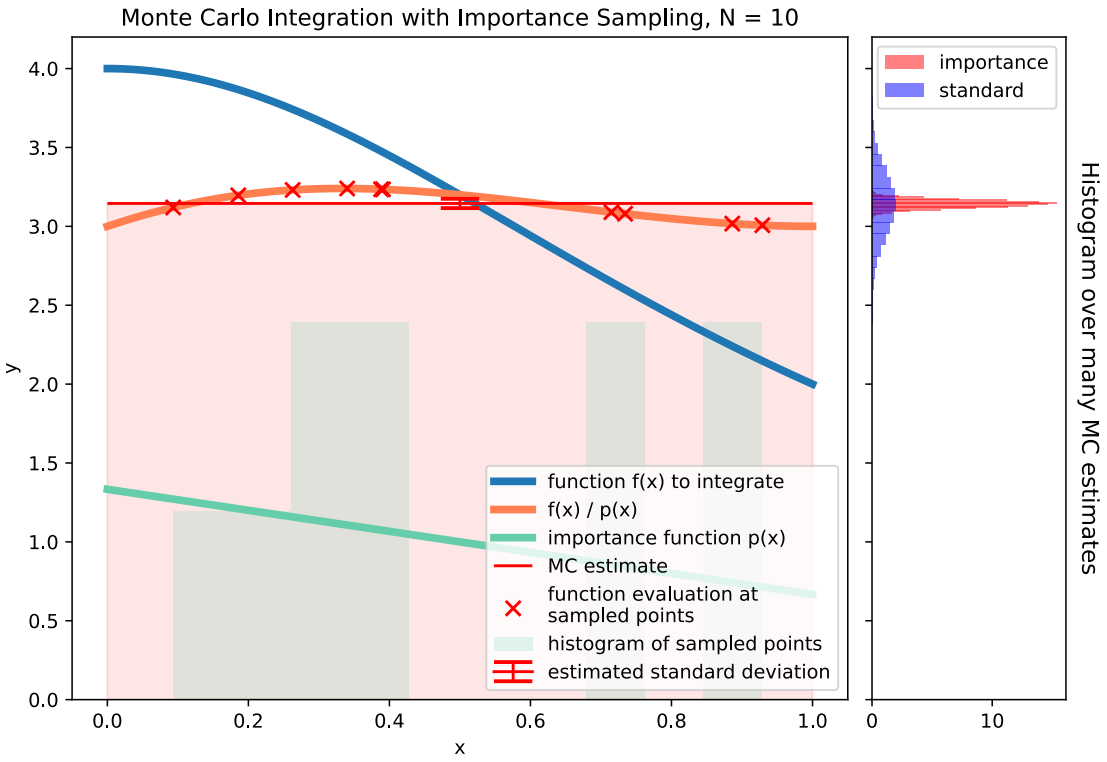



Figure 34: Illustration of Monte Carlo Integration with Importance Sampling for $N = 10$ samples.

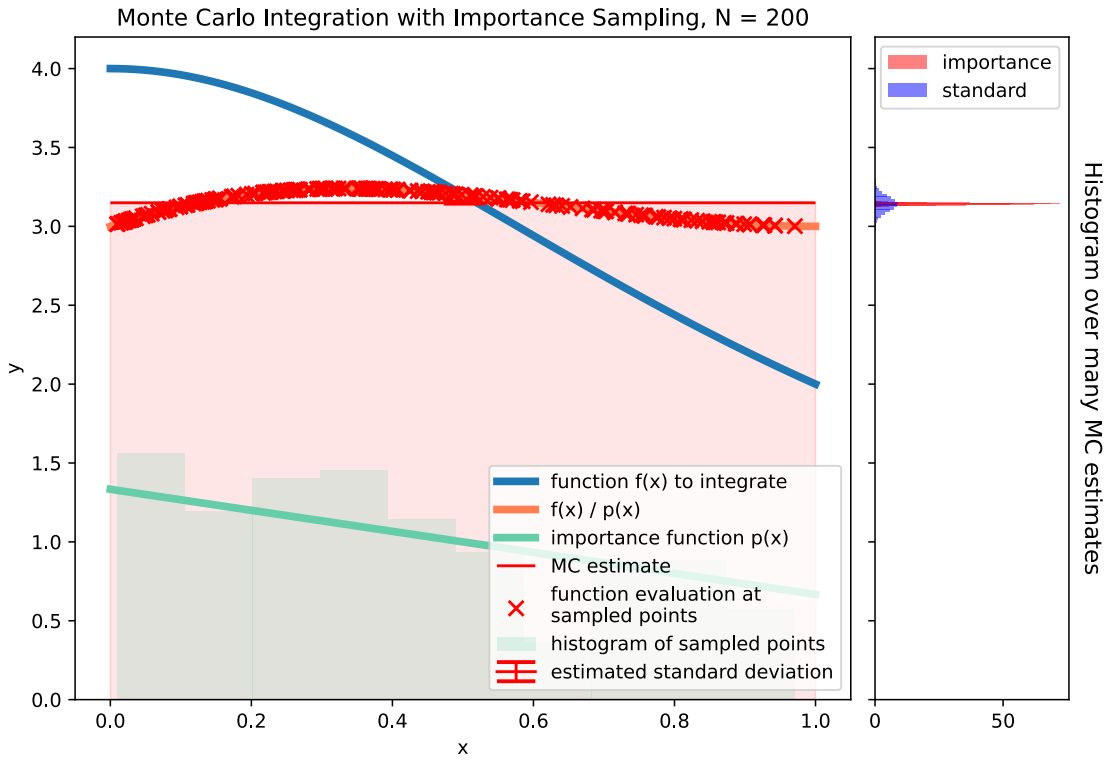



Figure 35: Illustration of Monte Carlo Integration with Importance Sampling for $N = 200$ samples.

# 7 Conclusion

# References


Baydin, Atilim Gunes et al. (2018). *Automatic differentiation in machine learning: a survey*. URL: https://arxiv.org/abs/1502.05767.

Blondel, Mathieu et al. (2022). *Efficient and Modular Implicit Differentiation*. URL: https://arxiv.org/abs/2105.15183.

Bradbury, James et al. (2018). *JAX: composable transformations of Python+NumPy programs*. Version 0.3.13. URL: http://github.com/google/jax.

Bradley, Andrew M. (2024). *PDE-constrained optimization and the adjoint method*. https://cs.stanford.edu/~ambrad/adjoint_tutorial.pdf. Originally published November 16, 2010. URL: https://cs.stanford.edu/~ambrad/adjoint_tutorial.pdf.

Bryant, Randal E and David Richard O'Hallaron (2011). *Computer systems: a programmer's perspective*. Prentice Hall.

Higham, Nicholas J. (2002). *Accuracy and Stability of Numerical Algorithms*. Second. Society for Industrial and Applied Mathematics. DOI: 10.1137/1.9780898718027. URL: https://epubs.siam.org/doi/abs/10.1137/1.9780898718027.

Kidger, Patrick (2021). »On Neural Differential Equations«. PhD thesis. University of Oxford.

Rackauckas, Christopher et al. (Nov. 2022). *SciML/SciMLBook: v1.1*. Version v1.1. DOI: 10.5281/zenodo.7347643. URL: https://doi.org/10.5281/zenodo.7347643.

Saad, Youcef and Martin H. Schultz (1986). »GMRES: A Generalized Minimal Residual Algorithm for Solving Nonsymmetric Linear Systems«. In: *SIAM Journal on Scientific and Statistical Computing* 7.3, pages 856–869. DOI: 10.1137/0907058. URL: https://doi.org/10.1137/0907058.

Strassen, Volker (1969). »Gaussian elimination is not optimal«. In: *Numerische mathematik* 13.4, pages 354–356.

Stumm, Philipp and Andrea Walther (2010). »New algorithms for optimal online checkpointing«. In: *SIAM Journal on Scientific Computing* 32.2, pages 836–854.

Thuerey, Nils et al. (2025). *Physics-based Deep Learning*. URL: https://arxiv.org/abs/2109.05237.

Um, Kiwon et al. (2021). *Solver-in-the-Loop: Learning from Differentiable Physics to Interact with Iterative PDE-Solvers*. URL: https://arxiv.org/abs/2007.00016.

Wang, Wujie et al. (Jan. 2023). »Learning pair potentials using differentiable simulations«. In: *The Journal of Chemical Physics* 158.4. DOI: 10.1063/5.0126475. URL: http://dx.doi.org/10.1063/5.0126475.

Wilcox, Lucas C. et al. (2013). *Discretely exact derivatives for hyperbolic PDE-constrained optimization problems discretized by the discontinuous Galerkin method*. URL: https://arxiv.org/abs/1311.6900.

# A Appendices